2주차 · ROS 2 기초 — 노드와 토픽

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-15

★ 이번 학기 **녹화 강의 영상**은 여기에 올라감 — 수업에서 놓친 부분은 영상으로 확인할 것

https://youtube.com/playlist?list=PLK_f1-krzJG8

- 매주 수업 뒤 그 주차 영상이 이 재생목록에 추가됨
- 아래 "참조 강의" 는 **지난 학기까지의 선수 과목**이고, 이 재생목록은 **이번 학기 본 수업** 녹화임

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 **본 과목 담당 교수가 직접 강의한 것**이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것

#	과목	수준	언어	영상	자료·코드
1	**제어공학** — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	**제어시스템설계** — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	**캡스톤디자인** — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	**제어공학특론** — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	**센서신호처리 및 융합** — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB·Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것. 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`

저장소	주소	받는 자리 (WSL)
강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트	**https://github.com/wkyouncnu/Capstone-Design**	`~/Capstone-Design`
ROS 2 예제 패키지 `usv_basics` — 2주차부터 쓰는 노드 코드	**https://github.com/wkyouncnu/usv_basics**	`~/capstone_ws/src/usv_basics`

언제	명령 (VS Code 의 WSL 창 터미널)	하는 일
처음 한 번	`cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git`	강의자료 전체를 받음 (약 400 MB)
처음 한 번	`mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git`	예제 패키지를 받음. 이어서 `cd ~/capstone_ws && colcon build --symlink-install`
매주 수업 전	`cd ~/Capstone-Design && git pull` · `cd ~/capstone_ws/src/usv_basics && git pull`	**바뀐 파일만** 받음. 다시 clone 하지 않음
무엇이 바뀌었는지	`git log --oneline -10` · `git show --stat HEAD`	교수가 갱신한 내역을 확인

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엶
- `usv_basics` 를 받은 뒤 새 노드가 생겼으면 `colcon build --symlink-install` 을 한 번 더 돌림
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → 강의자료는-한-번-받고-git-pull-로-갱신한다

- **과목**: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용**: ① ROS 2 설치 ② **VS Code 로 WSL 안의 코드 편집** ③ 두 프로그램이 서로 데이터를 주고받게 만들기 ④ **화면 도구(rqt · RViz2)로 눈으로 확인** ⑤ **중계 노드로 Simulink 와 연결해 turtlesim 을 목표 자세로 보내기**

★ 시작 전 확인

- 1주차 WSL 설치가 끝나 있어야 함
- `xeyes` 가 안 뜨는 학생은 **지금 손 들 것**. 이번 주차 설치를 해도 3주차에 막힘
- 저장공간 **20 GB 이상** 여유 확인: `df -h` 실행 후 `/` 행의 Avail 값 확인

이 주차를 마치면 할 수 있어야 하는 것

1. ROS 2가 **왜 필요한지**, ROS 1과 무엇이 다른지 설명
2. **노드 · 토픽 · 서비스 · 액션 · 파라미터**가 각각 무엇이고 언제 쓰는지 설명
3. **워크스페이스 · 패키지 · 노드**의 3층 구조를 설명

4. **VS Code 를 WSL 에 연결**해서 우분투 안의 코드를 편집·빌드·실행
5. **Windows 탐색기로 WSL 폴더**를 열고 파일을 주고받기
6. **turtlesim** 으로 네 가지 통신을 직접 조작
7. `ros2` 명령어로 실행 중인 노드와 토픽을 조사
8. **노드를 직접 작성**해서 데이터를 주고받기
9. **rqt_graph · Topic Monitor · rqt_console · RViz2** 를 띄워 상태를 눈으로 확인
10. **QoS 불일치**를 재현하고 원인을 진단
11. **표본화 주기**가 왜 중요한지 설명
12. MATLAB 에 없는 메시지를 **중계 노드**로 표준 형식에 옮겨 발행
13. Simulink 모델로 turtlesim 을 **목표 위치·선수각**까지 보내고 오프라인과 수치로 대조

준비물

항목	내용
환경	1주차에 만든 **WSL2 + Ubuntu 22.04** (`wsl -l -v` 로 확인)
저장공간	**20 GB 이상** — ROS 2 설치에 필요
편집기	**VS Code** (Windows 에 설치). 이번 주차 2-2절에서 함께 설치함
터미널	VS Code 통합 터미널을 기본으로 씀. `terminator` (1주차 2-6절)도 그대로 사용 가능
인터넷	패키지 내려받기. 학교 와이파이면 시간이 더 걸림
MATLAB (§2-11 만)	**R2024b** + Simulink + ROS Toolbox. 없으면 §2-11 은 4주차 뒤로 미룸

> **ℹ 이번 주차부터 편집기가 바뀜**
> 1주차에서는 `nano` 로 파일을 만들었음. 이번 주차부터는 **VS Code** 를 씀
> `nano` 는 저장(`Ctrl+O`)과 종료(`Ctrl+X`)를 외워야 하고, 파이썬 들여쓰기가 틀려도
> 알려 주지 않음. VS Code 는 오타·들여쓰기 오류를 **빨간 밑줄로 즉시 표시**함

> **⚠ 1주차 설치를 완료하지 못한 경우**
> 이번 주차 실습은 **전부 WSL 안에서** 이뤄짐. 설치부터 하면 따라올 수 없음
> 수업 시작 전에 조교에게 알릴 것

이 문서를 읽는 순서

> **✎ 이 주차는 분량이 많음 (PDF 90쪽 이상). 전부를 한 번에 읽을 필요는 없음**
> 설치 절은 **수업 전에**, 참고 절은 **필요할 때** 찾아봄. 수업 시간에는 가운데 줄만 따라감

언제	절	무엇을 얻는가
수업 전 — 설치	2-1 ROS 2 설치 · 2-2 VS Code	수업 시간에 설치로 막히지 않음
수업 중 — 개념	1-1 ~ 1-2 · 1-4 ~ 1-6	노드 · 토픽 · 메시지 · Domain ID · QoS 가 무엇인가
수업 중 — 손으로	2-3 첫 통신 → 2-4 turtlesim → 2-6 패키지 → 2-7 첫 노드 → 2-9 QoS 실험	이 주차의 뼈대. 순서대로 함
수업 중 — Simulink	2-10 중계 노드 → 2-11 목표 자세 제어	4주차 Simulink 연동의 예고편. 2-10 을 먼저 해야 2-11 이 돎
과제	1-7 표본화	과제 2(가상 IMU 표본화) 가 이 절에서 나옴
필요할 때 — 참고	1-3 워크스페이스 · 2-5 조사 명령어 · 2-8 rqt · RViz2	명령어가 기억나지 않을 때 돌아와 찾음

1부 · 이론

1-1. ROS 2는 무엇을 해결하는가

문제 상황

- 자율운항 보트 한 대에 붙는 프로그램들

```
GNSS 드라이버   IMU 드라이버   LiDAR 드라이버   카메라 드라이버
      |            |              |               |
      +------------+------+-------+---------------+
                           |
                    상태추정 (EKF)
                           |
          +----------------+----------------+
          |                |                |
     장애물 탐지       경로 계획          제어기
                                           |
                                      추진기 명령
```

- 이것을 **하나의 큰 프로그램**으로 만들면 생기는 문제

문제	결과
모듈이 서로 얽힘	LiDAR 처리가 느려지면 제어기까지 멈춤
부분 수정 불가	카메라 알고리즘 하나 바꾸려고 전체 재컴파일
언어 고정	제어는 MATLAB, 인지는 Python으로 짜고 싶은데 방법 없음
장애 전파	한 곳이 죽으면 배 전체가 죽음

ROS 2의 해법

- **프로그램을 독립된 조각(노드)으로 쪼개고, 그 사이 통신을 표준화**

문제	ROS 2의 해법
모듈 결합	노드를 별도 프로세스로 분리
언어 다양성	C++ · Python · **MATLAB** 이 같은 토픽으로 대화
데이터 형식	표준 메시지 타입 (`sensor_msgs/Imu` 등)
분산 실행	같은 네트워크의 다른 컴퓨터와도 자동 연결
재현성	`rosbag2` 로 모든 데이터를 기록·재생

> ℹ **본 과목에서 결정적인 지점**
>
> 4주차에 **Windows의 Simulink**와 **WSL의 Gazebo**가 서로 대화하게 됨.
> 그것을 가능하게 하는 것이 바로 이 구조임.

ROS 1과 ROS 2

> **ℹ 인터넷 예제가 ROS 1인지 ROS 2인지 구분하는 것이 실전 능력임**
> 검색으로 찾은 코드가 `rospy` · `catkin_make` · `roscore` 를 쓰면 **ROS 1 예제**임
> 그대로 붙여 넣으면 동작하지 않음. 아래 표의 왼쪽 칸이 곧 "ROS 1 판별 단어" 목록임

항목	ROS 1 (Noetic)	ROS 2 (Humble)
중앙 관리자	**`roscore` 를 먼저 띄워야 함**	**없음.** 노드끼리 DDS 로 자동 발견
통신 미들웨어	자체 TCPROS/UDPROS	**DDS** (산업 표준, `rmw` 로 교체 가능)
파이썬 라이브러리	`rospy`	**`rclpy`**
C++ 라이브러리	`roscpp`	**`rclcpp`**
빌드 도구	`catkin_make` · `catkin build`	**`colcon build`**
패키지 명세	`package.xml` format 2	`package.xml` **format 3**
노드 실행	`rosrun <패키지> <실행파일>`	**`ros2 run <패키지> <실행파일>`**
여러 노드 동시 실행	`roslaunch` , **XML 만**	**`ros2 launch` , Python · XML · YAML**
조사 명령	`rostopic` · `rosnode`	**`ros2 topic`** · **`ros2 node`** (하위 명령 방식)
파라미터	**중앙 파라미터 서버** 하나	**노드마다 각자** 보유
통신 품질(QoS)	없음 (사실상 TCP 고정)	**정책으로 선택** — §1-6
기록·재생	`rosbag` (`.bag`)	**`ros2 bag`** (`.db3` = SQLite)
실시간성	약함	지원 (DDS 계층에서)
운영체제	사실상 리눅스	리눅스 · Windows · macOS
지원 상태	**2025-05-31 지원 종료**	Humble 은 **2027년 5월까지**

- 본 과목에서 사용할 것: **ROS 2 Humble Hawksbill** (Ubuntu 22.04 용 장기 지원 버전)
- 출처 — ROS 1 Noetic 종료일은 Open Robotics 공지 (https://discourse.openrobotics.org/t/ros-noetic-end-of-life-may-31-2025/43160)

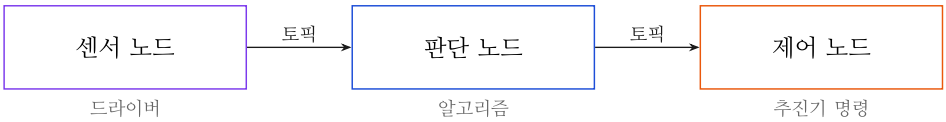
> **⚠ 명령어를 바꿔 쓰는 표**
> ROS 1 예제를 ROS 2 로 옮길 때 가장 자주 고치는 다섯 줄

ROS 1	ROS 2
`import rospy`	`import rclpy`
`rospy.init_node('x')`	`rclpy.init()` 후 `Node('x')`
`rospy.Publisher('t', String, queue_size=10)`	`node.create_publisher(String, 't', 10)`
`rospy.spin()`	`rclpy.spin(node)`
`rostopic echo /t`	`ros2 topic echo /t`

1-2. 노드와 통신 — 토픽 · 서비스 · 액션 · 파라미터

전체 그림 먼저

① 기본 단위는 노드 — 하나의 일을 하는 프로그램



② 노드끼리 잇는 방법은 네 가지

토픽 Topic

계속 흐르는 데이터 · 단방향 · 여럿이 구독
센서값 · 명령 — **본 과목의 90%**

서비스 Service

요청 → 응답 · 1회성 · 끝나면 닫힘
"지금 거북이 한 마리 더 만들어줘"

액션 Action

목표 → 피드백 → 결과 · 중간 취소 가능
"저 각도까지 돌아라"

파라미터 Parameter

노드의 설정값 · 실행 중에도 바꿀 수 있다
PID 게인 · 배경색

③ 그 밑을 받치는 것이 DDS

DDS — 노드끼리 서로 찾아 준다. `ROS_DOMAIN_ID` 로 구역을 나눈다

ROS 1 의 `roscore` 같은 중앙 관리자가 없다. 노드가 서로를 직접 찾는다

층	무엇인가	이번 주차에서
노드	실행되는 프로그램 하나	§2-7 에서 직접 만듦
토픽	계속 흐르는 데이터	§2-3 · §2-4 · §2-7
서비스	한 번 요청하고 한 번 응답	§2-4 turtlesim
액션	오래 걸리는 일 + 중간 보고	§2-4 turtlesim
파라미터	노드의 설정값	§2-4 turtlesim
DDS	노드끼리 서로 찾는 계층	§1-5 Domain ID

★ 본 과목의 90%는 토픽임

나머지 셋은 "이런 것이 있고, 언제 쓰는지" 를 알아 두는 수준으로 충분함
다만 **8주차 Stateflow 미션**의 상태 전환이 액션의 goal → feedback → result 와 같은 발상임 (ROS 액션을 쓰지는 않는다)
파라미터는 11주차 이후 직접 작성하는 ROS 노드에서 씀. Simulink 제어기의 게인은 `W0X_setup.m` 변수로 다룸

토픽 — 흘려보내고 잊는다



1. 발행자는 누가 듣는지 모른다. 구독자도 누가 보내는지 모른다
2. 한 토픽을 여러 노드가 동시에 구독할 수 있다
3. 토픽 이름 · 메시지 타입 · QoS 가 모두 맞아야 연결된다. 하나라도 다르면 오류 없이 조용히 끊긴다

용어

용어	뜻	비유
노드 (Node)	하나의 일을 하는 독립 프로그램	직원 한 명
토픽 (Topic)	데이터가 흐르는 이름 있는 통로	사내 게시판
발행 (Publish)	토픽에 데이터를 올림	게시판에 글 쓰기
구독 (Subscribe)	토픽에서 데이터를 받음	게시판 구독
메시지 (Message)	데이터의 형식	정해진 양식의 문서

핵심 성질 3가지

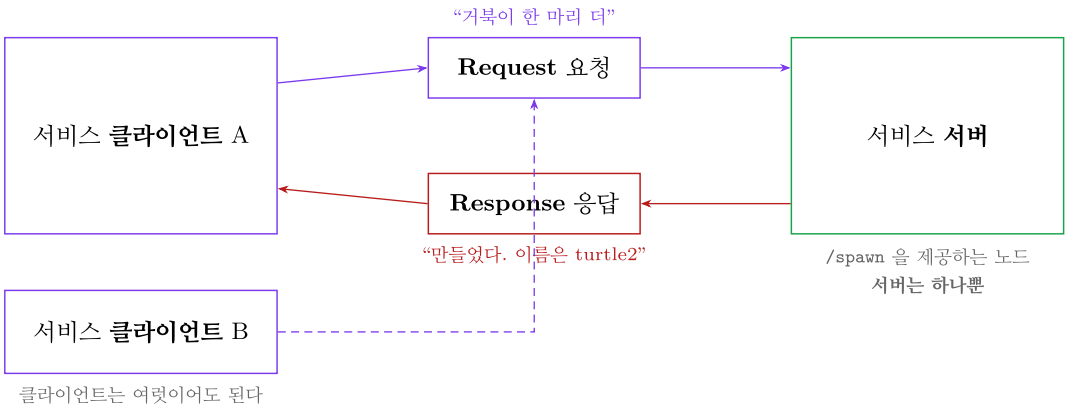
1. **발행자는 누가 듣는지 모름.** 구독자도 누가 보내는지 모름
2. **한 토픽을 여러 노드가 동시에 구독** 가능
3. **토픽 이름 + 메시지 타입 + QoS** 가 전부 맞아야 연결됨

어느 명령이 그림의 어디를 보여 주는가

명령	그림에서 보이는 것
`ros2 node list`	타원 — 살아 있는 노드 이름
`ros2 topic list`	사각형 — 오가는 토픽 이름
`ros2 topic info <토픽>`	그 토픽에 붙은 발행자·구독자 수와 QoS
`ros2 topic echo <토픽>`	흐르는 내용. **구독자를 하나 더 붙이는 것과 같음**
`rqt_graph`	그림 전체 (연결 관계)

- `echo` 가 구독자로 동작하기 때문에, 켜 두면 그래프에 노드가 하나 더 생김

서비스 — 부르고 답을 기다린다



토픽과 다른 점 — 클라이언트는 답이 올 때까지 **기다린다**. 서버가 없으면 그 자리에서 멈춘 채 대기한다. `ros2 service call` 이 응답 없이 멈춰 있으면 **서버 노드가 안 떠 있는 것**이다

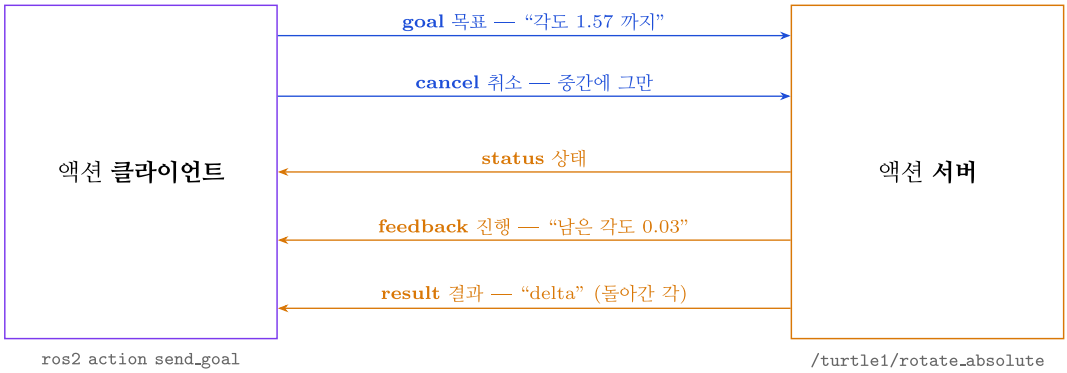
항목	토픽	서비스
방향	한 방향	**요청 ↔ 응답 양방향**
횟수	계속 흐름	**한 번 부르고 끝**
기다림	안 기다림	**답이 올 때까지 기다림**
개수	발행자·구독자 여럿 가능	**서버는 하나**, 클라이언트는 여럿

- 쓰는 곳 — "지금 이것 한 번만 해 줘"

예	서비스
거북이 한 마리 추가	`/spawn`
화면 지우기	`/clear`
상태 초기화	`/reset`
VRX 에서 배 위치 되돌리기	3주차 이후

- 실제 조작은 §2-4 에서 함

액션 — 맡기고 중간 보고를 받는다



토픽은 보내고 잊는다 · **서비스**는 한 번 묻고 한 번 답한다 · **액션**은 맡기고 **중간 보고**를 받으며 **취소**할 수 있다

- 서비스로 "저 지점까지 가라" 를 시키면 **도착할 때까지 아무 소식이 없음.** 취소도 못 함
- 액션은 그 문제를 풂

주고받는 것	뜻
goal	목표를 줌
feedback	진행 상황을 계속 받음
result	끝났을 때 결과를 받음
cancel	중간에 그만두게 함

- 쓰는 곳 — 이동, 도킹, 탐색처럼 **오래 걸리는 일**
- **8주차 Stateflow 미션**의 상태 전환이 goal → feedback → result 와 같은 발상임 (ROS 액션을 쓰지는 않는다)

파라미터 — 노드의 설정값

- 코드에 숫자를 박아 두면 바꿀 때마다 **다시 빌드**해야 함
- 파라미터로 빼 두면 **실행 중에** 바꿀 수 있음

명령	하는 일
`ros2 param list`	어떤 설정값이 있는가
`ros2 param get <노드> <이름>`	현재 값
`ros2 param set <노드> <이름> <값>`	값 변경
`ros2 param dump <노드>`	전체를 YAML 로 저장

- 11주차 이후 직접 작성하는 ROS 노드에서 씀. Simulink 제어기의 **게인**은 `W0X_setup.m` 변수로 다룸

> **네 가지를 한 문장으로**
> **토픽**은 방송, **서비스**는 전화, **액션**은 택배 배송 조회, **파라미터**는 설정 화면

1-3. 패키지와 워크스페이스 — ROS 2 코드가 사는 곳

3층 구조

- ROS 2 코드는 **세 겹**으로 담김. 이 구분을 못 하면 빌드가 계속 실패함

```
capstone_ws/                ← ① 워크스페이스 (작업 상자)
├── src/                      직접 작성한 코드만 여기에 둔다
│   └── usv_basics/         ← ② 패키지 (배포 단위)
│       ├── package.xml       이름·의존성·라이선스
│       ├── setup.py          실행파일 등록
│       └── usv_basics/
│           ├── simple_talker.py    ← ③ 노드 (실행 단위)
│           └── simple_listener.py  ← ③ 노드
├── build/                    colcon 이 만든 중간 산출물
├── install/                  colcon 이 만든 결과물. ros2 run 이 여기를 본다
└── log/                      빌드 기록
```

층	이름	무엇인가	만드는 명령
①	**워크스페이스**	패키지를 모아 한 번에 빌드하는 폴더	`mkdir -p ~/capstone_ws/src`
②	**패키지**	배포·재사용 단위. 이름이 곧 `ros2 run` 의 첫 인자	`ros2 pkg create`
③	**노드**	실제로 실행되는 프로그램 하나	파이썬 파일 작성

> ★ **`build/` · `install/` · `log/` 는 손으로 만들지 않음**
>
> `colcon build` 가 만듦. **지워도 다시 빌드하면 생김**
> 반대로 `src/` 안의 내용은 지우면 복구되지 않음

두 가지 빌드 타입

빌드 타입	언어	본 과목에서
`ament_python`	파이썬	**이번 주차에 쓸 것**
`ament_cmake`	C++	10주차 이후 참고용

`package.xml` — 패키지의 신분증

- `ros2 pkg create` 가 만들어 준 실제 파일

```
<package format="3">
  <name>usv_basics</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="cnu@todo.todo">cnu</maintainer>
  <license>Apache-2.0</license>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

태그	뜻
`<name>`	패키지 이름. **`ros2 run` 의 첫 인자와 같아야 함**
`<license>`	라이선스. 비우면 빌드 경고가 남
`<build_type>`	`ament_python` 인지 `ament_cmake` 인지
`<depend>`	이 패키지가 필요로 하는 다른 패키지 (직접 추가)

- `rclpy` 와 `std_msgs` 를 쓰므로 아래 두 줄을 `<license>` 아래에 넣어 두면 좋음

```
  <depend>rclpy</depend>
  <depend>std_msgs</depend>
```

> ℹ **지금은 없어도 빌드됨**
> 파이썬은 실행 시점에 `import` 하므로 `<depend>` 가 없어도 돌아감
> 그러나 남이 이 패키지를 받아 `rosdep` 으로 의존성을 깔 때 **빠진 것을 알 수 없음**
> 팀 저장소에 올릴 패키지에는 반드시 적음

`setup.py` — 어떤 파이썬 파일이 실행파일이 되는지

- `entry_points` 의 `console_scripts` 한 줄이 곧 `ros2 run` 으로 부를 이름

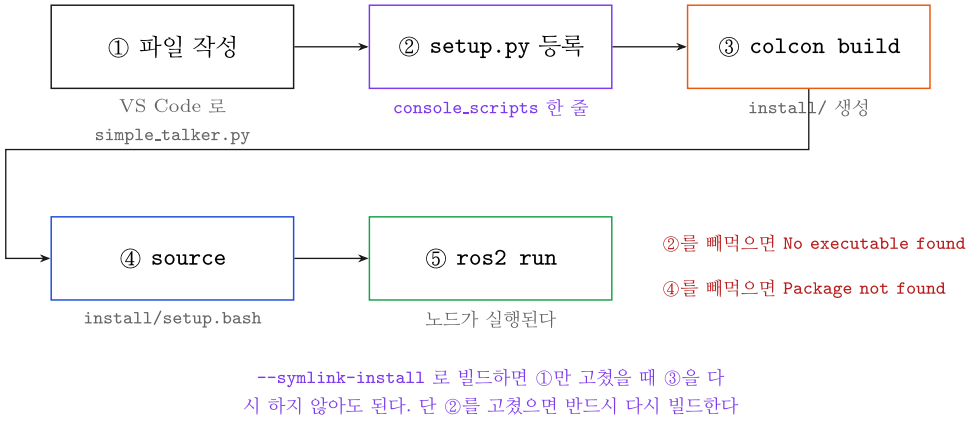
```
entry_points={
    'console_scripts': [
        'simple_talker = usv_basics.simple_talker:main',
    ],
},
```

조각	뜻
`simple_talker` (등호 왼쪽)	`ros2 run usv_basics simple_talker` 의 마지막 인자
`usv_basics.simple_talker`	패키지 폴더 안의 파이썬 파일 경로 (`.py` 없이)
`:main`	그 파일 안에서 호출할 함수 이름

> ⚠ **이 한 줄을 빼먹으면 `No executable found` 가 남**
> 파일을 아무리 잘 써도 `setup.py` 에 등록하지 않으면 `ros2 run` 이 찾지 못함
> **매 학기 가장 많이 나오는 오류**임

왜 `source install/setup.bash` 를 해야 하는가

- `ros2 run` 은 **환경변수에 등록된 경로**에서만 패키지를 찾음



단계	빼먹었을 때 나오는 메시지
② `setup.py` 등록	`No executable found`
③ `colcon build`	고친 내용이 반영되지 않음
④ `source install/setup.bash`	`Package 'usv_basics' not found`

명령	무엇을 등록하는가
`source /opt/ros/humble/setup.bash`	ROS 2 가 기본 제공하는 패키지 (`demo_nodes_cpp` 등)
`source ~/capstone_ws/install/setup.bash`	**학습자가 만든 패키지**

- 두 줄을 `~/.bashrc` 에 넣어 두면 터미널을 새로 열 때마다 자동 적용됨
- 새 터미널에서 직접 만든 패키지가 안 보이면 **두 번째 줄을 안 한 것**임

1-4. 메시지 타입

자주 쓰는 것

타입	내용	VRX에서
`std_msgs/Float64`	실수 하나	추력 명령
`sensor_msgs/NavSatFix`	위도 · 경도 · 고도	GPS
`sensor_msgs/Imu`	자세 · 각속도 · 가속도	IMU
`sensor_msgs/PointCloud2`	3D 점 덩어리	LiDAR
`sensor_msgs/LaserScan`	2D 거리 배열	LiDAR 축약
`sensor_msgs/Image`	영상	카메라
`nav_msgs/Odometry`	위치 + 자세 + 속도	상태추정 결과

- 구조를 확인하는 명령어

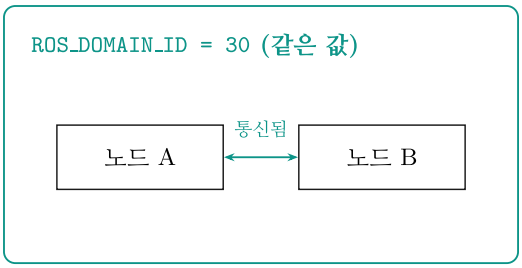
```
ros2 interface show sensor_msgs/msg/Imu
```

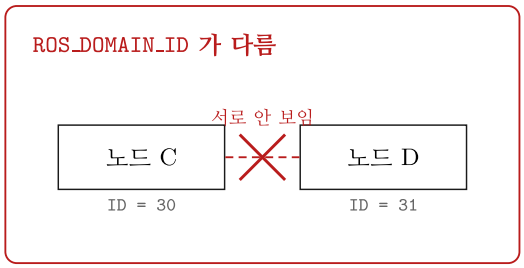
Header — 모든 센서 메시지의 앞부분

```
std_msgs/Header header
  builtin_interfaces/Time stamp    # 이 데이터를 측정한 시각
  string frame_id                  # 어느 좌표계 기준인가
```

> ⚠ **`stamp` 와 `frame_id` 는 필수 항목임**
>
> - `stamp` 가 없으면 GPS(20 Hz)와 IMU(100 Hz)를 **융합할 수 없음**.
> 어느 시점끼리 짝지어야 하는지 알 수 없기 때문
> - `frame_id` 가 없으면 LiDAR가 본 장애물이 **배 기준인지 지구 기준인지** 알 수 없음
> - 3주차 TF2에서 다시 나옴

1-5. DDS 와 Domain ID





1. 켜지면 자기 존재를 방송한다
2. 같은 도메인의 노드가 응답한다
3. 이름 · 타입 · QoS 가 맞으면 연결한다

roscore 같은 중앙 서버가 없다

증상 — 오류 메시지가 **전혀 없다**.
`ros2 topic list` 에 `/parameter_events` · `/rosout` 만 보인다.
강의실에서 서로 간섭하지 않게 하는 장치

```
export ROS_DOMAIN_ID=30
```

터미널마다. 안 하면 기본값 0. 범위는 0~101

문제 상황

- ROS 2에는 중앙 관리자가 없음 → 노드들이 네트워크에서 **서로를 자동으로 찾음**
- 실습실에서 20명이 동시에 작업하면 → **모두의 노드가 서로 보임**
- 결과: 옆 사람의 추력 명령이 자기 배를 움직임

해결 — `ROS_DOMAIN_ID`

- 같은 번호를 가진 노드끼리만 통신
- 범위: `0` ~ `101`

> **★ 본 과목의 규칙**
>
> **팀 번호를 Domain ID로 사용.** 1팀 → 1, 2팀 → 2 ...

- 설정 방법 (한 번만 하면 됨)

```
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
source ~/.bashrc
```

- 확인

```
echo $ROS_DOMAIN_ID
```

- 정상 출력: `7` (자기 팀 번호)

> **ℹ 4주차 복선**
>
> MATLAB과 연동할 때 **양쪽 Domain ID가 같아야** 통신됨. 지금 기억해 둘 것.

1-6. QoS — 통신 품질 정책

무엇인가

- "이 데이터를 어떻게 전달할지" 선택하는 설정
- ROS 1에는 없던 개념

두 가지 핵심 항목

① Reliability (신뢰성)

정책	동작	적합한 데이터
`RELIABLE`	도착 보장. 실패하면 재전송	명령, 목표점
`BEST_EFFORT`	빠르게 보내고 잊음. 유실 허용	**고빈도 센서** (카메라, LiDAR)

② Durability (지속성)

정책	동작
`VOLATILE`	늦게 들어온 구독자는 과거 데이터를 못 받음
`TRANSIENT_LOCAL`	늦게 들어와도 마지막 값을 받음 (지도, 정적 TF)

QoS 불일치 — 본 과목에서 가장 혼동하기 쉬운 부분

발행자	구독자	결과
RELIABLE	RELIABLE	연결
RELIABLE	BEST_EFFORT	연결
BEST_EFFORT	**RELIABLE**	**연결 안 됨**

- 규칙: **구독자의 요구가 발행자가 제공하는 것보다 엄격하면 실패**

> **✕ 프로그램은 죽지 않음. 조용히 아무것도 안 받음**
> - `ros2 topic list` → 토픽이 **보임**
> - `ros2 topic hz` → 데이터가 **흐른다고 나옴**
> - **구독 노드만** 콜백이 한 번도 안 불림
> - 종료 코드도 정상이고 예외도 없음. 자기 콜백 코드를 의심하며 시간을 버리게 됨

- 다만 Humble 은 **경고 한 줄**을 찍어 줌. 이 문장을 알아보는 것이 이번 절의 목표
- 아래는 §2-9 실습에서 실제로 받은 출력

```
[WARN] [1788862828.210407869] [qos_test_sub]: New publisher discovered on topic 'qos_topic',
offering incompatible QoS. No messages will be received from it.
Last incompatible policy: RELIABILITY
```

- 발행자 쪽에도 짝이 되는 경고가 뜸

```
[WARN] [1788862828.210020463] [qos_test_pub]: New subscription discovered on topic 'qos_topic',
requesting incompatible QoS. No messages will be sent to it.
Last incompatible policy: RELIABILITY
```

읽는 법	뜻
`incompatible QoS`	QoS 가 맞지 않음
`No messages will be received`	**연결 자체가 안 됐음.** 유실이 아니라 0건
`Last incompatible policy: RELIABILITY`	어긋난 항목이 **Reliability** 라는 뜻

> **⚠ 이 경고는 터미널을 위로 올려야 보임**
> 발행자·구독자가 계속 `[INFO]` 를 찍으면 경고가 **화면 위로 밀려 올라감**
> 노드를 켠 **첫 화면**을 확인하거나, `rqt_console` 에서 Severity 를 `Warn` 으로 걸러 볼 것.

- 진단 명령어 (한 줄)

```
ros2 topic info /토픽이름 --verbose
```

- 이번 주차 실습에서 **일부러 재현해 봄**

1-7. 표본화 — 왜 신호처리를 알아야 하는가

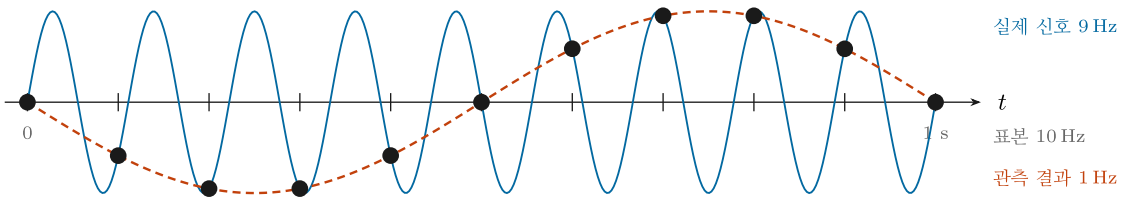
센서는 연속 신호를 잘라서 준다

센서	주파수	VRX (설계값, 10주차 2-3-7)
IMU	100~200 Hz	100 Hz
GNSS	1~20 Hz	20 Hz
LiDAR	10~20 Hz	10 Hz
카메라	15~30 Hz	30 Hz
제어기	10~100 Hz	설계자가 정함

표본화 정리 (Nyquist)

- 최대 주파수 f_{max} 인 신호를 잃지 않으려면
- 표본화 주파수가 $f_s > 2f_{max}$ 여야 함

에일리어싱 — 조건을 어기면



검은 점은 두 곡선 위에 **동시에** 놓인다. 표본값만 보면 9 Hz 인지 1 Hz 인지 구분할 방법이 없다

나이퀴스트 조건 — 표본화 주파수는 신호 최고 주파수의 **2배를 넘어야** 한다. $f_s = 10$ Hz 로 9 Hz 를 보려면 최소 18 Hz 가 필요하다

- 위 그림 읽는 법

요소	의미
파란 촘촘한 파형	실제 신호 9 Hz
검은 점	0.1초마다 읽은 값 (10 Hz 표본화)
주황 점선	표본값만으로 **보이는** 신호 — 1 Hz

- **결론**: 9 Hz 신호를 10 Hz로 읽으면 **존재하지 않는 1 Hz 신호**가 보임

배에서 실제로 일어나는 일

1. 파랑에 의한 선체 진동이 IMU에 실려 옴
2. 표본화 주기가 안 맞으면 **없는 저주파 흔들림**이 관측됨
3. 제어기가 그것을 쫓아가려 함
4. 배가 진동함

대책

- 표본화 **전에** 저역통과 필터로 고주파를 자름
- 또는 충분히 빠르게 표본화한 뒤 디지털 필터를 거쳐 다운샘플링
- → 이번 주차 과제가 이것

지연 (Latency)

```
물리량 발생 → 센서 측정 → 드라이버 → 전송 → 수신 노드 → 처리 → 명령
  t=0         t=2ms      t=5ms    t=8ms   t=10ms   t=15ms  t=16ms
```

- 제어에서 지연은 **위상 지연**이 되어 시스템을 불안정하게 만듦
- 그래서 `header.stamp` 는 **수신 시각이 아니라 측정 시각**으로 채워야 함

2부 · 실습

2-1. ROS 2 Humble 설치

> ℹ **소요 시간 약 20~40분**
> 다운로드가 큼. 중간에 노트북을 절전 모드로 두지 말 것.

1단계 — 언어 설정

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

- 확인

```
locale
```

- 정상 출력에 `LANG=en_US.UTF-8` 이 보이면 됨

2단계 — ROS 저장소 등록

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
```

- 중간에 `Press [ENTER] to continue` 가 나오면 `Enter`

```
sudo apt update && sudo apt install -y curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
     -o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] ht
tp://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/
apt/sources.list.d/ros2.list > /dev/null
```

> ⚠ **위 명령은 한 줄임**
> 문서에서 복사할 때 줄이 나뉘지 않도록 주의. 회색 상자를 통째로 복사할 것.

3단계 — 설치

```
sudo apt update
sudo apt upgrade -y
```

```
sudo apt install -y ros-humble-desktop
```

```
sudo apt install -y python3-colcon-common-extensions python3-rosdep
```

4단계 — 자동 적용 설정 (매번 터미널에서 다시 치지 않도록)

> ★ **이 단계를 건너뛰면 터미널을 열 때마다 `source` 를 쳐야 함**
> 새 터미널에서 `ros2` 를 치면 `bash: ros2: command not found` 가 남
> 매 학기 가장 자주 나오는 질문이 이것임

- `~/.bashrc` 는 **터미널을 열 때마다 자동으로 실행되는 파일**임. 여기에 넣어 두면 됨

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
```

> ★ `7` **을 자기 팀 번호로 바꿀 것**

- 지금 열려 있는 터미널에도 즉시 반영하려면

```
source ~/.bashrc
```

제대로 들어갔는지 확인한다

```
tail -3 ~/.bashrc
```

- 정상 출력 (기준 환경 실측) — 마지막 두 줄이 방금 넣은 것

```
fi
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=7
```

```
printenv ROS_DISTRO ROS_VERSION ROS_DOMAIN_ID
```

- 정상 출력

```
humble
2
7
```

> ⚠ `>>` **를** `>` **로 잘못 치면** `.bashrc` **가 통째로 날아감**
> `>>` 는 **덧붙이기**, `>` 는 **덮어쓰기**임. 화살표 개수를 반드시 확인할 것
> 날렸다면 `cp /etc/skel/.bashrc ~/.bashrc` 로 기본값을 복구한 뒤 두 줄을 다시 넣음

> ℹ **같은 줄이 두 번 들어가면**
> 명령을 두 번 실행했다면 `.bashrc` 에 같은 줄이 두 개 생김
> 동작에는 문제가 없지만 지저분하므로 VS Code 로 열어 지움
>
> ```
> code ~/.bashrc
> ```
>
> - 중복 확인 — 각각 **1** 이 나와야 함
>
> ```
> grep -c "opt/ros/humble/setup.bash" ~/.bashrc
> grep -c "ROS_DOMAIN_ID" ~/.bashrc
> ```

- 워크스페이스 줄(`source ~/capstone_ws/install/setup.bash`)은 **첫 빌드 뒤** §2-6 에서 넣음
  - 빌드 전에 넣으면 새 터미널마다 `No such file or directory` 가 찍힘

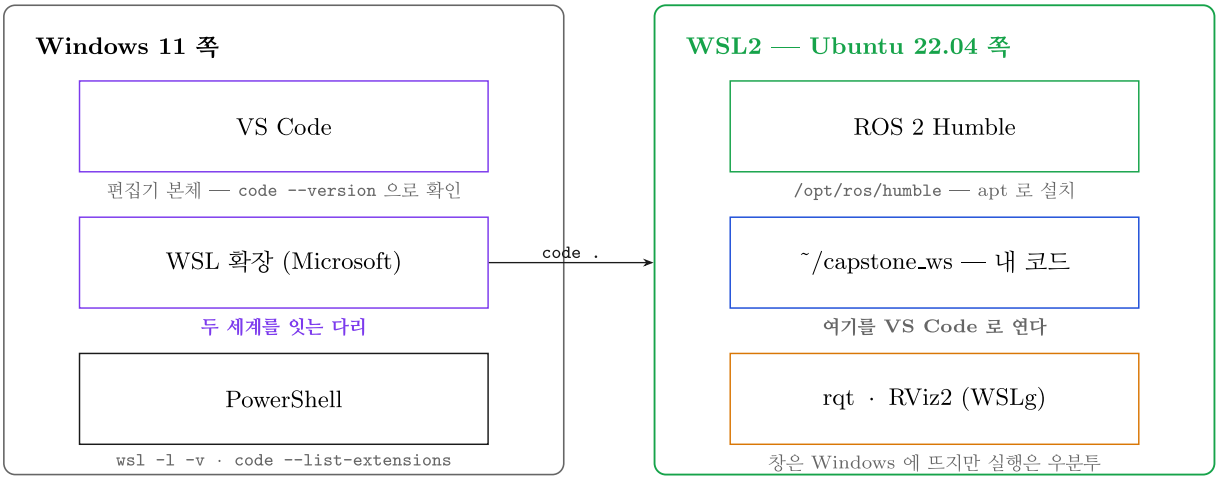
5단계 — rosdep 초기화

```
sudo rosdep init
rosdep update
```

- `sudo rosdep init` 에서 "already exists" 오류가 나면 무시하고 진행

2-2. VS Code 설치와 WSL 연결

> ★ **무엇이 어디에 설치되는지 먼저 이해할 것**
>
> - **VS Code → Windows** 에 설치됨
> - **ROS 2 · 학습자 코드 → WSL 안(Ubuntu)** 에 있음
> - 둘을 잇는 것이 **WSL 확장**임. 이 확장이 없으면 Windows 쪽 파일만 편집하게 됨



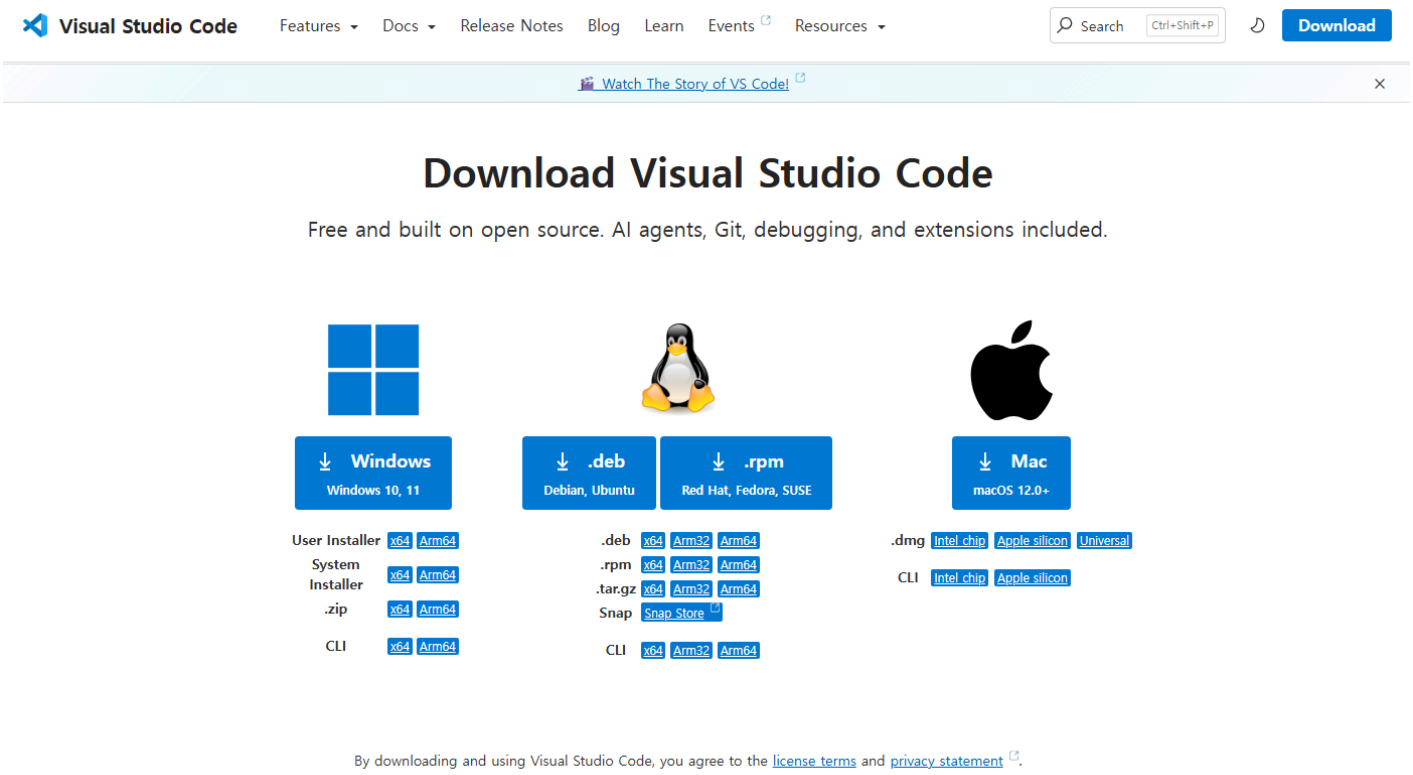
왼쪽 아래에 `WSL: Ubuntu-22.04` 가 보이지 않으면 Windows 쪽 파일을 고치고 있는 것이다. 그 상태로 저장하면 `colcon build` 에 아무것도 반영되지 않는다

그림에서	확인 명령
Windows 쪽 VS Code	PowerShell 에서 `code --version`
WSL 확장	PowerShell 에서 `code --list-extensions`
WSL 쪽 ROS 2	우분투에서 `printenv ROS_DISTRO` → `humble`
WSL 쪽 학습자 코드	우분투에서 `ls ~/capstone_ws`

- 앞 절의 `apt install` 이 도는 동안 **동시에 진행해도 됨**. 서로 방해하지 않음

1단계 — VS Code 내려받기

1. 브라우저에서 https://code.visualstudio.com/download 접속



2. 왼쪽 **Windows** 칸의 파란 버튼(`Windows / Windows 10, 11`) 클릭
3. 받아진 `VSCodeUserSetup-x64-*.exe` 실행

화면의 위치	무엇을 고르는가
왼쪽 큰 파란 버튼 **Windows**	**이것을 누름.** 기본이 User Installer 임
그 아래 `User Installer` / `x64`	직접 고를 때 씀
가운데 펭귄(리눅스) · 오른쪽 사과(맥)	**누르지 않음.** 우분투 안에는 설치하지 않음

> ✎ **User Installer 를 씀**
> 관리자 권한이 필요 없어 실습실 PC 에서도 통과함
> System Installer 는 관리자 권한을 요구해 막히는 경우가 있음

- 설치 중 **"추가 작업 선택"** 화면에서 아래를 **반드시 체크**

체크할 항목	이유
`PATH에 추가`	터미널에서 `code` 명령을 쓸 수 있음
`Code로 열기` — 파일 상황에 맞는 메뉴	우클릭으로 파일을 열 수 있음
`Code로 열기` — 디렉터리 상황에 맞는 메뉴	폴더째로 열 수 있음

- 설치 확인 — **PowerShell** 에서 실행

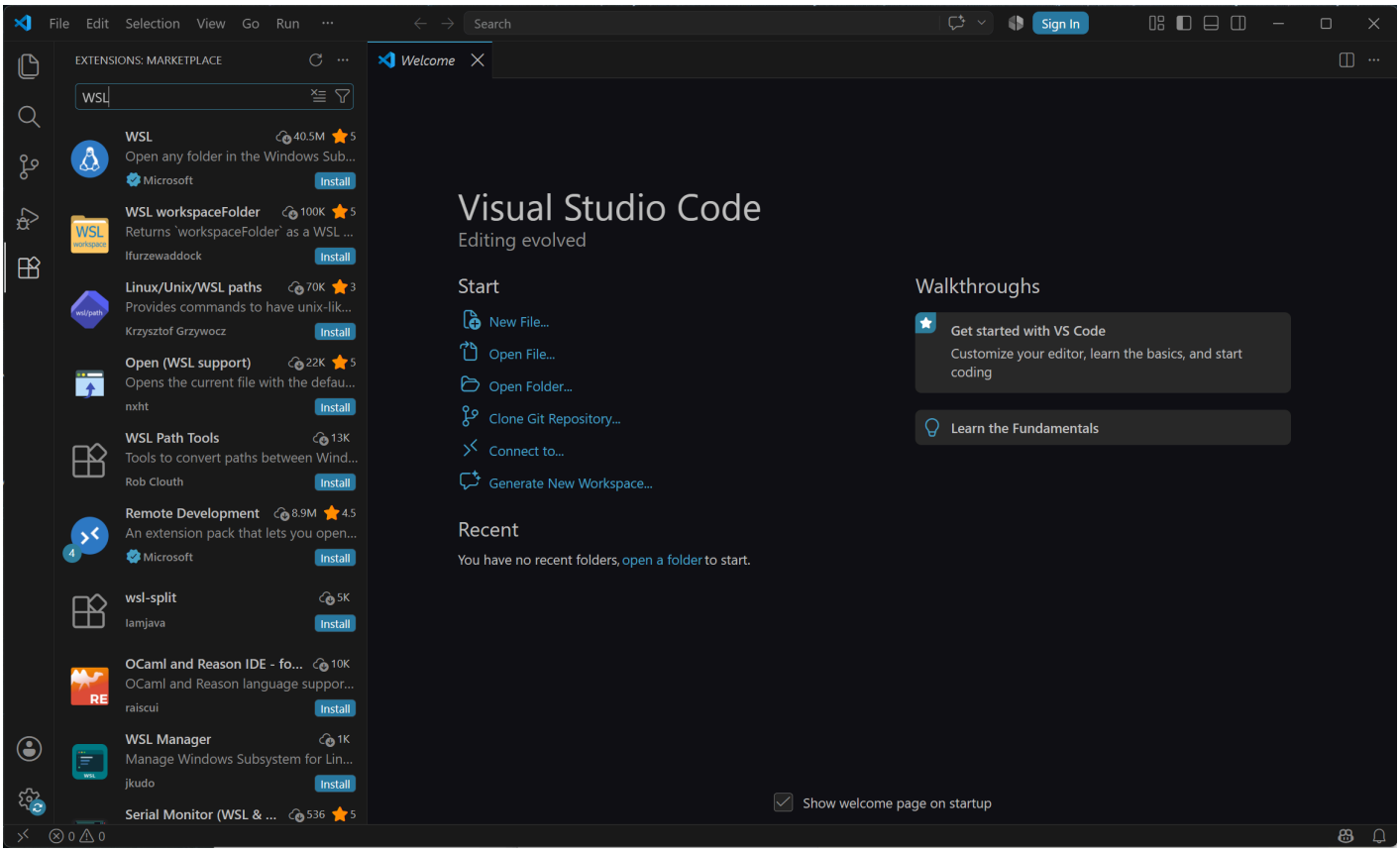
```
code --version
```

- 정상 출력 (기준 환경 실측)

```
1.132.0
df53daabb18cd157bdb08c7f01c34df936cf12f4
x64
```

2단계 — WSL 확장 설치

1. VS Code 실행
2. 왼쪽 세로 막대(활동 표시줄)에서 **블록 4개 아이콘**(확장) 클릭 — 단축키 `Ctrl + Shift + X`
3. 검색창에 `WSL` 입력
4. 맨 위 **WSL** (게시자: **Microsoft**) 의 **Install** 클릭



> ⚠ **비슷한 이름이 여럿 나옴**
> 게시자가 **Microsoft** 이고 설치 수가 가장 많은 것(4천만 회 이상)이 정답
> `WSL workspaceFolder` · `Linux/Unix/WSL paths` · `wsl-split` 은 **다른 확장**임

- 설치 확인 — PowerShell 에서

```
code --list-extensions
```

- 정상 출력

```
ms-vscode-remote.remote-wsl
```

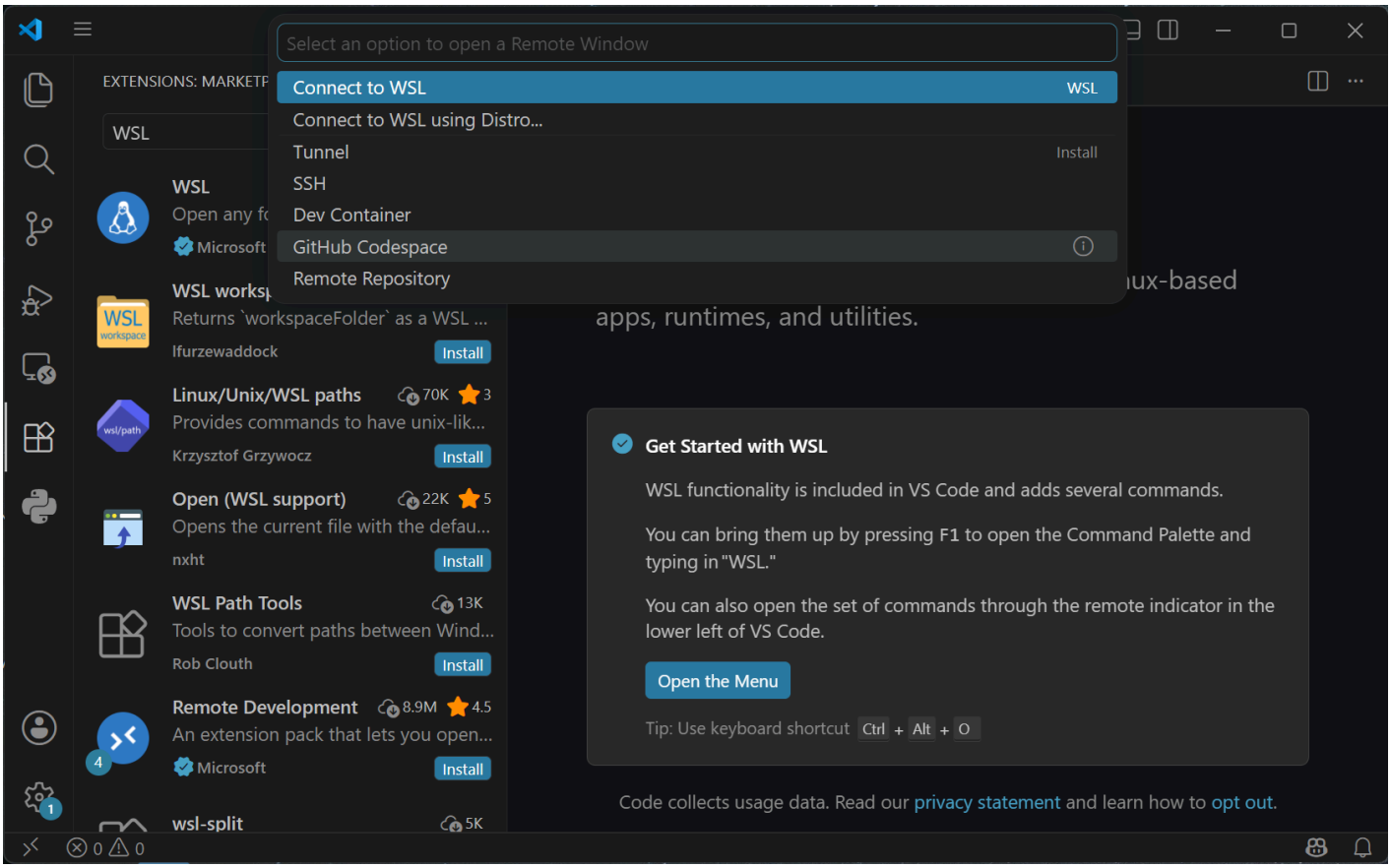
3단계 — WSL 에 연결

- **방법 1 — 우분투 터미널에서 (권장)**

```
mkdir -p ~/capstone_ws/src
cd ~/capstone_ws
code .
```

- 빈 폴더(`src` 하나)가 열리는 것이 정상. `build` `install` `log` 는 §2-6 빌드 뒤에 생김
- 처음 실행하면 VS Code 서버가 WSL 안에 자동 설치됨 (1~2분)
- **방법 2 — VS Code 안에서**

1. 창 **왼쪽 맨 아래 모서리**의 파란 `><` 버튼 클릭 (단축키 `Ctrl + Alt + O`)
2. 위에 목록이 내려옴

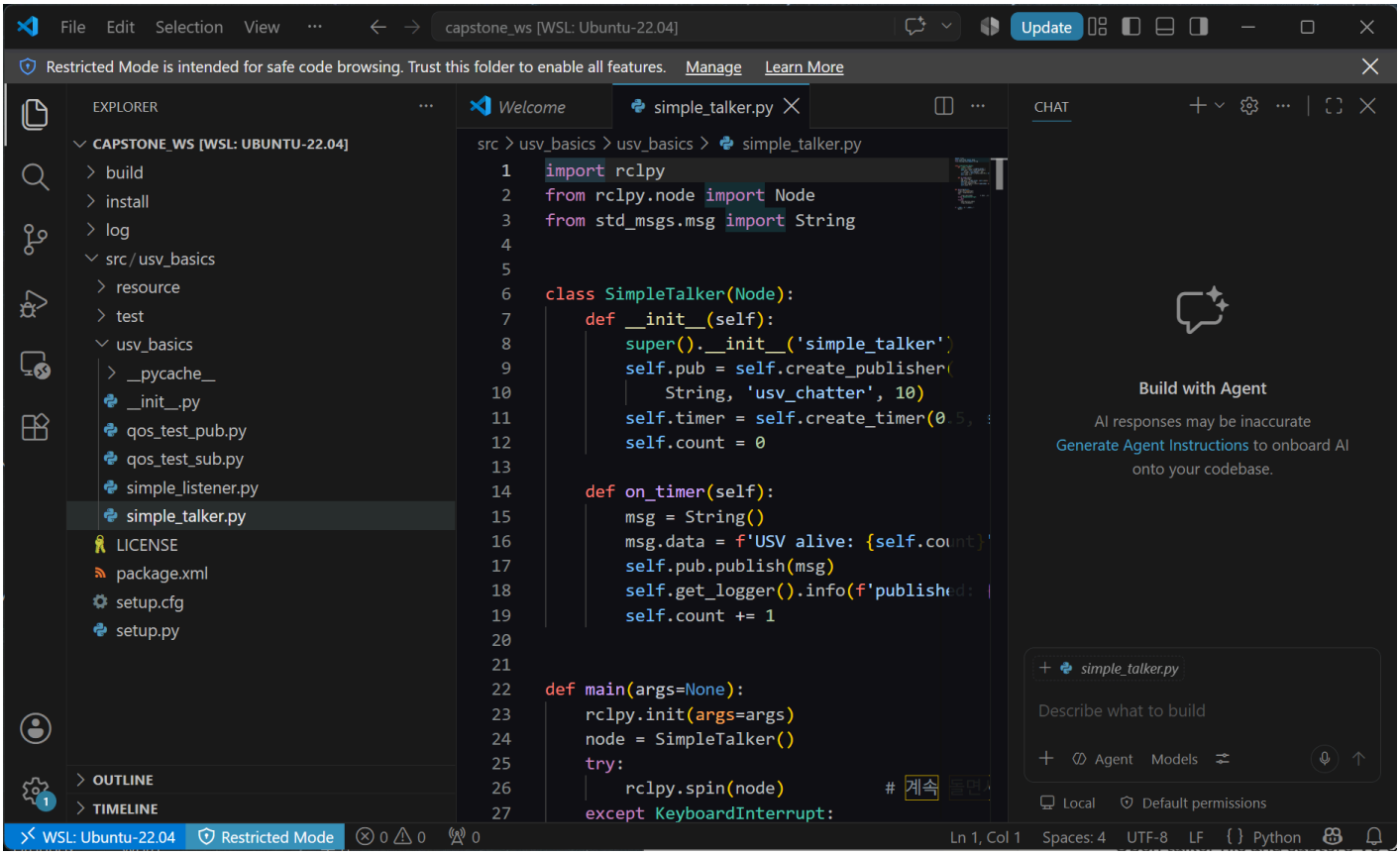


3. 맨 위 **Connect to WSL** 선택
   - 배포판이 여러 개면 **Connect to WSL using Distro...** 에서 `Ubuntu-22.04` 를 고름
   - `Tunnel` · `SSH` · `Dev Container` 는 **이번 과목에서 쓰지 않음**
4. 새 창이 뜨면 **File → Open Folder** 로 `/home/사용자명/capstone_ws` 를 엶

- 연결에 성공하면 **왼쪽 아래**가 이렇게 바뀜

```
WSL: Ubuntu-22.04
```

4단계 — 연결된 화면 읽는 법



#	화면의 위치	무엇인가
1	**왼쪽 아래 파란 칸** `WSL: Ubuntu-22.04`	**가장 중요.** 우분투 안을 편집 중이라는 표시
2	왼쪽 세로 막대	활동 표시줄 — 탐색기 · 검색 · 소스 제어 · 실행 · 확장
3	왼쪽 넓은 칸	**탐색기.** 그림은 §2-6 빌드 뒤 화면이라 `build` `install` `log` `src` 가 보임
4	가운데	**편집기.** 파이썬 문법이 색으로 구분됨
5	맨 아래 오른쪽	줄·열 번호, 인코딩(`UTF-8`), 줄바꿈(`LF`), 언어(`Python`)

✕ 왼쪽 아래 표시가 없으면 Windows 쪽 파일을 고치고 있는 것임

그 상태로 ROS 코드를 고치면 **아무리 저장해도 빌드에 반영되지 않음**
매 학기 가장 많이 헤매는 지점임. **파일을 고치기 전에 항상 이 칸을 봄**

⚠ 위쪽에 노란 띠로 "Restricted Mode" 가 뜨면

VS Code 가 **처음 여는 폴더를 신뢰하지 않는 상태**임. 이 상태에서는 작업 실행이 막힘
띠의 **Manage** → **Trust** 를 눌러 신뢰하도록 바꿈. 직접 만든 폴더일 때만 풂

ℹ 상태 표시줄의 `LF` / `CRLF`

리눅스 스크립트가 `CRLF` 로 저장되면 WSL 에서 실행되지 않음
증상은 `/bin/bash^M: bad interpreter` . 상태 표시줄에서 `LF` 로 바꾸면 해결됨

5단계 — WSL 쪽 확장 설치

> **★ WSL 에 연결한 상태에서 설치해야 WSL 쪽에 깔림**
> Windows 쪽에만 깔면 파이썬 자동완성이 동작하지 않음
> 확장 이름 옆에 **"Install in WSL: Ubuntu-22.04"** 버튼이 보이면 그것을 누름

확장	게시자	용도
Python	Microsoft	문법 검사 · 자동완성 · 들여쓰기 오류 표시
XML	Red Hat	3주차 이후 URDF · Xacro · SDF 편집
YAML	Red Hat	설정 파일 편집

- 설치 확인 — PowerShell 에서

```
code --remote wsl+Ubuntu-22.04 --list-extensions
```

- 정상 출력 (Python 확장은 부속 확장 3개를 함께 설치한다)

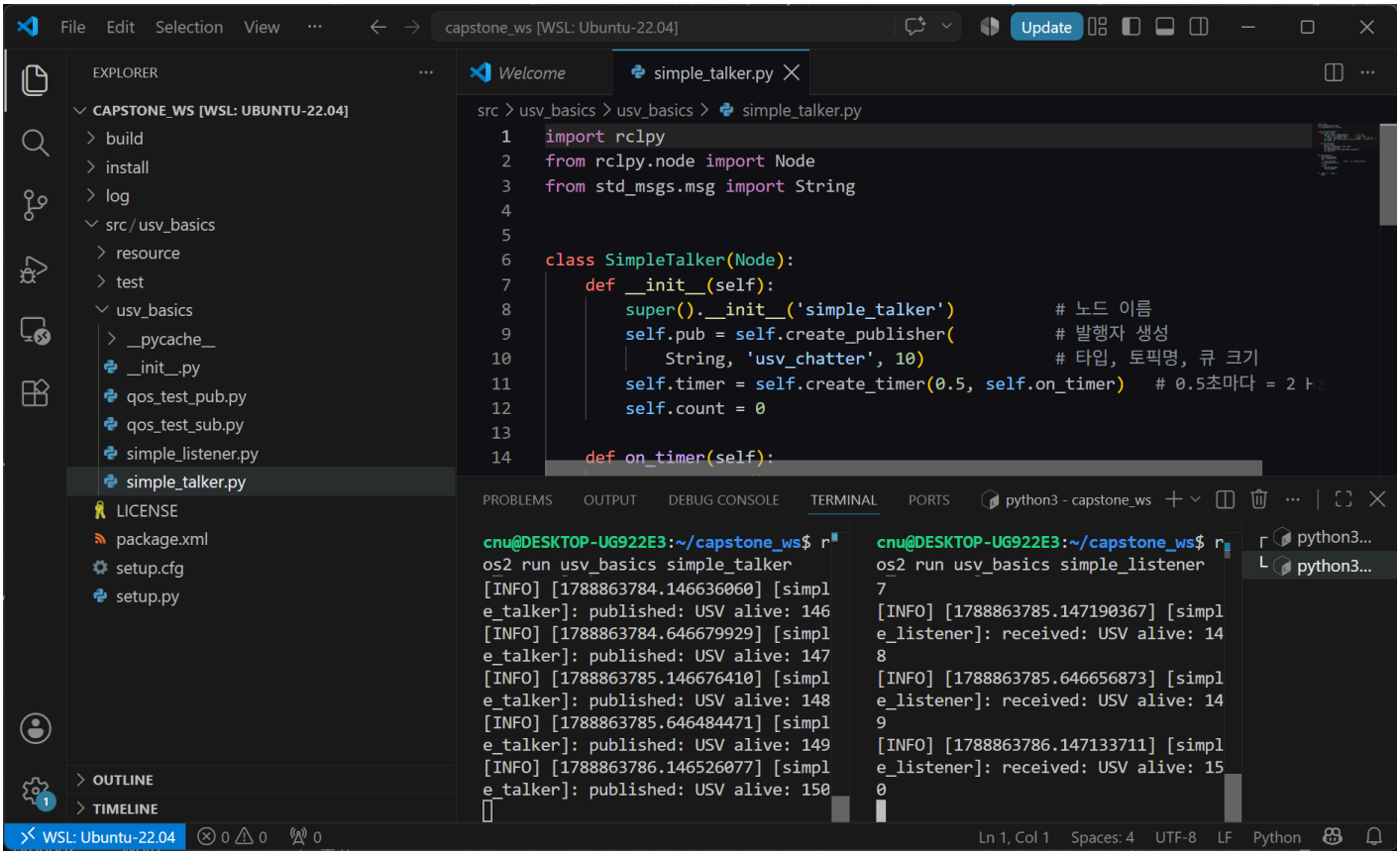
```
ms-python.debugpy
ms-python.python
ms-python.vscode-pylance
ms-python.vscode-python-envs
ms-vscode-remote.remote-wsl
```

6단계 — 통합 터미널 열기

- 단축키 `Ctrl + ``` (백틱, `Esc` 아래 키) 또는 상단 메뉴 **Terminal → New Terminal**
- 프롬프트가 아래 형태면 **우분투 안의 터미널**임

```
cnu@DESKTOP-XXXXXX:~/capstone_ws$
```

- 터미널을 **좌우로 나누는** 버튼이 터미널 패널 오른쪽 위에 있음 (네모 두 개 아이콘)
- 이 분할이 `terminator` 의 `Ctrl + Shift + E` 를 대신함



- 위 화면은 §2-7 을 끝낸 뒤의 실제 모습. 이번 주차 실습의 **목표 화면**

화면에서 보이는 것	뜻
왼쪽 터미널 `published: USV alive: 146`	발행자가 2 Hz 로 보내는 중
오른쪽 터미널 `received: USV alive: 148`	구독자가 같은 값을 받는 중
두 숫자가 나란히 올라감	**연결 성공**

> ✎ **편집기와 터미널을 한 화면에 두는 것이 이번 주차의 요령임**
> 코드를 고치고(`Ctrl + S`), 아래 터미널에서 바로 빌드·실행함
> 창을 오가지 않으므로 "어느 창에서 뭘 쳤더라" 가 사라짐

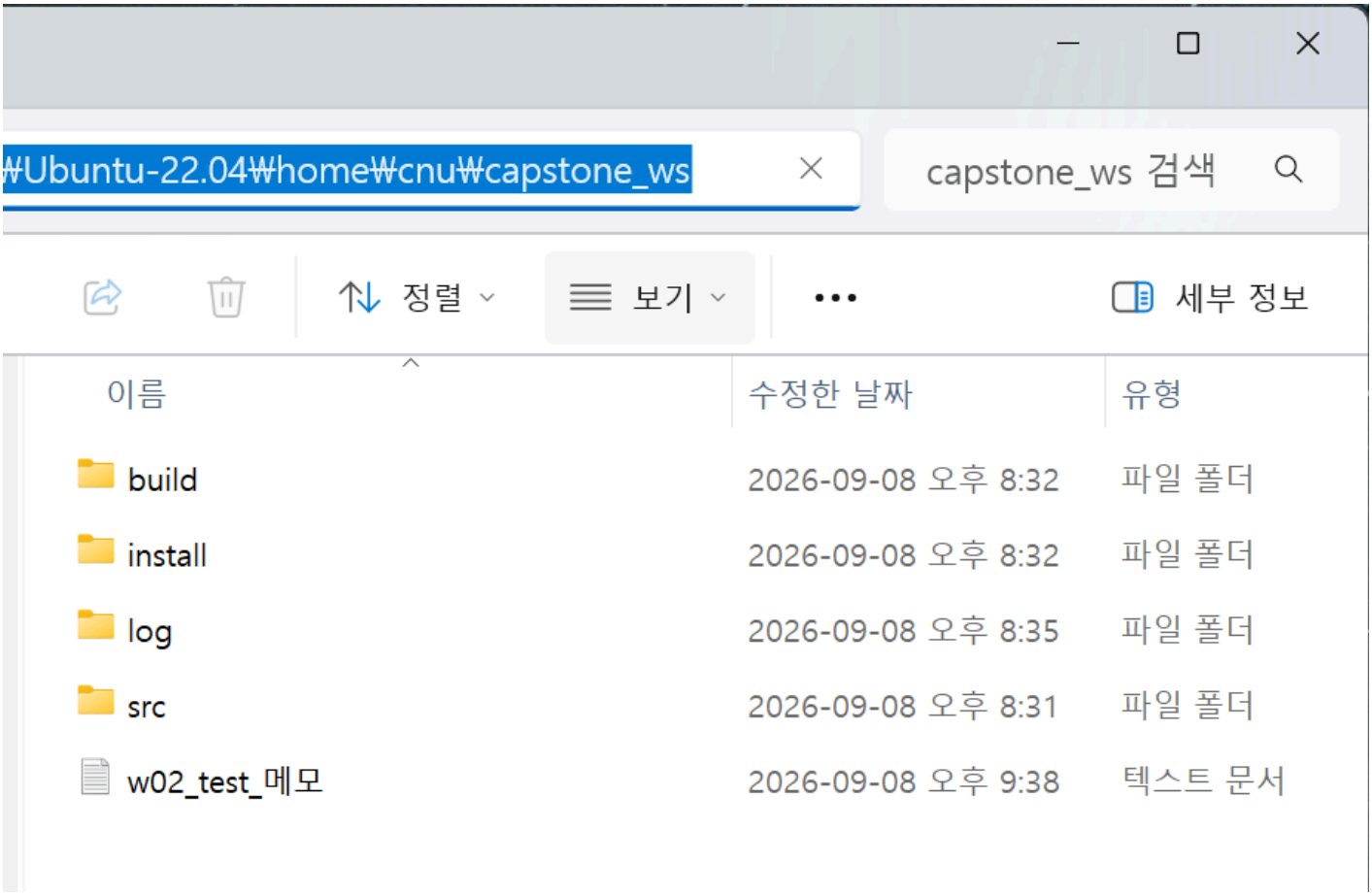
7단계 — WSL 안의 파일을 Windows 탐색기로 열기

> ★ **WSL 폴더는 Windows 탐색기에서 그대로 열림**
> 별도 설치가 필요 없음. 주소창에 아래를 그대로 붙여 넣으면 됨

```
\\wsl.localhost\Ubuntu-22.04\home\사용자명
```

- 예 — 사용자명이 `cnu` 이고 워크스페이스를 열려면

```
\\wsl.localhost\Ubuntu-22.04\home\cnu\capstone_ws
```



확인 항목	화면에서
주소창이 `\\wsl.localhost\Ubuntu-22.04\home\cnu\capstone_ws`	WSL 안을 보고 있음
`build` `install` `log`	§2-6 의 `colcon build` 가 만든 폴더 (`src` 는 직접 만든 것)
파일을 **끌어다 놓기**로 복사 가능	바탕화면 ↔ WSL 양방향

- 우분투 터미널에서 **탐색기를 바로 여는 명령**도 있음

```
explorer.exe .
```

- 현재 폴더가 탐색기 창으로 열림. 마지막의 **점(`.`)을 빠뜨리지 말 것**

바탕화면의 파일을 WSL 로 넣기

1. 탐색기 창을 두 개 엶 — 하나는 **바탕화면**, 하나는 위 WSL 경로
2. 파일을 **끌어다 놓음**. 일반 폴더처럼 복사됨
3. 우분투 터미널에서 확인

```
ls ~/capstone_ws
```

- 반대 방향(WSL → 바탕화면)도 똑같이 됨

경로 대응표

Windows 에서 보는 경로	우분투 안에서의 경로
`\\wsl.localhost\Ubuntu-22.04\home\cnu`	`/home/cnu` (또는 `~`)
`\\wsl.localhost\Ubuntu-22.04\home\cnu\capstone_ws`	`~/capstone_ws`
`C:\Users\사용자\Desktop`	`/mnt/c/Users/사용자/Desktop`

- 우분투 터미널에서 Windows 쪽 파일을 보려면 `/mnt/c/...` 를 씀

```
ls /mnt/c/Users/$USER/Desktop
```

✕ ROS 2 소스는 `/mnt/c/...` 에 두지 않음

`/mnt/c` 는 Windows 디스크를 빌려 쓰는 것이라 **파일 접근이 매우 느림**
`colcon build` 가 몇 배로 오래 걸리고, 파일 권한 문제도 생김
코드는 반드시 `~/capstone_ws` (우분투 안) 에 둠

⚠ 탐색기에서 WSL 파일을 편집하지 않음

메모장으로 열어 저장하면 줄바꿈이 **CRLF** 로 바뀌어 스크립트가 실행되지 않음
증상은 `/bin/bash^M: bad interpreter`. 편집은 **VS Code** 로 함

2-3. 설치 확인 — 첫 통신

실행

1. VS Code 통합 터미널을 엶 — `Ctrl + ``
2. 터미널 패널 오른쪽 위의 **분할 버튼**으로 좌우로 나눔
   - `terminator` 를 쓰는 경우 `Ctrl + Shift + E`

왼쪽 칸

```
ros2 run demo_nodes_cpp talker
```

- 정상 출력 (기준 환경 실측)

```
[INFO] [1788862636.484848636] [talker]: Publishing: 'Hello World: 1'
[INFO] [1788862637.466627284] [talker]: Publishing: 'Hello World: 2'
[INFO] [1788862638.466612785] [talker]: Publishing: 'Hello World: 3'
```

오른쪽 칸

```
ros2 run demo_nodes_py listener
```

- 정상 출력 (기준 환경 실측)

```
[INFO] [1788862636.495295736] [listener]: I heard: [Hello World: 1]
[INFO] [1788862637.467678514] [listener]: I heard: [Hello World: 2]
[INFO] [1788862638.467464411] [listener]: I heard: [Hello World: 3]
```

확인 항목	화면에서
두 칸의 숫자가 **같이** 올라감	연결 성공
대괄호 안 숫자가 **초 단위 시각**	첫 메시지(약 0.010 s)를 뺀 나머지는 두 로그의 시각 차이가 약 0.001 s → 지연이 1 ms 수준
왼쪽은 `Publishing`, 오른쪽은 `I heard`	발행·구독이 각각 동작

> ✎ **위 과정에서 일어난 일**
>
> - **C++로 짠 노드**와 **Python으로 짠 노드**가
> - 서로의 존재를 모르는 채로
> - 자동으로 찾아서 대화했음
> - 이것이 ROS 2의 핵심임

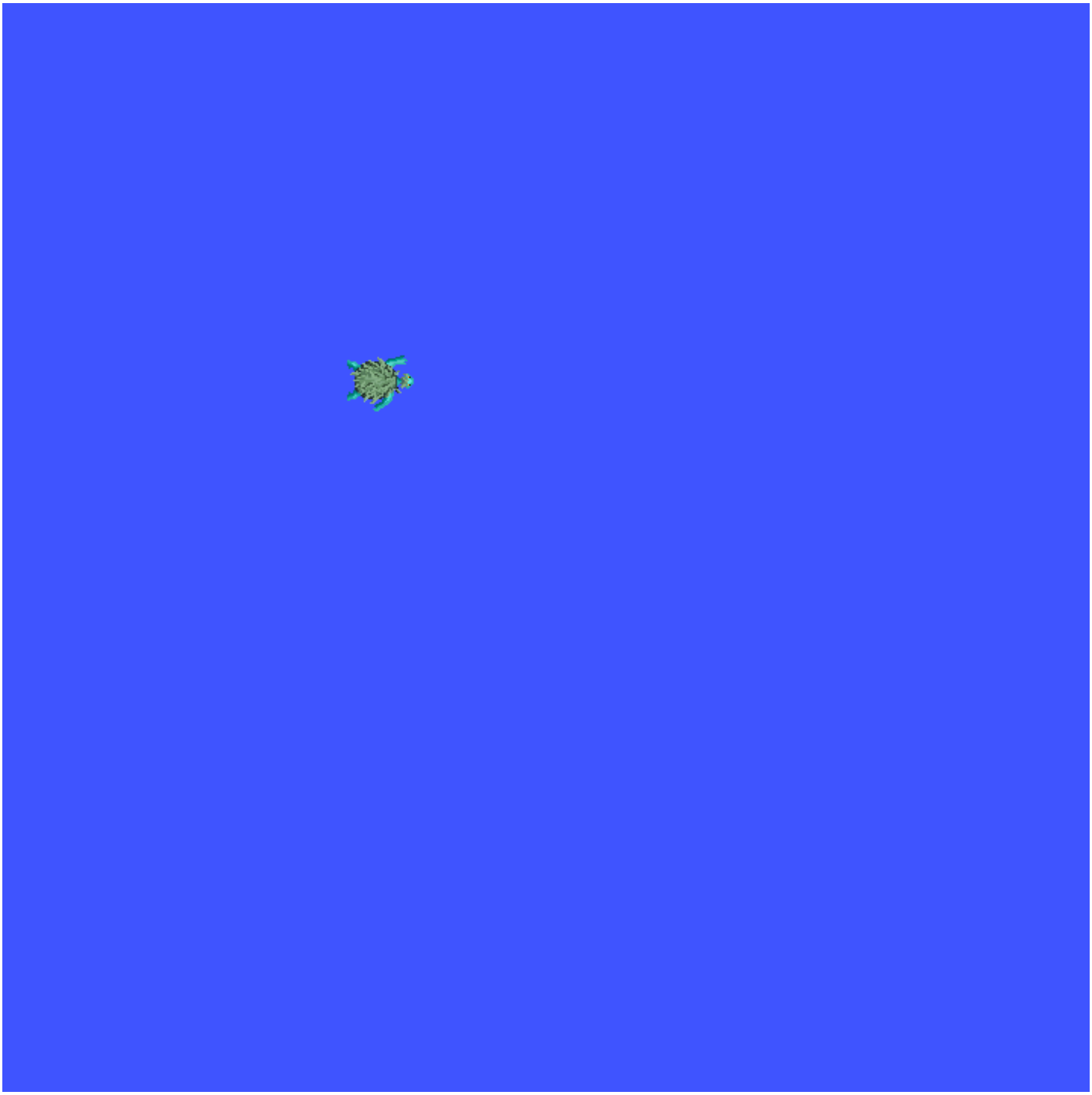
- 종료: 각 칸에서 `Ctrl + C`

2-4. turtlesim — 네 가지 통신을 눈으로 확인한다

> ★ **이 절이 ROS 2 를 처음 배울 때 가장 빠른 길임**
>
> 배도 센서도 없이 **거북이 한 마리**로 토픽 · 서비스 · 액션 · 파라미터를 전부 만져 봄
> `ros-humble-desktop` 에 이미 들어 있으므로 따로 설치하지 않음

1단계 — 띄운다

```
ros2 run turtlesim turtlesim_node
```



확인 항목	화면에서
파란 정사각형 창이 뜸	정상
가운데에 거북이 한 마리	이름은 `turtle1`. 거북이 그림·색은 실행마다 무작위
창이 안 뜸	GUI 문제. 1주차 2-4절 `xeyes` 로 복귀

- **다른 터미널**에서 무엇이 생겼는지 봄

```
ros2 node list
```

```
/turtlesim
```

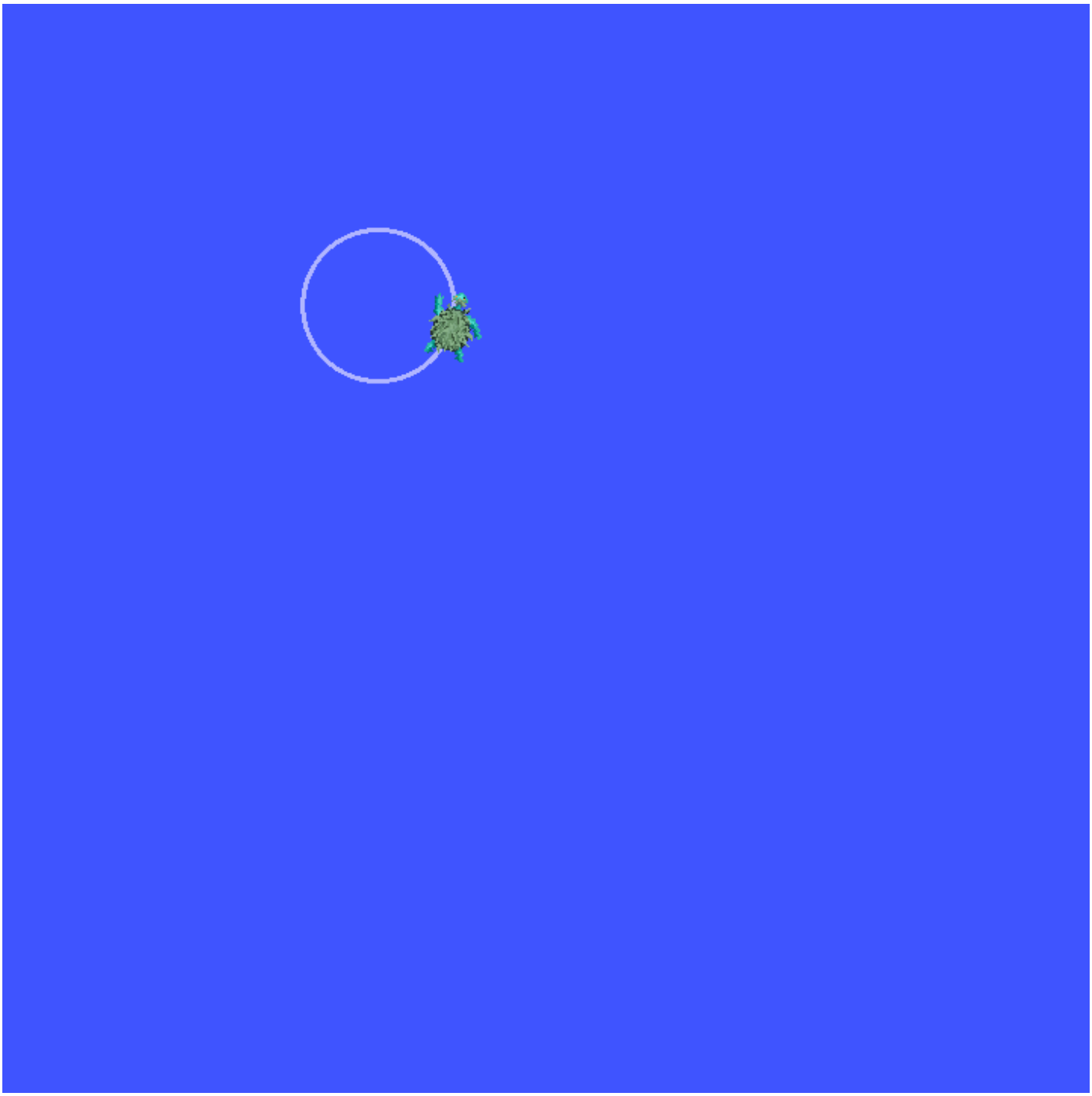
```
ros2 topic list -t
```

```
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

2단계 — 토픽으로 움직인다

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 1.8}}" -r 10
```

- `Ctrl + C` 로 멈춤. 멈추면 거북이도 섬



값	뜻
`linear.x`	앞으로 가는 속도 (m/s)
`angular.z`	도는 속도 (rad/s)
둘 다 주면	**원을 그림** — 위 화면의 궤적

> **이것이 3주차 이후 배를 움직이는 방식과 같음**
> WAM-V 도 결국 `/wamv/thrusters/...` 토픽에 숫자를 넣어 움직임
> 거북이의 `cmd_vel` 이 배의 추력 명령에 해당함

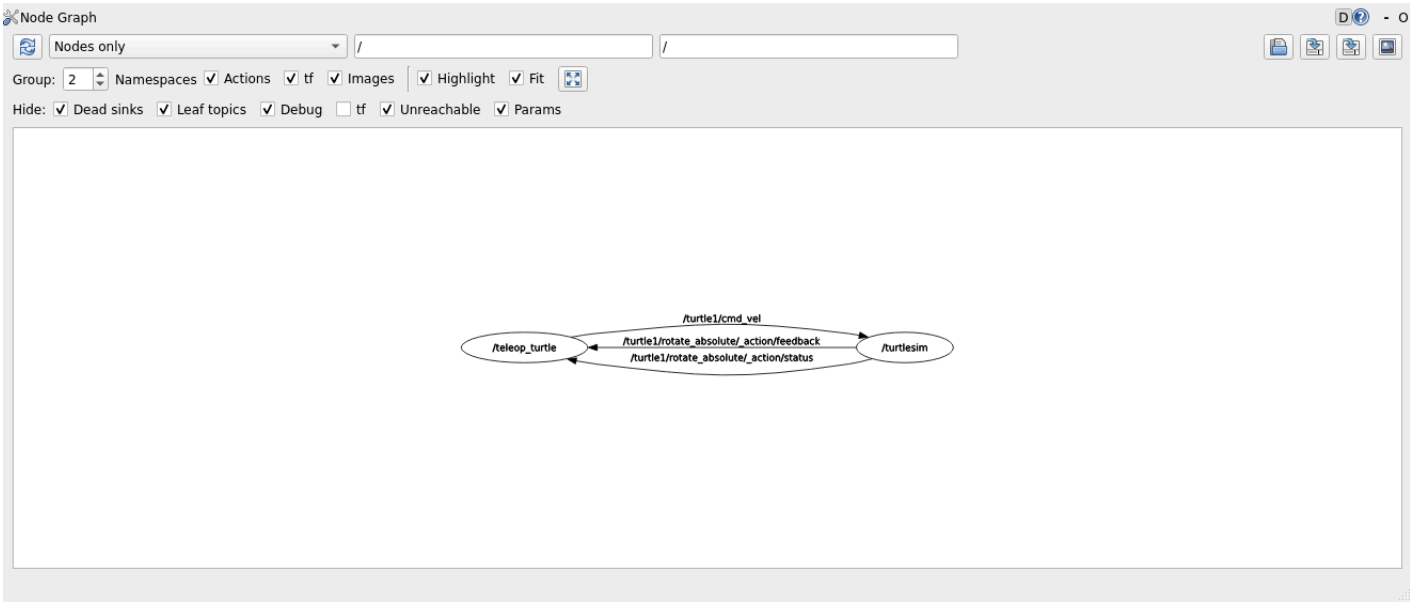
- 키보드로 몰아 보려면 **또 다른 터미널**에서

```
ros2 run turtlesim turtle_teleop_key
```

- 이 터미널에 **포커스를 둔 채로** 방향키를 누름. 다른 창을 클릭하면 안 먹음

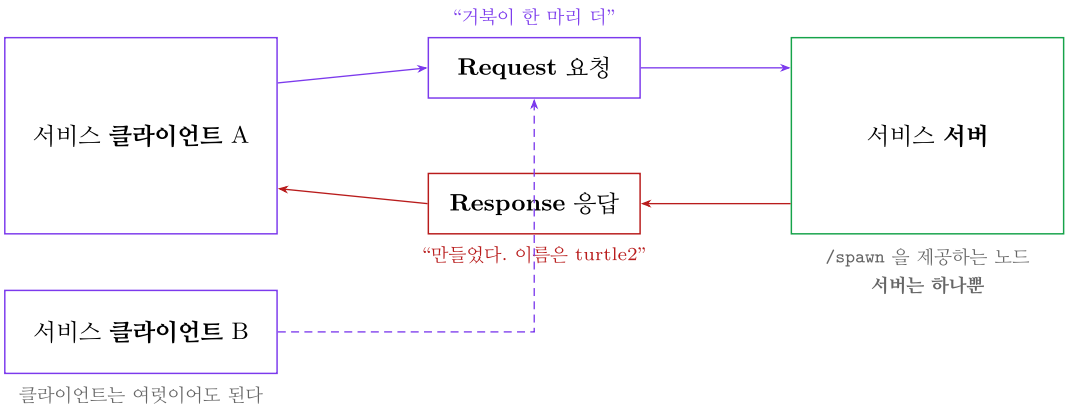
노드 그래프로 확인

```
ros2 run rqt_graph rqt_graph
```



그림에서	뜻
`/teleop_turtle` → `/turtle1/cmd_vel` → `/turtlesim`	**토픽** 연결
`/turtle1/rotate_absolute/_action/feedback` · `.../status`	**액션**도 내부적으로 토픽을 씀

3단계 — 서비스로 거북이를 하나 더 만든다



토픽과 다른 점 — 클라이언트는 답이 올 때까지 **기다린다**. 서버가 없으면 그 자리에서 멈춘 채 대기한다. `ros2 service call` 이 응답 없이 멈춰 있으면 **서버 노드가 안 떠 있는 것**이다

```
ros2 service list
```

- 정상 출력 (앞부분)

```
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
```

- 어떤 형식으로 불러야 하는지 확인

```
ros2 service type /spawn
```

```
turtlesim/srv/Spawn
```

```
ros2 interface show turtlesim/srv/Spawn
```

- 정상 출력 — `---` 위가 **요청**, 아래가 **응답**

```
float32 x
float32 y
float32 theta
string name # Optional.  A unique name will be created and returned if this is empty
---
string name
```

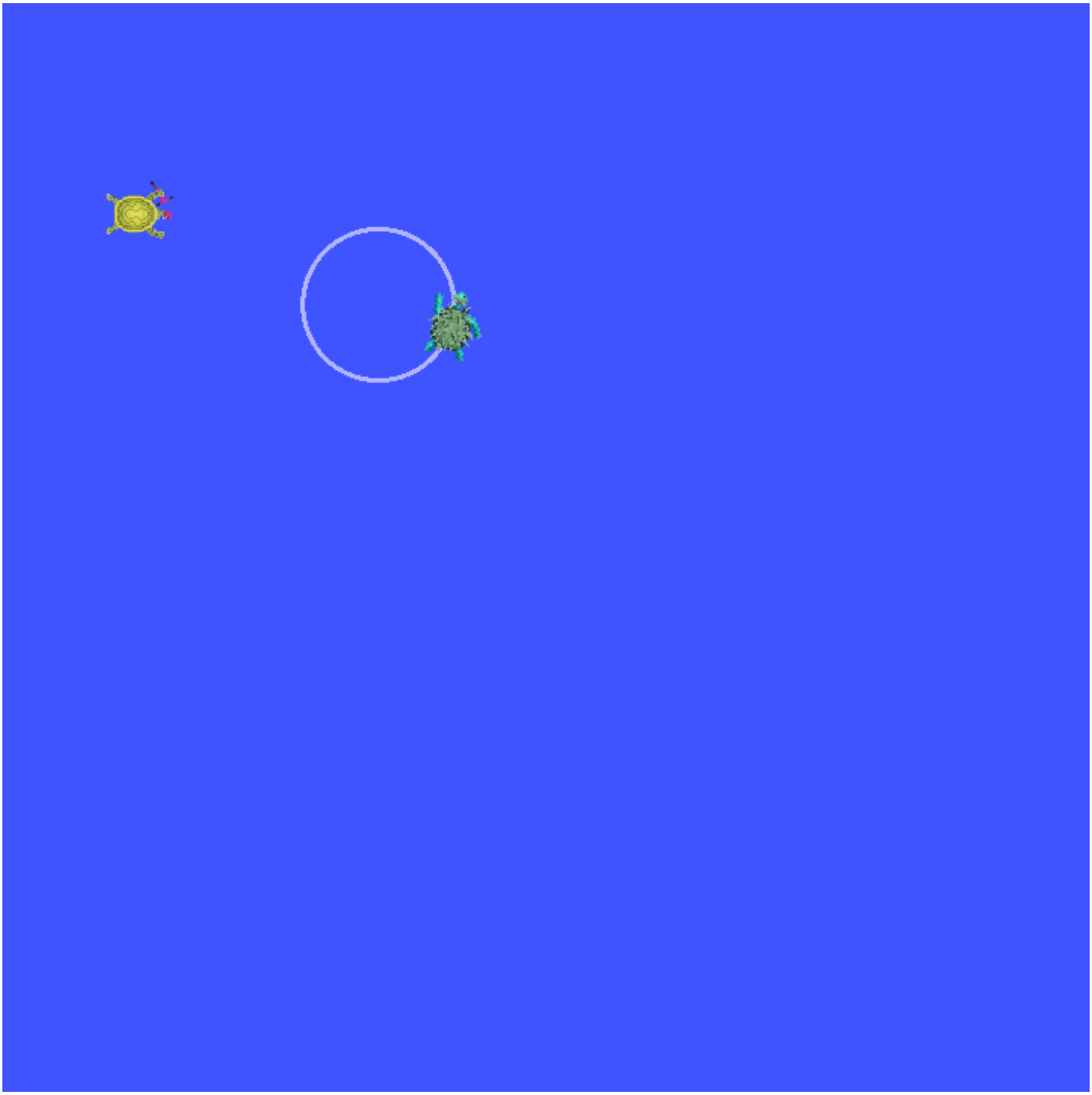
- 실제로 부름

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 8.0, theta: 0.0, name: 'turtle2'}"
```

- 정상 출력 (기준 환경 실측)

```
requester: making request: turtlesim.srv.Spawn_Request(x=2.0, y=8.0, theta=0.0, name='turtle2')

response:
turtlesim.srv.Spawn_Response(name='turtle2')
```



확인 항목	화면에서
왼쪽 위에 거북이 한 마리 더	`turtle2` 가 생겼음 (그림의 노란색은 무작위)
가운데 원형 궤적과 다른 거북이	2단계에서 움직인 `turtle1`

- 위치를 바로 옮기는 서비스도 있음

```
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 8.0, y: 8.0, theta: 1.57}"
```

```
response:
turtlesim.srv.TeleportAbsolute_Response()
```

ℹ 응답이 비어 있어도 정상임

`TeleportAbsolute` 는 돌려줄 값이 없음. `---` 아래가 비어 있는 것이 그 뜻임

⚠ `ros2 service call` 이 응답 없이 멈춰 있으면

서버 노드가 안 떠 있는 것임. `turtlesim_node` 를 껐는지 확인함
서비스는 답이 올 때까지 기다리므로, 아무 메시지 없이 멈춘 것처럼 보임

4단계 — 파라미터로 설정을 바꾼다

```
ros2 param list
```

- 정상 출력

```
/turtlesim:
  background_b
  background_g
  background_r
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  use_sim_time
```

```
ros2 param get /turtlesim background_b
```

```
Integer value is: 255
```

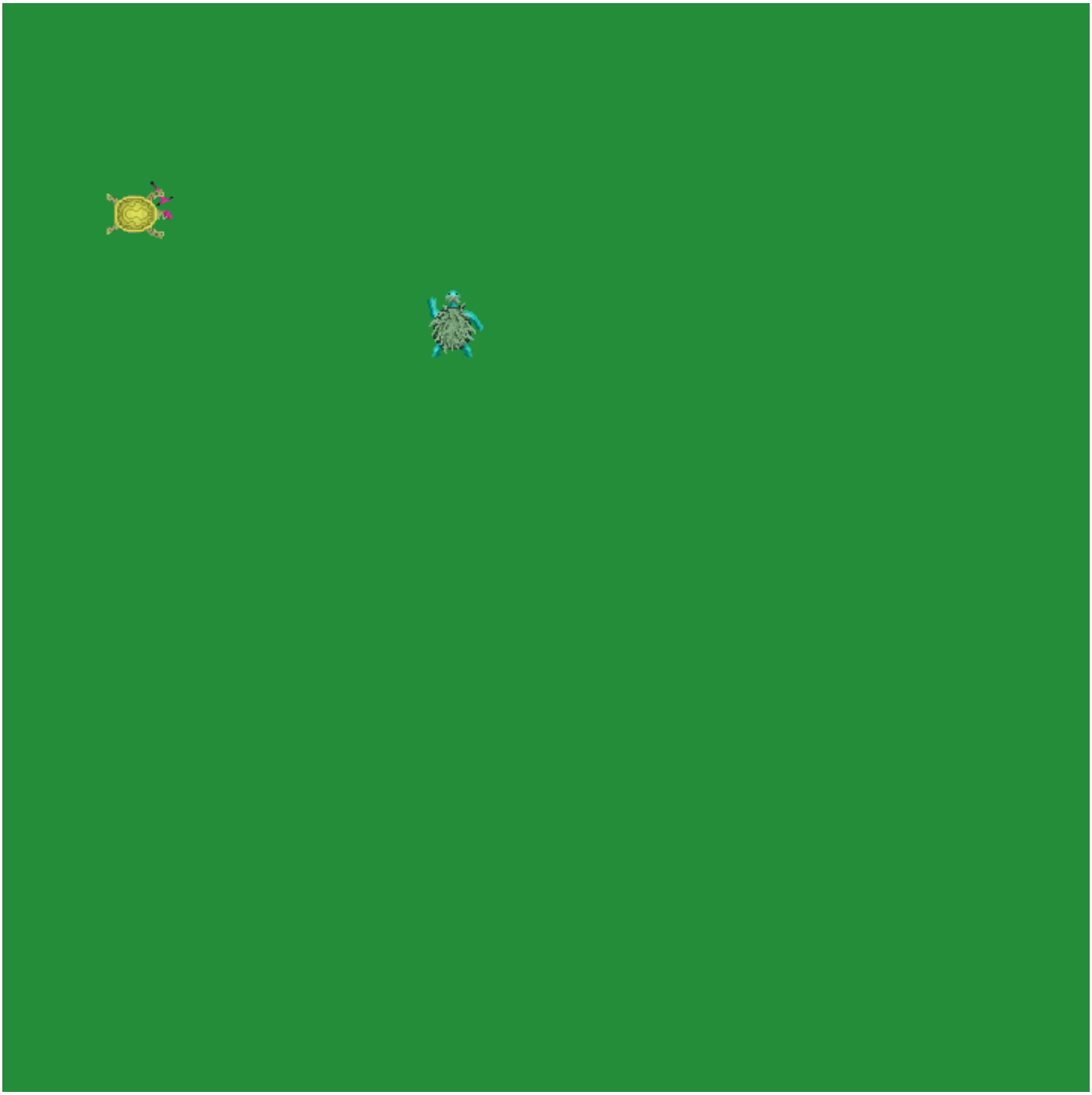
```
ros2 param set /turtlesim background_r 40
ros2 param set /turtlesim background_g 140
ros2 param set /turtlesim background_b 60
```

```
Set parameter successful
```

★ 여기서 화면은 아직 안 바뀜

turtlesim 은 배경색을 **다시 그릴 때** 반영함
`/clear` 서비스를 불러야 눈에 보임 — **파라미터와 서비스를 같이 쓰는 예**

```
ros2 service call /clear std_srvs/srv/Empty {}
```



- 배경이 초록으로 바뀜. 궤적은 지워짐
- 현재 설정을 파일로 저장할 수 있음

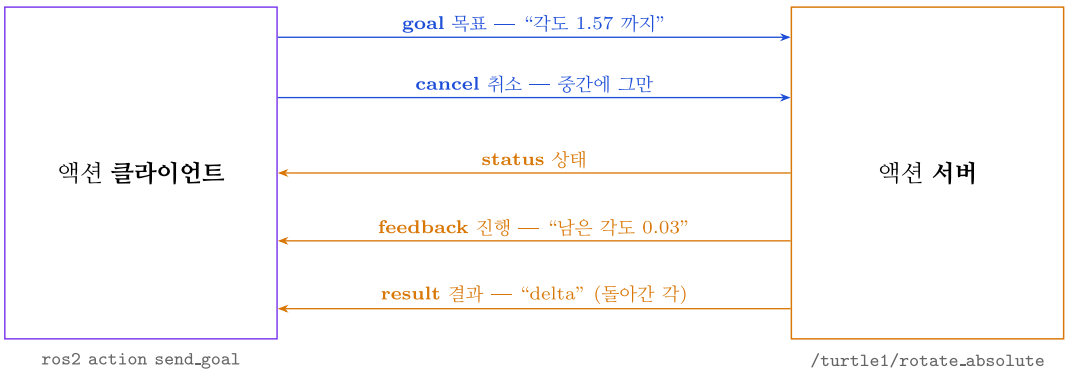
```
ros2 param dump /turtlesim
```

- 정상 출력

```
/turtlesim:
  ros__parameters:
    background_b: 60
    background_g: 140
    background_r: 40
    qos_overrides:
      /parameter_events:
        publisher:
          depth: 1000
          durability: volatile
          history: keep_last
          reliability: reliable
    use_sim_time: false
```

> **파라미터는 직접 작성하는 ROS 노드에서 씀**
> 코드를 고쳐 다시 빌드하는 대신 **파라미터로 빼 두면 실행 중에 바꿔 가며 튜닝**할 수 있음
> 11주차 이후 직접 작성하는 ROS 노드가 대상임. Simulink 제어기의 게인은 `W0X_setup.m` 변수로 다룸

5단계 — 액션으로 목표를 준다



토픽은 보내고 잊는다 · **서비스**는 한 번 묻고 한 번 답한다 · **액션**은 맡기고 **중간 보고**를 받으며 **취소**할 수 있다

```
ros2 action list
```

```
/turtle1/rotate_absolute
```

```
ros2 interface show turtlesim/action/RotateAbsolute
```

- 정상 출력 — `---` 로 **목표 / 결과 / 피드백** 세 부분으로 나뉨

```
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

> ℹ **3단계 teleport 로 거북이는 지금 1.57 rad (위쪽)를 보고 있음**
> 그래서 목표를 **반대쪽 −1.57 rad** 로 줌. 목표가 지금 각과 같으면 받자마자 끝나 `delta` 가 0 에 가까움

```
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: -1.57}" --feedback
```

- 정상 출력 (2026-09-19 기준 환경 실측, 1.57 rad 에서 시작. 가운데 Feedback 은 줄임)

```
Waiting for an action server to become available...
Sending goal:
     theta: -1.57

Goal accepted with ID: 62eaffdf9f504a28a2c4998ab187aeda

Feedback:
    remaining: -3.140000104904175

Feedback:
    remaining: -3.124000072479248

...

Feedback:
    remaining: -0.004000067710876465

Result:
    delta: 3.135999917984009

Goal finished with status: SUCCEEDED
```

나오는 것	뜻
`Feedback: remaining`	**진행 중** 보고. 남은 각도가 줄어듦
`Result: delta`	결과. 시작 각 − 끝 각 (rad) $= 1.57 - (-1.566) = 3.136$. 양수 = 각이 줄어든 쪽(시계방향)으로 약 180° 돌았다는 뜻
`Goal finished with status: SUCCEEDED`	성공 종료

> ★ **액션을 쓰는 이유가 여기 있음**
> 서비스였다면 다 돌 때까지 **아무 소식이 없음**
> 액션은 **남은 각도를 계속 알려 주고**, 중간에 취소할 수도 있음
> 8주차 Stateflow 미션의 상태 전환이 goal → feedback → result 와 같은 발상임 (ROS 액션을 쓰지는 않는다)

정리 — 네 가지를 언제 쓰는가

상황	쓰는 것	turtlesim 예
계속 흐르는 값	**토픽**	`/turtle1/cmd_vel` · `/turtle1/pose`
한 번 시키고 답을 받음	**서비스**	`/spawn` · `/clear`
오래 걸리고 중간 보고가 필요	**액션**	`/turtle1/rotate_absolute`
노드의 설정값	**파라미터**	`background_r`

정리하고 끝내기

```
ros2 service call /reset std_srvs/srv/Empty {}
```

- 거북이와 궤적이 처음 상태로 돌아감
- 창은 `Ctrl + C` 로 닫음

2-5. 조사 명령어 익히기

- `talker` 를 켜 둔 채로 **새 분할**에서 실행

> ✎ **명령어 다섯 개면 대부분의 문제를 진단할 수 있음**
>
> `node list` → `topic list -t` → `topic hz` → `topic echo` → `topic info --verbose`.
> **위에서 아래로 순서대로** 좁혀 나가는 것이 요령

노드 조사

```
ros2 node list
```

- 정상 출력 (talker 와 listener 를 둘 다 켠 상태)

```
/listener
/talker
```

```
ros2 node info /talker
```

- 정상 출력 (앞부분)

```
/talker
  Subscribers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
  Publishers:
    /chatter: std_msgs/msg/String
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
  Service Servers:
    /talker/describe_parameters: rcl_interfaces/srv/DescribeParameters
    ...
```

읽는 법	뜻
`Publishers:` 에 `/chatter`	이 노드가 `/chatter` 로 내보냄
`/rosout` 이 항상 있음	모든 노드가 로그를 이리로 보냄. `rqt_console` 이 이것을 봄
`/talker/set_parameters` 등	파라미터 서비스는 **자동으로** 생김. 직접 만든 것이 아님

토픽 조사

```
ros2 topic list
```

- 정상 출력

```
/chatter
/parameter_events
/rosout
```

```
ros2 topic list -t
```

- `-t` 를 붙이면 **메시지 타입까지** 표시됨

```
/chatter [std_msgs/msg/String]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
```

```
ros2 topic echo /chatter
```

- 실제 데이터가 흘러나옴. 종료는 `Ctrl + C`

```
data: 'Hello World: 12'
---
data: 'Hello World: 13'
---
```

- 한 개만 보고 끝내려면 `--once` 를 붙임

```
ros2 topic echo --once /chatter
```

```
ros2 topic hz /chatter
```

- 발행 주기 측정. **가장 자주 쓰는 명령어**
- 정상 출력 (기준 환경 실측, `demo_nodes_cpp talker` 는 1 Hz)

```
average rate: 1.018
    min: 0.978s max: 1.000s std dev: 0.00745s window: 7
```

읽는 법	뜻
`average rate`	초당 몇 번 오는가
`min` / `max`	간격의 최소·최대. 벌어지면 **주기가 흔들린다는 뜻**
`std dev`	간격의 표준편차. 클수록 불안정
`window`	몇 개를 보고 계산했는가

```
ros2 topic bw /chatter
```

- 대역폭 측정. 3주차 이후 카메라·LiDAR 토픽에서 씀

```
ros2 topic info /chatter --verbose
```

- **QoS 를 포함한 상세 정보.** 문제 진단의 핵심
- 정상 출력 (발행자 부분만)

```
Type: std_msgs/msg/String

Publisher count: 1

Node name: talker
Node namespace: /
Topic type: std_msgs/msg/String
Endpoint type: PUBLISHER
GID: 01.0f.89.6f.1e.00.c3.66.00.00.00.00.00.00.12.03.00.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 1
...
```

> **★ 이 출력의 `Reliability` 두 개를 비교하는 것이 QoS 진단임**
> 발행자와 구독자의 `Reliability` 가 다르면 §2-9 의 유의 사항이 발생함

메시지 구조 확인

```
ros2 interface show std_msgs/msg/String
```

- 정상 출력

```
# This was originally provided as an example message.
# It is deprecated as of Foxy
# It is recommended to create your own semantically meaningful message.
# However if you would like to continue using this please use the equivalent in example_msgs.

string data
```

- 센서 메시지의 공통 앞부분인 `Header` 도 같은 방법으로 봄

```
ros2 interface show std_msgs/msg/Header
```

```
# Standard metadata for higher-level stamped data types.
# This is generally used to communicate timestamped data
# in a particular coordinate frame.

# Two-integer timestamp that is expressed as seconds and nanoseconds.
builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec

# Transform frame with which this data is associated.
string frame_id
```

명령줄에서 직접 발행

- 노드를 만들지 않고도 토픽을 쏴 볼 수 있음. **구독자 쪽을 시험할 때** 씀

```
ros2 topic pub /chatter std_msgs/msg/String "{data: 'from CLI'}" -r 1
```

옵션	뜻
`-r 1`	1 Hz 로 반복 발행
`--once`	한 번만 보내고 종료

설치·환경을 한 번에 점검하는 명령

```
ros2 doctor
```

- ROS 2 설치 상태, 네트워크, `rmw` 구현, 패키지 버전을 한꺼번에 점검함
- 중간에 `UserWarning: ... has been updated to a new version` 이 여러 줄 나오는 것은 정상
- **마지막 줄**만 확인

```
All 5 checks passed
```

패키지가 어떤 실행파일을 갖고 있는지

```
ros2 pkg executables demo_nodes_cpp
```

- 정상 출력 (앞부분)

```
demo_nodes_cpp add_two_ints_client
demo_nodes_cpp add_two_ints_client_async
demo_nodes_cpp add_two_ints_server
demo_nodes_cpp allocator_tutorial
demo_nodes_cpp content_filtering_publisher
demo_nodes_cpp content_filtering_subscriber
```

- `ros2 run` **의 두 번째 인자로 무엇을 써야 하는지** 이 명령으로 확인

실습 — 공식 예제로 확인하기

- ROS 2 개발팀이 관리하는 **공식 예제 저장소**를 그대로 받아 씀

```
cd ~
git clone -b humble https://github.com/ros2/examples.git ros2_examples
```

- 출처 — `ros2/examples` (Apache License 2.0), `humble` 브랜치 (작성 시점 최신 태그 `0.15.5`)
- 이 과목에서는 `rclpy/topics/` 아래의 두 파일만 씀
- 빌드하지 않고 **파이썬 파일을 직접 실행**해도 됨. 의존성이 `rclpy` 와 `std_msgs` 뿐임

터미널 1 — 발행자

```
python3 ~/ros2_examples/rclpy/topics/minimal_publisher/examples_rclpy_minimal_publisher/publisher_memb
er_function.py
```

- 정상 출력

```
[INFO] [1788684118.901490181] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [1788684119.408459082] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [1788684119.903331908] [minimal_publisher]: Publishing: "Hello World: 2"
```

터미널 2 — 구독자

```
python3 ~/ros2_examples/rclpy/topics/minimal_subscriber/examples_rclpy_minimal_subscriber/subscriber_m
ember_function.py
```

- 정상 출력

```
[INFO] [1788684120.397561412] [minimal_subscriber]: I heard: "Hello World: 3"
[INFO] [1788684120.893497587] [minimal_subscriber]: I heard: "Hello World: 4"
[INFO] [1788684121.390922502] [minimal_subscriber]: I heard: "Hello World: 5"
```

> **ℹ 구독자의 첫 숫자가 0 이 아닌 이유**
> 발행자를 먼저 켰기 때문. 구독자는 **켜진 뒤부터** 받음
> 위 실측에서는 3번부터 받았음. 놓친 0~2번은 되돌아오지 않음
> 이것을 바꾸는 설정이 QoS 의 **Durability** (§1-6)

발행자·구독자의 QoS 를 눈으로 확인한다

```
ros2 topic info /topic --verbose
```

- 정상 출력 (일부)

```
Type: std_msgs/msg/String
Publisher count: 1

Node name: minimal_publisher
Endpoint type: PUBLISHER
QoS profile:
  Reliability: RELIABLE
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
```

항목	이 예제의 값	뜻
`Reliability`	**RELIABLE**	빠지면 다시 보냄
`Durability`	**VOLATILE**	늦게 들어온 구독자에게 과거 것을 주지 않음
`Lifespan` · `Deadline`	Infinite	제한 없음

- **양쪽의 이 표가 서로 호환되어야 연결됨.** 안 맞으면 오류 없이 조용히 끊김
- 실제로 끊어 보는 실험이 §2-9 에 있음

데이터 기록 · 재생

```
ros2 bag record /chatter
```

```
ros2 bag play rosbag2_2026_09_10-14_30_00
```

> ℹ **왜 중요한가**
>
> 12주차에 충돌회피 알고리즘을 비교할 때
> **같은 LiDAR 데이터를 반복 재생**해서 알고리즘만 바꿔 비교함.
> 공정한 비교의 필수 도구임.

2-6. 패키지 만들기

> ★ **두 가지 길이 있음. 수업에서는 ①로 진행함**
>
> | 길 | 무엇을 하는가 | 언제 |
> |---|---|---|
> | ① **직접 만들기** | `ros2 pkg create` 부터 손으로 | **수업 시간.** 구조를 알아야 10주차부터 스스로 만듦 |
> | ② **저장소에서 받기** | `git clone` 한 줄 | 복습할 때 · 실습이 밀렸을 때 · 코드가 꼬였을 때 |
>
> ②는 아래 "저장소에서 받기" 를 보면 됨. **①을 건너뛰지 말 것**

워크스페이스 생성

```
mkdir -p ~/capstone_ws/src
cd ~/capstone_ws/src
```

패키지 생성

```
ros2 pkg create --build-type ament_python --license Apache-2.0 usv_basics
```

- 정상 출력 (기준 환경 실측)

```
going to create a new package
package name: usv_basics
destination directory: /home/cnu/capstone_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['cnu <cnu@todo.todo>']
licenses: ['Apache-2.0']
build type: ament_python
dependencies: []
creating folder ./usv_basics
creating ./usv_basics/package.xml
creating source folder
creating folder ./usv_basics/usv_basics
creating ./usv_basics/setup.py
creating ./usv_basics/setup.cfg
creating folder ./usv_basics/resource
creating ./usv_basics/resource/usv_basics
creating ./usv_basics/usv_basics/__init__.py
creating folder ./usv_basics/test
creating ./usv_basics/test/test_copyright.py
creating ./usv_basics/test/test_flake8.py
creating ./usv_basics/test/test_pep257.py
```

- 실제로 만들어진 구조 — `find` 로 확인한 결과

```
usv_basics/
├── LICENSE                 Apache-2.0 전문
├── package.xml             패키지 정보, 의존성
├── setup.py                빌드 설정, 실행파일 등록
├── setup.cfg               실행파일이 깔릴 경로
├── resource/usv_basics     ROS 2 가 패키지를 찾는 표식 파일 (비어 있음)
├── test/                   자동 생성된 검사 3종
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py
└── usv_basics/             <- 여기에 .py 파일을 넣는다
    └── __init__.py
```

> **`--license` 를 빼면 경고가 남**
> 라이선스가 없으면 `colcon build` 가 경고를 냄. 팀 저장소에 올릴 것이므로 붙여 둠

- VS Code 탐색기에서도 같은 구조가 보임 (§2-2 의 화면)

빌드

```
cd ~/capstone_ws
colcon build --symlink-install
source install/setup.bash
```

- 정상 출력 (2026-09-19 기준 환경 실측, 패키지가 비어 있을 때). 시간은 PC 마다 다르며 1 초 안팎이면 정상

```
Starting >>> usv_basics
Finished <<< usv_basics [0.60s]

Summary: 1 package finished [0.90s]
```

- 빌드 후 `~/capstone_ws` 에 **`build/`** · **`install/`** · **`log/`** 세 폴더가 새로 생김

> ✎ **`--symlink-install` 사용을 권장함**
> Python 파일이 링크로 연결되어 **코드를 고칠 때마다 다시 빌드할 필요가 없음**.

- 자동 적용 등록 — **한 번만** 실행함 (`grep -c capstone_ws ~/.bashrc` 가 1)

```
echo "source ~/capstone_ws/install/setup.bash" >> ~/.bashrc
```

저장소에서 받기 (복습·복구용)

- 강의에서 만드는 패키지와 **똑같은 것**을 아래 저장소에 올려 두었음
  - https://github.com/wkyouncnu/usv_basics · Apache-2.0 · **로그인 불필요**

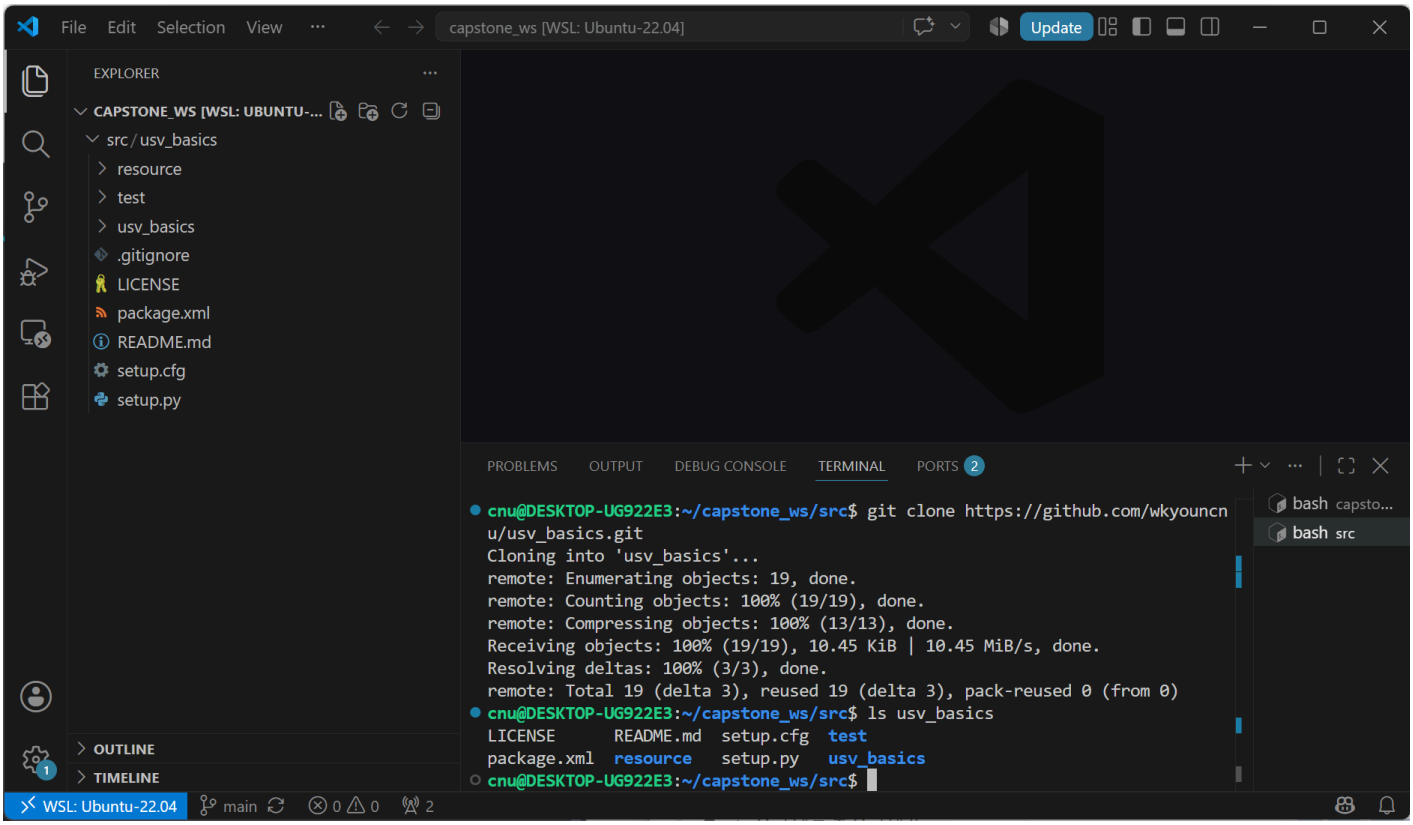
> ★ **순서가 중요함 — 우분투 안에서 받고, 그 다음 VS Code 로 엶**
>
> | 순서 | 어디서 | 무엇을 |
> |---|---|---|
> | 1 | **WSL 우분투 터미널** | `git clone` |
> | 2 | **WSL 우분투 터미널** | `colcon build` |
> | 3 | VS Code | `code .` 로 열어서 편집 |
>
> Windows 쪽(`C:\`)에 받으면 **빌드가 되지 않음.** `colcon` 은 우분투 안에만 있음

1단계 — 받기

```
mkdir -p ~/capstone_ws/src
cd ~/capstone_ws/src
git clone https://github.com/wkyouncnu/usv_basics.git
```

- `git` 이 없으면 먼저 설치

```
sudo apt update && sudo apt install -y git
```

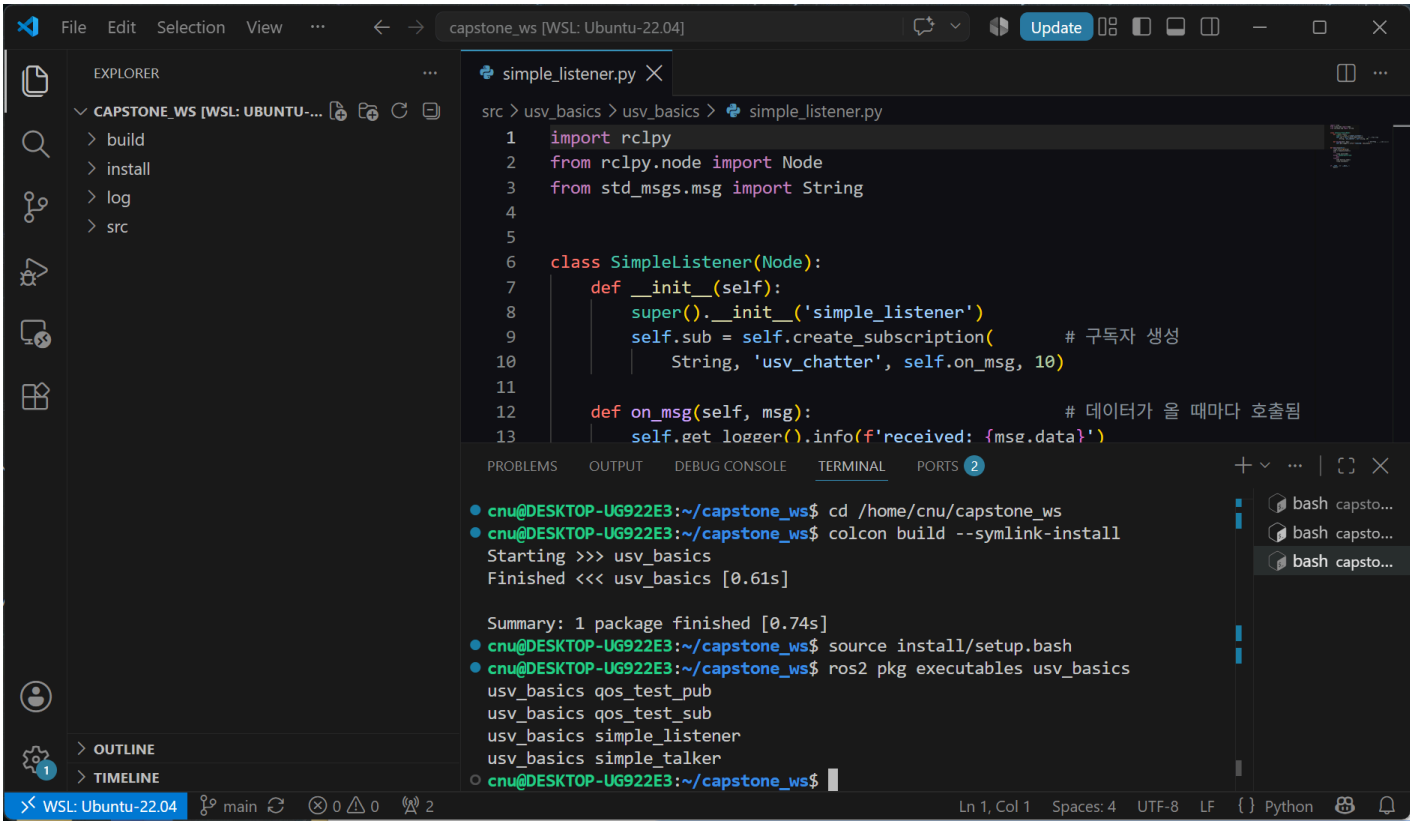


화면에서 확인할 것	무엇
프롬프트가 `~/capstone_ws/src$`	**받는 위치가 맞음**
`Cloning into 'usv_basics'...`	내려받기 시작
`Receiving objects: 100%`	내려받기 완료 (괄호 안 숫자는 git 객체 수. 저장소 갱신에 따라 달라짐)
`ls usv_basics` 결과에 `package.xml` `setup.py`	제대로 받아졌음
왼쪽 탐색기에 `src/usv_basics` 가 생김	VS Code 가 자동으로 알아봄
맨 아래 `main`	git 저장소로 인식됨

- 아이디·비밀번호를 묻지 않음. **공개 저장소**이므로 별도 인증 없이 받아짐

2단계 — 빌드

```
cd ~/capstone_ws
colcon build --symlink-install
source install/setup.bash
ros2 pkg executables usv_basics
```



화면에서 확인할 것	무엇
프롬프트가 `~/capstone_ws$`	**`src` 가 아니라 한 단계 위**에서 빌드함
`Finished <<< usv_basics [0.61s]`	빌드 성공
`Summary: 1 package finished`	패키지 1개
탐색기에 `build` `install` `log` 가 생김	colcon 이 만든 것
실행파일 6개가 나열됨	등록 완료. 3주차 `wamv_teleop_key` · §2-10 `turtle_pose_relay` 까지 들어 있음

> ⚠ **`src` 안에서 `colcon build` 를 하면 안 됨**
>
> `src/` 안에 또 `build/` `install/` `log/` 가 생겨 버림
> 그렇게 되면 세 폴더를 지우고 **한 단계 위에서 다시** 빌드함
>
> ```
> rm -rf ~/capstone_ws/src/build ~/capstone_ws/src/install ~/capstone_ws/src/log
> cd ~/capstone_ws && colcon build --symlink-install
> ```

3단계 — VS Code 로 열기

```
cd ~/capstone_ws
code .
```

- 이후 편집·실행은 §2-2 의 통합 터미널에서 그대로 함

> ℹ **저장소의 `qos_test_sub.py` 는 정상 상태로 배포함**
> 받자마자 `qos_test_pub` ↔ `qos_test_sub` 가 서로 통함
> §2-9 의 불일치를 재현하려면 `ReliabilityPolicy.BEST_EFFORT` 를 **`ReliabilityPolicy.RELIABLE` 로 직접 바꿔** 보면 됨. 고치는 방향이 반대일 뿐 실험은 같음

강의자료 저장소 — 한 번 받고, 매주 `git pull`

- 주차 문서 · Simulink 모델 · MATLAB 스크립트는 강의자료 저장소 하나에 모여 있고 **수업 중에도 계속 갱신됨**
  - https://github.com/wkyouncnu/Capstone-Design · 공개 · 로그인 불필요
- 받는 곳은 `usv_basics` 와 같음 — **VS Code 의 WSL 창 터미널** (`Ctrl + ` `)

처음 한 번 — clone

```
git config --global core.quotepath false      # 한글 파일 이름이 \355\... 로 깨져 보이지 않게
cd ~
git clone https://github.com/wkyouncnu/Capstone-Design.git
```

- 약 320 MB (2026-09-19 기준). 처음 한 번만 걸림
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/`

매주 수업 전 — pull

```
cd ~/Capstone-Design
git pull
```

출력	뜻
`Updating b690ad2..ce9c217` · `Fast-forward` · 바뀐 파일 목록	새 판을 받았음 — 정상
`Already up to date.`	이미 최신 — 정상
`error: Your local changes to the following files would be overwritten by merge:`	받은 파일을 고쳐 둔 상태 → `git stash` → `git pull` → `git stash pop`

- **다시 `git clone` 하지 않음.** 같은 자리에 또 받으면 `already exists` 로 멈추고, 지우고 받으면 고친 것이 사라짐
- MATLAB 은 이 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 경로로 그대로 엶
- 전체 절차 · 실측 출력 · 충돌 처리 → 강의자료는-한-번-받고-git-pull-로-갱신한다

> ✎ **`usv_basics` 도 같은 방법으로 갱신함**
> `cd ~/capstone_ws/src/usv_basics && git pull` 뒤 `cd ~/capstone_ws && colcon build --symlink-install` (3주차 2-6)

2-7. 첫 노드 작성

> ★ **이 절부터는 VS Code 로 파일을 만듦**
> §2-2 에서 `code .` 로 `~/capstone_ws` 를 열어 둔 상태여야 함
> 왼쪽 아래에 **`WSL: Ubuntu-22.04`** 가 보이는지 먼저 확인할 것.

VS Code 에서 새 파일을 만드는 방법

1. 왼쪽 **탐색기**에서 `src` → `usv_basics` → `usv_basics` 폴더를 펼침
2. 그 **폴더 이름 위에 마우스를 올리면** 오른쪽에 아이콘 네 개가 나타남
3. 맨 왼쪽 **새 파일** 아이콘(문서에 `+`) 클릭
4. 파일 이름을 입력하고 `Enter`

- 만들 위치를 헷갈리지 않도록 **전체 경로**를 적어 둠

```
~/capstone_ws/src/usv_basics/usv_basics/
```

> ⚠ `usv_basics` **폴더가 두 번 나옴**
> 바깥쪽은 **패키지 폴더**, 안쪽은 **파이썬 모듈 폴더**
> `.py` 파일은 **안쪽**에 넣음. 바깥쪽에 넣으면 `colcon build` 가 무시함

동작	단축키
저장	`Ctrl + S`
붙여넣기	`Ctrl + V` (터미널에서는 `Ctrl + Shift + V`)
파일 찾기	`Ctrl + P` 후 파일명 입력
통합 터미널 열기·닫기	`Ctrl + ` `

발행자

- 파일 위치: `~/capstone_ws/src/usv_basics/usv_basics/simple_talker.py`
- 위 방법으로 `simple_talker.py` 를 만들고 아래 내용을 붙여넣음 (`Ctrl + V`)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String


class SimpleTalker(Node):
    def __init__(self):
        super().__init__('simple_talker')          # 노드 이름
        self.pub = self.create_publisher(          # 발행자 생성
            String, 'usv_chatter', 10)             # 타입, 토픽명, 큐 크기
        self.timer = self.create_timer(0.5, self.on_timer)   # 0.5초마다 = 2 Hz
        self.count = 0

    def on_timer(self):
        msg = String()
        msg.data = f'USV alive: {self.count}'
        self.pub.publish(msg)
        self.get_logger().info(f'published: {msg.data}')
        self.count += 1


def main(args=None):
    rclpy.init(args=args)
    node = SimpleTalker()
    try:
        rclpy.spin(node)          # 계속 돌면서 타이머·콜백 처리
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

- **저장**: `Ctrl + S`. 탭 이름 옆의 **흰 점(●)이 사라지면** 저장된 것

> **✎ 빨간 밑줄이 뜨면 저장 전에 고침**
> Python 확장이 오타·들여쓰기 오류를 즉시 표시함
> 화면 왼쪽 아래 `✕ 0 ⚠ 0` 이 **둘 다 0** 이어야 정상

구독자

- 같은 폴더에 `simple_listener.py` 를 새로 만듦

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String


class SimpleListener(Node):
    def __init__(self):
        super().__init__('simple_listener')
        self.sub = self.create_subscription(       # 구독자 생성
            String, 'usv_chatter', self.on_msg, 10)

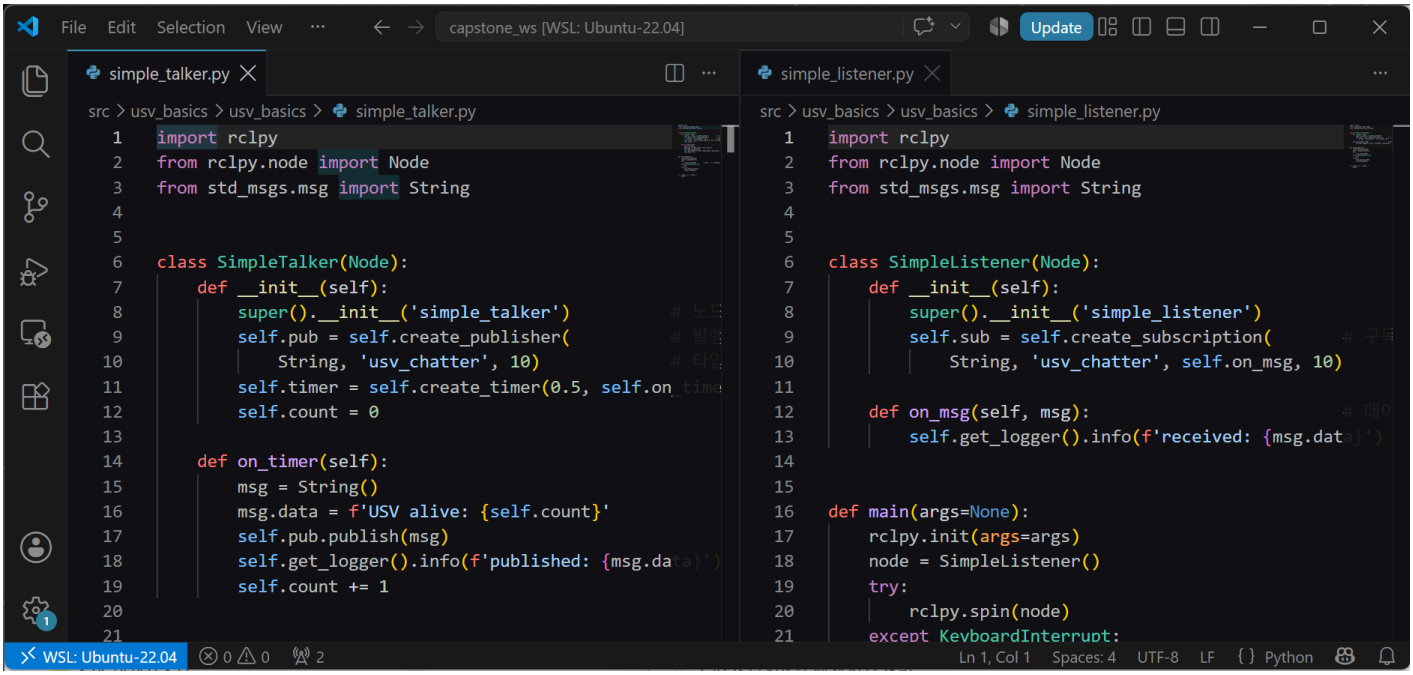
    def on_msg(self, msg):                         # 데이터가 올 때마다 호출됨
        self.get_logger().info(f'received: {msg.data}')


def main(args=None):
    rclpy.init(args=args)
    node = SimpleListener()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

발행자와 구독자, 무엇이 다른가 — 나란히 놓고 본다

- VS Code 에서 **편집기를 좌우로 나누면** 두 파일을 나란히 볼 수 있음
  - 왼쪽 파일을 연 상태에서 오른쪽 위 **편집기 분할** 아이콘(네모 두 개) 클릭
  - 오른쪽 칸에서 `Ctrl + P` → `simple_listener.py` → `Enter`



- **1~3행은 완전히 같음.** 두 노드 모두 `rclpy` · `Node` · `String` 을 씀

줄	발행자 `simple_talker.py`	구독자 `simple_listener.py`	무엇이 다른가
6	`class SimpleTalker(Node)`	`class SimpleListener(Node)`	클래스 이름만 다름. **둘 다 `Node` 를 상속**함
8	`super().__init__('simple_talker')`	`super().__init__('simple_listener')`	**노드 이름.** `ros2 node list` 에 이 이름이 뜸
9~10	`self.create_publisher(String, 'usv_chatter', 10)`	`self.create_subscription(String, 'usv_chatter', self.on_msg, 10)`	**핵심 차이.** 구독자는 인자가 하나 더 있음
11	`self.create_timer(0.5, self.on_timer)`	(없음)	발행자만 **스스로 주기적으로** 움직임
12	`self.count = 0`	(없음)	발행자만 셀 것이 있음
14~19	`def on_timer(self)` — 0.5초마다 호출	`def on_msg(self, msg)` — **메시지가 올 때마다** 호출	**누가 부르는가가 다름**

> ★ **이 표의 마지막 줄이 이번 절의 핵심**
>
> - **발행자의 `on_timer` 는 시계**가 부름. 아무도 안 들어도 계속 돎
> - **구독자의 `on_msg` 는 메시지**가 부름. 안 오면 한 번도 안 돎
> - 그래서 §2-9 의 QoS 불일치에서 **구독자 화면만 조용**해짐

- `create_publisher` 와 `create_subscription` 의 인자 순서

순서	발행자	구독자
1	메시지 타입 `String`	메시지 타입 `String`
2	토픽 이름 `'usv_chatter'`	토픽 이름 `'usv_chatter'`
3	큐 크기 `10`	**콜백 함수** `self.on_msg`
4	—	큐 크기 `10`

> ⚠ **콜백을 `self.on_msg()` 로 쓰면 안 됨**
> 괄호를 붙이면 **지금 한 번 실행**해서 그 결과를 넘김
> 괄호 없이 `self.on_msg` 로 써야 **함수 자체**를 넘김. 초보자가 가장 많이 틀리는 한 글자

- 두 파일에 공통으로 들어가는 `main()` 의 뼈대

줄	하는 일
`rclpy.init(args=args)`	ROS 2 통신 시작
`node = SimpleTalker()`	노드 객체를 만듦 (`__init__` 이 여기서 돈다)
`rclpy.spin(node)`	**여기서 멈춰 서서** 타이머·콜백을 처리함
`except KeyboardInterrupt`	`Ctrl + C` 를 눌렀을 때 조용히 빠져나옴
`node.destroy_node()` / `rclpy.shutdown()`	뒷정리

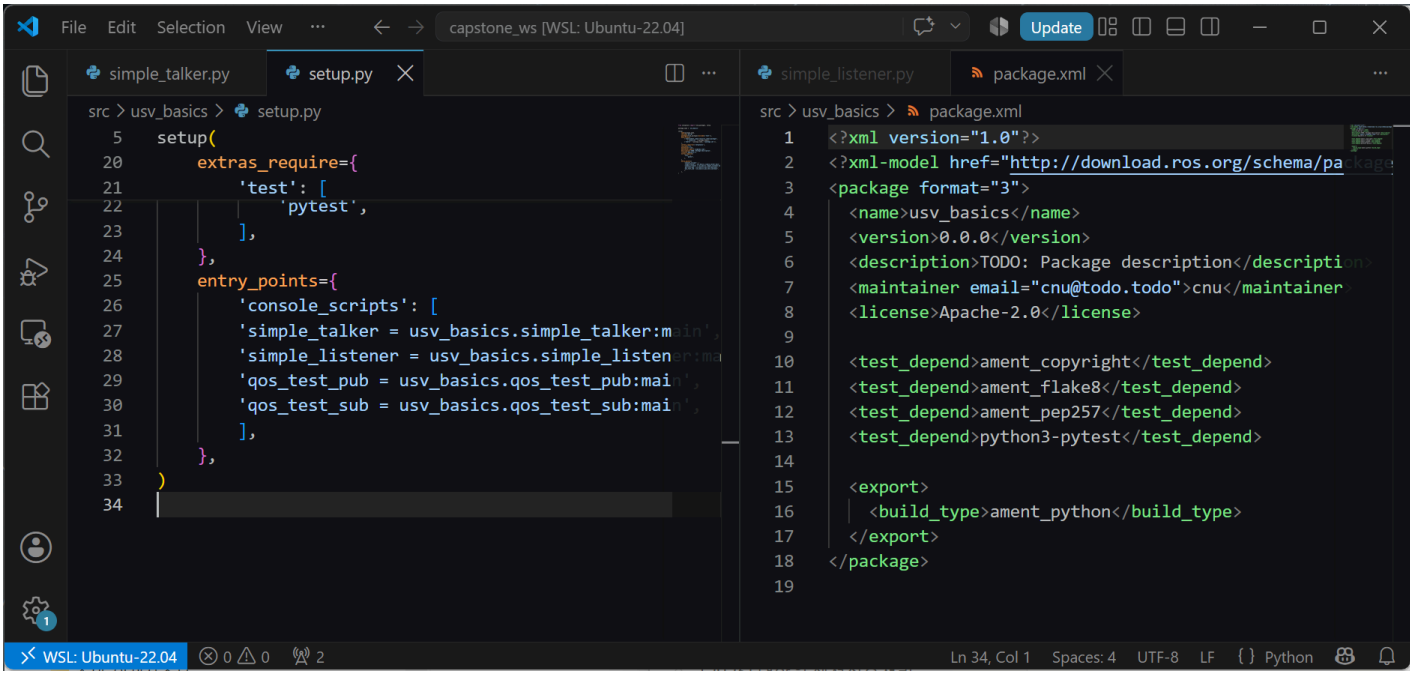
- `rclpy.spin()` 이 없으면 프로그램이 **즉시 끝남.** 콜백이 한 번도 안 불림

실행파일 등록

- VS Code 탐색기에서 `src/usv_basics/setup.py` 를 **클릭해서 엶**
  - 단축키로 열려면 `Ctrl + P` → `setup.py` 입력 → `Enter`
- 파일 아래쪽의 `entry_points` 부분을 아래로 수정

```
entry_points={
    'console_scripts': [
        'simple_talker = usv_basics.simple_talker:main',
        'simple_listener = usv_basics.simple_listener:main',
    ],
},
```

- 아래는 §2-9 까지 마친 뒤의 실제 화면 (`entry_points` 4줄, 오른쪽은 `package.xml`)



두 설정 파일이 하는 일

파일	누가 읽는가	무엇을 정하는가
`package.xml`	**ROS 2 / colcon**	패키지 이름 · 의존성 · 라이선스 · 빌드 타입
`setup.py`	**파이썬(setuptools)**	어떤 `.py` 가 **실행파일**이 되는가, 어디에 설치되는가

- `package.xml` 을 읽는 법 (위 화면 오른쪽)

줄	태그	뜻
3	`<package format="3">`	ROS 2 의 패키지 명세 **3판**. ROS 1은 2판
4	`<name>usv_basics</name>`	**`ros2 run` 의 첫 인자.** 폴더 이름과 같아야 함
8	`<license>Apache-2.0</license>`	비우면 빌드 경고
10~13	`<test_depend>`	자동 생성된 검사 도구. 지우지 않음
15~17	`<build_type>ament_python</build_type>`	**파이썬 패키지**라는 선언. C++ 이면 `ament_cmake`

- `setup.py` 를 읽는 법 (위 화면 왼쪽)

줄	항목	뜻
3	`package_name = 'usv_basics'`	아래에서 계속 쓰이는 이름
8	`packages=find_packages(exclude=['test'])`	어떤 폴더를 파이썬 모듈로 볼지
10~12	`data_files=[...]`	`resource/` 표식과 `package.xml` 을 설치 경로로 복사
19	`license='Apache-2.0'`	`package.xml` 과 **같게** 맞춤
25~31	**`entry_points`**	**여기가 핵심.** 실행파일 목록

- `entry_points` 한 줄의 구조

```
'simple_talker = usv_basics.simple_talker:main'
 └ 실행 이름      └ 패키지  └ 파일     └ 함수
```

조각	대응하는 것	틀리면
`simple_talker` (등호 왼쪽)	`ros2 run usv_basics simple_talker` 의 마지막 인자	`No executable found`
`usv_basics.simple_talker`	`usv_basics/simple_talker.py` (`.py` 없이)	`ModuleNotFoundError`
`:main`	그 파일의 `def main()`	`AttributeError`

> **✕ 쉼표와 따옴표를 빠뜨리면 빌드가 통째로 실패함**
>
> `console_scripts` 목록은 파이썬 리스트임. **각 줄 끝에 쉼표**가 있어야 함
> 저장 후 VS Code 왼쪽 아래의 `✕ 0` 을 확인하고 빌드할 것.

빌드 및 실행

```
cd ~/capstone_ws
colcon build --symlink-install
source install/setup.bash
```

- 정상 출력 (기준 환경 실측)

```
Starting >>> usv_basics
Finished <<< usv_basics [0.56s]

Summary: 1 package finished [0.71s]
```

- 등록이 제대로 됐는지 확인

```
ros2 pkg executables usv_basics
```

- 정상 출력 (이 시점에는 두 줄. §2-9 를 마치면 `qos_test_pub` · `qos_test_sub` 가 더해진다)

```
usv_basics simple_listener
usv_basics simple_talker
```

- VS Code 통합 터미널을 **좌우로 나눔** (터미널 패널 오른쪽 위 분할 버튼)
- **왼쪽 터미널**

```
ros2 run usv_basics simple_talker
```

- 정상 출력 (기준 환경 실측)

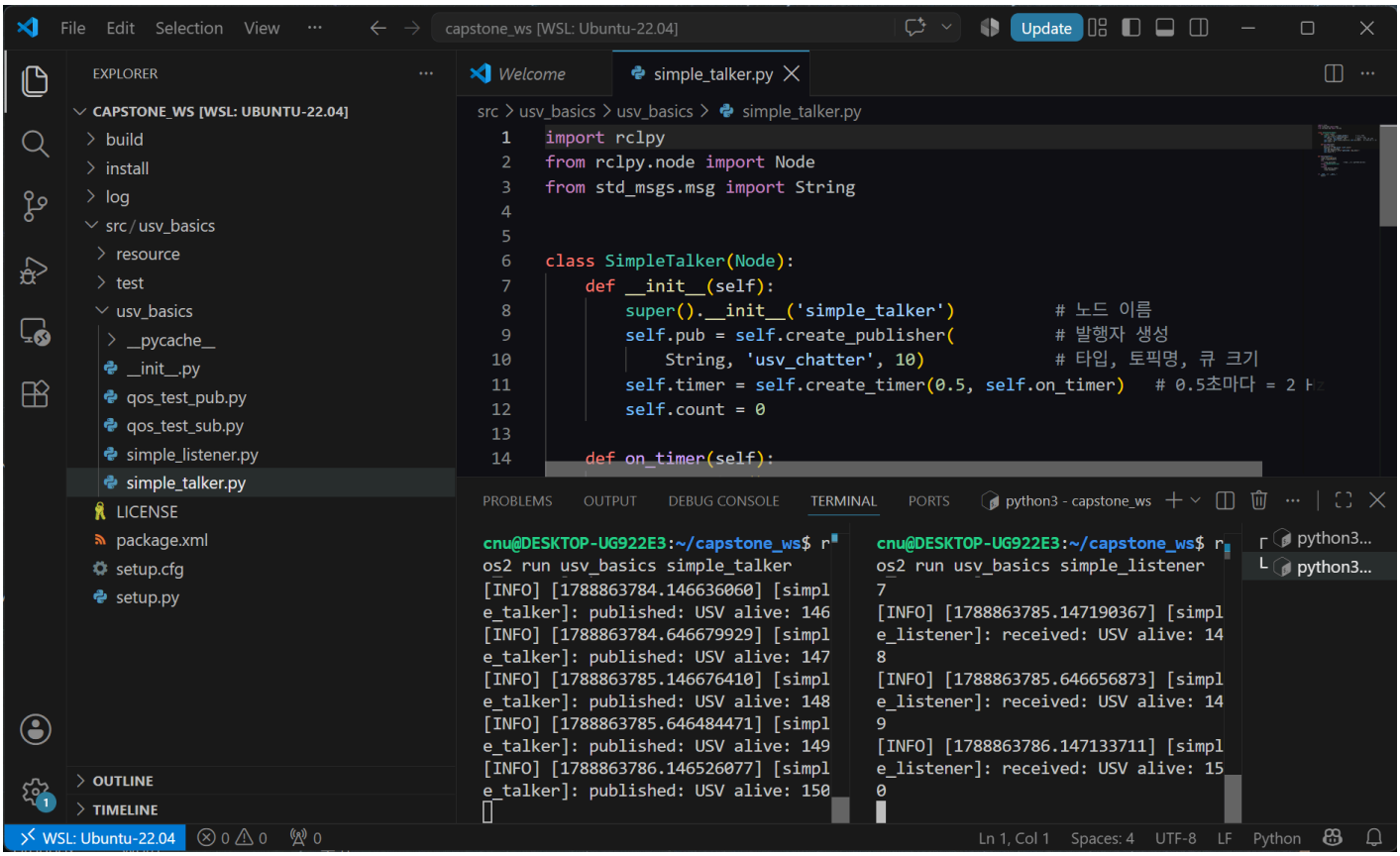
```
[INFO] [1788862815.369895386] [simple_talker]: published: USV alive: 0
[INFO] [1788862815.860327737] [simple_talker]: published: USV alive: 1
[INFO] [1788862816.361112878] [simple_talker]: published: USV alive: 2
```

- **오른쪽 터미널**

```
ros2 run usv_basics simple_listener
```

- 정상 출력 (기준 환경 실측)

```
[INFO] [1788862815.370363776] [simple_listener]: received: USV alive: 0
[INFO] [1788862815.860455555] [simple_listener]: received: USV alive: 1
[INFO] [1788862816.361306687] [simple_listener]: received: USV alive: 2
```



> **★ 이 화면이 이번 주차의 통과 기준**
> 두 터미널의 숫자가 **같은 값으로 나란히** 올라가야 함

- 주기를 수치로 확인 (세 번째 터미널)

```
ros2 topic hz /usv_chatter
```

- 정상 출력 (기준 환경 실측) — 코드의 `create_timer(0.5, ...)` 와 일치

```
average rate: 2.000
    min: 0.500s max: 0.500s std dev: 0.00022s window: 8
```

확인 항목	기대값	실측
발행 주기	2 Hz (`0.5 s`)	`average rate: 2.000`
간격 흔들림	작을수록 좋음	`std dev: 0.00022 s`
발행–수신 지연	1 ms 수준	로그 시각 차 `0.0001~0.0005 s`

2-8. 화면으로 보는 도구 — rqt 와 RViz2

> **★ 명령줄만으로는 "누가 누구에게" 를 못 봄**
>
> `ros2 topic list` 는 토픽 **이름**만 줌. 연결 관계·값의 변화·경고 로그는
> 아래 네 도구로 봄. 전부 `ros-humble-desktop` 에 이미 들어 있음

도구	무엇을 보는가	실행 명령
rqt_graph	노드–토픽 연결 관계	`ros2 run rqt_graph rqt_graph`
Topic Monitor	토픽의 주기·대역폭·**현재 값**	`ros2 run rqt_topic rqt_topic`
rqt_console	모든 노드의 로그를 한곳에서	`ros2 run rqt_console rqt_console`
RViz2	좌표계·센서 데이터의 **3차원 시각화**	`rviz2`

> **ℹ 창은 Windows 에 뜨지만 실행되는 곳은 우분투**
>
> WSLg 가 우분투의 창을 Windows 화면에 그려 줌. 1주차에서 `xeyes` 로 확인한 그 기능임
> 창이 안 뜨면 GUI 문제이지 ROS 문제가 아님 — 1주차 2-4절로 돌아감

준비 — 노드를 두 개 띄워 둔다

- 아래 실습은 §2-7 에서 만든 노드를 켠 상태를 가정함. 아직이면 데모 노드로 대신함

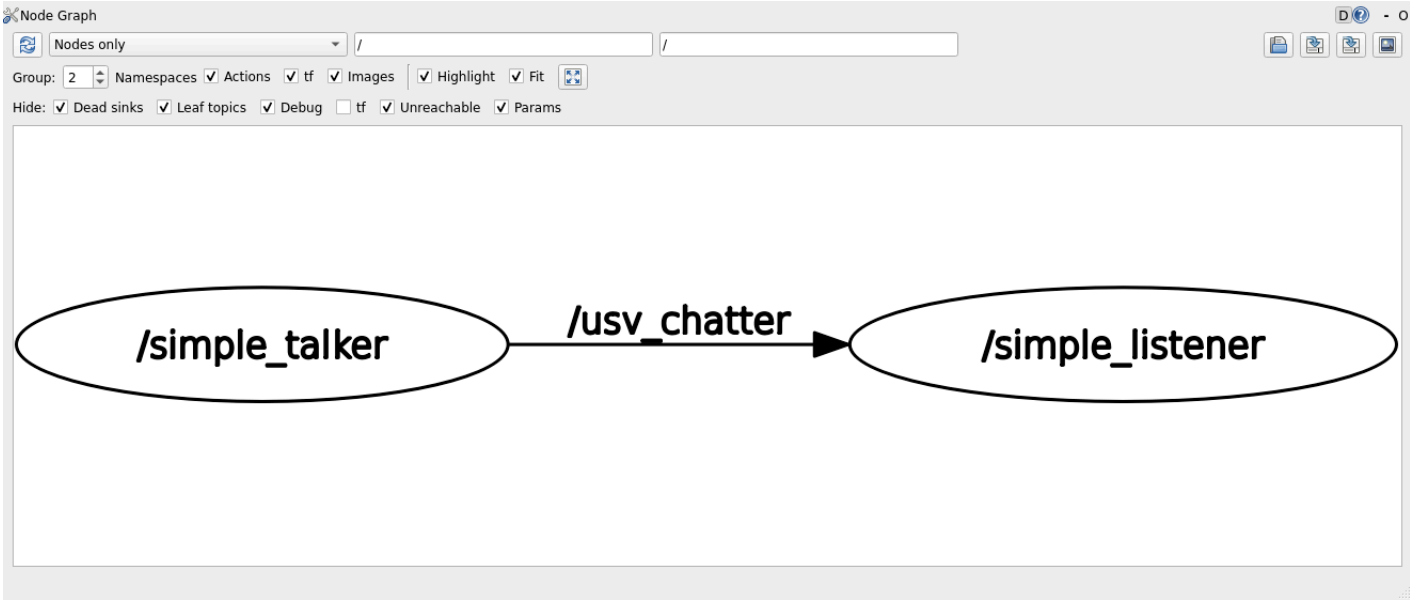
```
ros2 run demo_nodes_cpp talker
```

```
ros2 run demo_nodes_py listener
```

---

① rqt_graph — 연결 관계

```
ros2 run rqt_graph rqt_graph
```



- 위 화면은 §2-7 의 `simple_talker` · `simple_listener` 를 실제로 띄우고 캡처한 것

모양	뜻
타원	노드 (`/simple_talker` , `/simple_listener`)
화살표 위의 글자	토픽 (`/usv_chatter`)
화살표 방향	데이터가 흐르는 방향

- 화면 위쪽 조작

위치	하는 일
왼쪽 위 **파란 회전 화살표**	새로고침. **자동 갱신되지 않음**
`Nodes only` 드롭다운	`Nodes/Topics (all)` 로 바꾸면 토픽이 사각형으로 따로 보임
`Hide:` 체크박스들	체크된 항목을 **숨김.** 기본값(모두 체크)을 유지하면 화면이 단순함

> ✎ **비어 있으면 세 가지를 의심함**
>
> 1. 노드가 죽었음 → `ros2 node list` 로 확인
> 2. 새로고침을 안 눌렀음 → 파란 화살표
> 3. `ROS_DOMAIN_ID` 가 다름 → `echo $ROS_DOMAIN_ID`

---

② Topic Monitor — 값이 실제로 바뀌는지

```
ros2 run rqt_topic rqt_topic
```



- **왼쪽 체크박스를 켜야** 측정이 시작됨. 켜지 않으면 `not monitored` 로 남음
- 토픽 이름 왼쪽 **삼각형**을 누르면 메시지 내부 필드까지 펼쳐짐

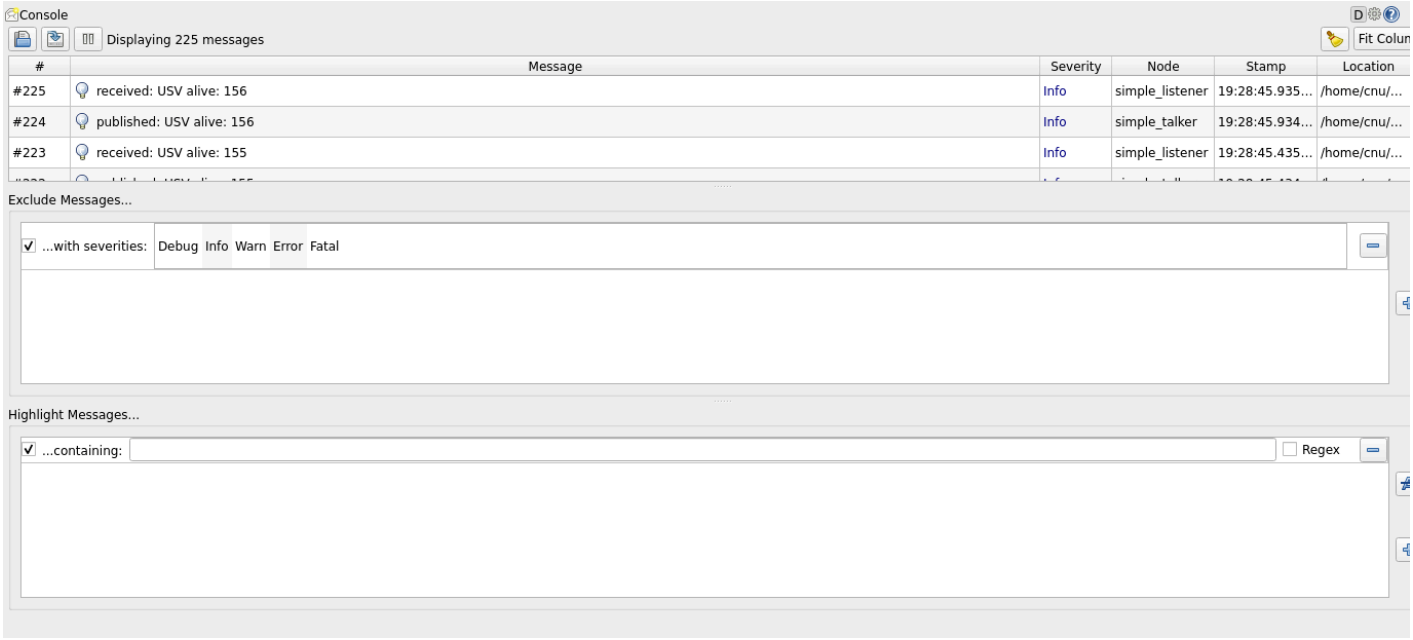
열	뜻	위 화면의 값
`Type`	메시지 타입	`std_msgs/msg/String`
`Hz`	초당 수신 횟수	**2.00** — `simple_talker` 가 0.5 s 주기이므로 일치
`Value`	현재 값	`'USV alive: 356'`

> ★ `Hz` 는 코드의 주기와 같은 값이 나와야 함
>
> `Hz` 가 코드의 주기(`create_timer(0.5)` → 2.0)와 다르면 **발행자가 둘 이상** 켜져 있는 것임
> `ros2 topic hz` 도 구독자이므로 같은 값(두 배)을 보여 줌. 비교 기준은 코드
> 실제로 `simple_talker` 를 두 번 실행하면 `Hz` 가 약 4.0 으로 찍힘

---

③ rqt_console — 로그를 한곳에서

```
ros2 run rqt_console rqt_console
```



- 모든 노드가 `/rosout` 으로 보낸 로그가 여기에 모임

열	뜻
`#`	도착 순서
`Message`	로그 내용
`Severity`	`Debug` · `Info` · `Warn` · `Error` · `Fatal`
`Node`	어느 노드가 찍었는가
`Stamp`	시각

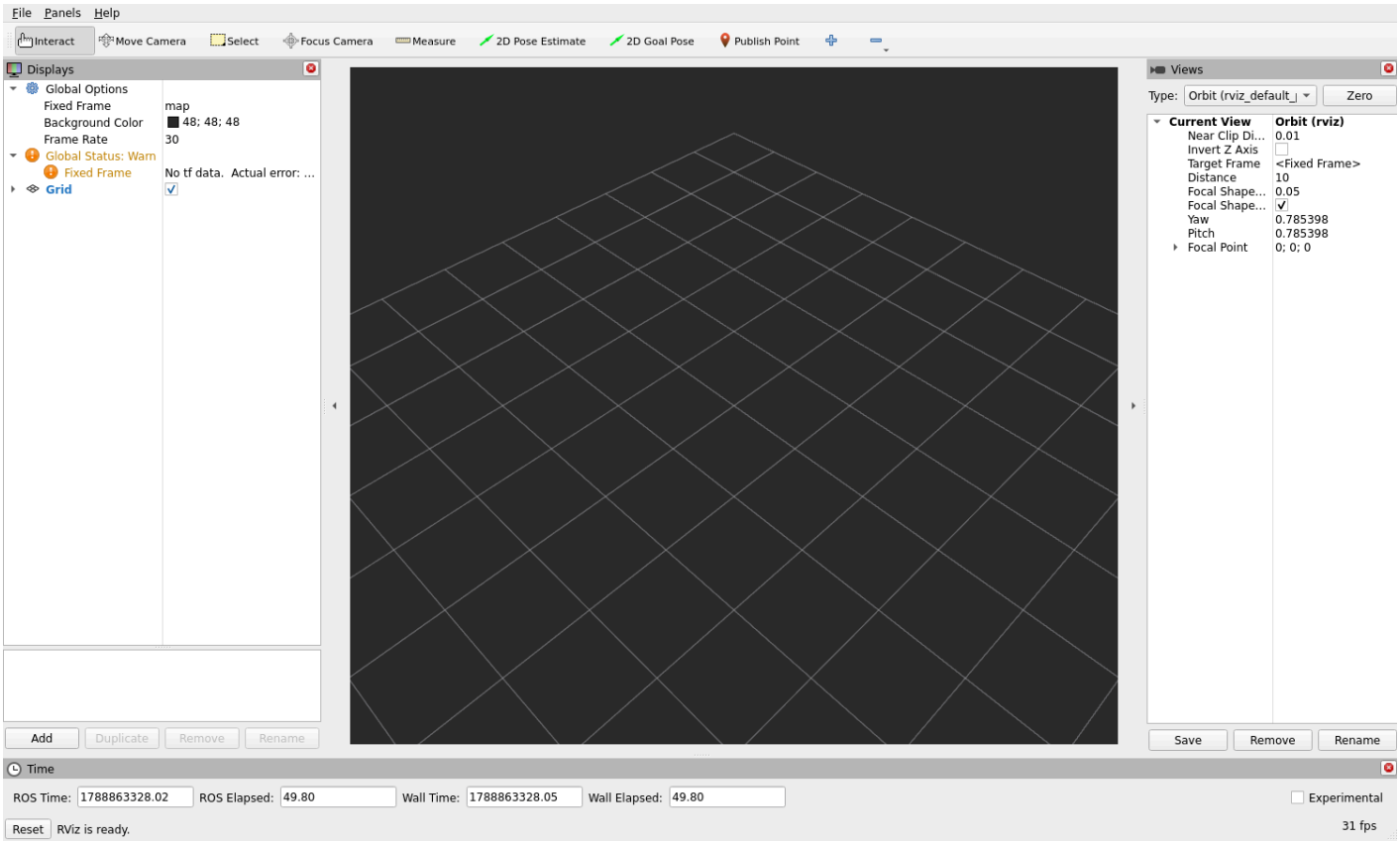
- 가운데 **Exclude Messages** 에서 `Info` 를 눌러 끄면 **경고만** 남음

> **★ QoS 경고를 찾는 가장 확실한 방법**
> §2-9 의 QoS 불일치 경고는 `[INFO]` 홍수에 묻혀 터미널에서 놓치기 쉬움
> 여기서 `Severity` 를 `Warn` 으로 걸러 보면 한눈에 보임

---

④ RViz2 — 3차원으로 보는 도구

```
rviz2
```



- 처음 켜면 **격자(Grid)만** 있는 빈 화면이 정상. 아직 보여 줄 데이터가 없음

화면의 위치	이름	하는 일
왼쪽 위	**Displays**	무엇을 그릴지 목록. `Add` 로 추가
가운데	**3D 뷰**	마우스 왼쪽 드래그 = 회전, 휠 = 확대
오른쪽	**Views**	카메라 종류와 거리
아래	**Time**	ROS 시각과 실제 시각
맨 아래 왼쪽	상태 문구	`RViz is ready.` 가 나오면 정상

- 왼쪽 `Global Status: Warn` 과 `Fixed Frame` 의 `No tf data` 는 **지금은 정상**
  - 좌표계(TF)를 발행하는 노드가 아직 없기 때문
  - 3주차에서 VRX 를 띄우면 이 경고가 사라짐

> ℹ **이번 주차에는 띄워 보는 것까지가 목표**
>
> RViz2 를 제대로 쓰는 것은 **3주차(좌표계·TF)** 와 **10주차(센서 토픽)** 의 내용임
> 지금은 "이런 도구가 있고, 실행하면 이런 화면이 나온다" 를 확인함

네 도구를 한 번에 확인하는 순서

1. `ros2 run demo_nodes_cpp talker` 를 켬
2. `rqt_graph` → 타원 하나가 보임
3. `ros2 run demo_nodes_py listener` 를 켬 → **새로고침** → 타원 둘과 화살표
4. `rqt_topic` → `/chatter` 체크 → `Hz` 가 약 1.0
5. `rqt_console` → `Publishing:` 로그가 쌓임
6. `rviz2` → 격자 화면과 `RViz is ready.`

2-9. QoS 불일치 재현 실험

> **★ 이번 주차 실습에서 가장 중요한 부분**
> 이 유의 사항을 지금 손으로 만들어 봐야, 실선 LiDAR·카메라 드라이버(흔히 BEST_EFFORT)를 만났을 때 스스로 알아챔. VRX 는 전부 RELIABLE 이라 시뮬레이터에서는 걸리지 않음 (10주차 2-3 절)

발행자 — BEST_EFFORT

- VS Code 탐색기에서 `src/usv_basics/usv_basics/` 에 `qos_test_pub.py` 를 새로 만듦

```
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from std_msgs.msg import String


class QosTestPub(Node):
    def __init__(self):
        super().__init__('qos_test_pub')
        qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,   # <-- 여기가 핵심
            history=HistoryPolicy.KEEP_LAST,
            depth=10,
        )
        self.pub = self.create_publisher(String, 'qos_topic', qos)
        self.create_timer(0.5, self.on_timer)

    def on_timer(self):
        msg = String()
        msg.data = 'best effort message'
        self.pub.publish(msg)
        self.get_logger().info('published')


def main():
    rclpy.init()
    node = QosTestPub()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

구독자 — RELIABLE (더 엄격)

- 같은 폴더에 `qos_test_sub.py` 를 새로 만듦

```
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from std_msgs.msg import String


class QosTestSub(Node):
    def __init__(self):
        super().__init__('qos_test_sub')
        qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,      # <-- 발행자보다 엄격
            history=HistoryPolicy.KEEP_LAST,
            depth=10,
        )
        self.sub = self.create_subscription(
            String, 'qos_topic', self.on_msg, qos)

    def on_msg(self, msg):
        self.get_logger().info(f'received: {msg.data}')


def main():
    rclpy.init()
    node = QosTestSub()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

- `setup.py` 의 `entry_points` 에 두 줄 추가

```
'qos_test_pub = usv_basics.qos_test_pub:main',
'qos_test_sub = usv_basics.qos_test_sub:main',
```

- 빌드

```
cd ~/capstone_ws && colcon build --symlink-install && source install/setup.bash
```

관찰 순서

1. 두 노드를 각각 실행 — VS Code 통합 터미널을 좌우로 나눔

```
ros2 run usv_basics qos_test_pub
```

- 발행자 쪽 출력 (기준 환경 실측) — 구독자를 켜는 순간 발행자 화면에 **경고 한 줄**이 끼어듦
  - 아래는 두 노드를 거의 동시에 켠 경우라 경고가 첫 줄임. 발행자를 먼저 켜면 `published` 여러 줄 뒤에 나옴

```
[WARN] [1788862828.210020463] [qos_test_pub]: New subscription discovered on topic 'qos_topic', reques
ting incompatible QoS. No messages will be sent to it. Last incompatible policy: RELIABILITY
[INFO] [1788862828.697631209] [qos_test_pub]: published
[INFO] [1788862829.197384841] [qos_test_pub]: published
```

```
ros2 run usv_basics qos_test_sub
```

- 구독자 쪽 출력 (기준 환경 실측) — **경고 한 줄뿐이고 그 뒤가 없음**

```
[WARN] [1788862828.210407869] [qos_test_sub]: New publisher discovered on topic 'qos_topic', offering
incompatible QoS. No messages will be received from it. Last incompatible policy: RELIABILITY
```

관찰	사실
발행자는 계속 `published` 를 찍음	보내는 쪽은 정상 동작함
구독자는 `received` 가 **한 줄도 없음**	8초 실행 중 수신 **0건** (실측)
프로그램이 죽지 않음	예외도 없고 종료도 안 함
경고 한 줄이 끼어 있음	이것을 못 보고 지나가는 것이 문제

> ✎ **경고를 놓쳤다면 `rqt_console` 로 봄**
> §2-8 의 `rqt_console` 에서 **Exclude Messages → `Info` 를 끄면** 경고만 남음

2. 진단

```
ros2 topic info /qos_topic --verbose
```

- 출력에서 `Endpoint type: PUBLISHER` 와 `SUBSCRIPTION` 각각의 `Reliability` 를 비교
- 정상 출력 (2026-09-18 실측, `GID` 는 실행마다 다름)

```
Type: std_msgs/msg/String

Publisher count: 1

Node name: qos_test_pub
Node namespace: /
Topic type: std_msgs/msg/String
Endpoint type: PUBLISHER
GID: 01.0f.45.06.fd.20.ad.f2.00.00.00.00.00.00.11.03.00.00.00.00.00.00.00.00
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  ...

Subscription count: 1

Node name: qos_test_sub
Node namespace: /
Topic type: std_msgs/msg/String
Endpoint type: SUBSCRIPTION
GID: 01.0f.45.06.fc.20.c1.02.00.00.00.00.00.00.11.04.00.00.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  ...
```

- 발행자는 `BEST_EFFORT`, 구독자는 `RELIABLE` — 서로 다름
- `Publisher count: 1` 인지도 봄. 2 이상이면 발행자를 두 번 띄운 것이라 아래 줄 수가 두 배로 나옴

3. 수정

1. VS Code 에서 `qos_test_sub.py` 를 엶
2. `ReliabilityPolicy.RELIABLE` 을 `ReliabilityPolicy.BEST_EFFORT` 로 바꿈
3. `Ctrl + S` 로 저장

```
cd ~/capstone_ws && colcon build --symlink-install && source install/setup.bash
```

4. 두 노드를 다시 실행

- 고친 뒤 구독자 출력 (기준 환경 실측)

```
[INFO] [1789718301.477341524] [qos_test_sub]: received: best effort message
[INFO] [1789718301.980649150] [qos_test_sub]: received: best effort message
[INFO] [1789718302.480484697] [qos_test_sub]: received: best effort message
```

- 시각 도장의 간격이 약 0.5 s — 발행자의 타이머 `create_timer(0.5, ...)` 와 같음

상태	8초 동안 구독자가 받은 줄 수 (실측)
불일치 — 발행 `BEST_EFFORT` / 구독 `RELIABLE`	**0**
일치 — 발행 `BEST_EFFORT` / 구독 `BEST_EFFORT`	**16**

- 2026-09-18 재측정, 발행자 하나 (`Publisher count: 1`). 2 Hz × 8 s = 16 과 맞음
  - 예전 판의 35 는 발행자가 둘 떠 있던 상태에서 잰 값이었음 (간격 0.25 s)

> **★ 0 과 16 의 차이를 만든 것은 코드 한 단어**
> 알고리즘도 배선도 바뀌지 않았음. QoS 정책 이름 하나만 바뀌었음

> **⚠ 3주차에서 다시 다룸**
> 실선의 LiDAR·카메라 드라이버는 `BEST_EFFORT` 로 발행하는 경우가 많음
> **다만 VRX 의 `ros_gz_bridge` 토픽은 전부 `RELIABLE` 임** (2026-09-15 실측 25개 전수조사)
> 그래서 외우지 말고 `ros2 topic info <토픽> --verbose` 로 **매번 확인**하는 습관을 들임
> `ros2 topic echo` 로는 보이는데 직접 작성한 노드만 못 받으면 **가장 먼저 QoS를 의심**할 것.

2-10. 중계 노드 — MATLAB 이 모르는 메시지를 표준 형식으로 옮긴다

> **★ 이 절에서 만드는 것**
> - `/turtle1/pose` 를 받아 **MATLAB 에 내장된 메시지 형식**으로 다시 발행하는 노드 하나
> - 4주차부터 Simulink 가 ROS 2 와 통신함. 그때 **"토픽은 보이는데 형식이 없다"** 는 문제를 반드시 만남
> - 해결 방법은 한 가지 — **중계 노드**. 이번 주차에 만든 노드 작성법 그대로 씀

용어 정리

용어	뜻	비유
메시지 형식 (message type)	토픽에 실리는 데이터의 구조. `turtlesim/msg/Pose` 처럼 `패키지/msg/이름`	서류 양식
내장 형식	MATLAB ROS Toolbox 가 처음부터 알고 있는 형식. MATLAB 명령 창의 `ros2 msg list` 로 확인 (우분투의 `ros2 interface list` 와 다름)	이미 인쇄해 둔 양식
중계 노드 (relay)	한 토픽을 받아 **다른 형식**으로 옮겨 담아 다시 내보내는 노드	서류를 다른 양식에 옮겨 적는 직원

문제 상황 — 토픽은 보이는데 받을 수 없다

- turtlesim 을 띄운 상태의 토픽 목록 (기준 환경 실측)

```
ros2 topic list -t
```

- 정상 출력

```
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

- MATLAB 명령 창에서 `/turtle1/pose` 를 구독하면 아래 오류가 남 (MATLAB R2024b 실측)

```
node = ros2node("/probe", 7);   % 두 번째 인자 = 자기 팀 Domain ID
sub  = ros2subscriber(node, "/turtle1/pose", "turtlesim/Pose");
```

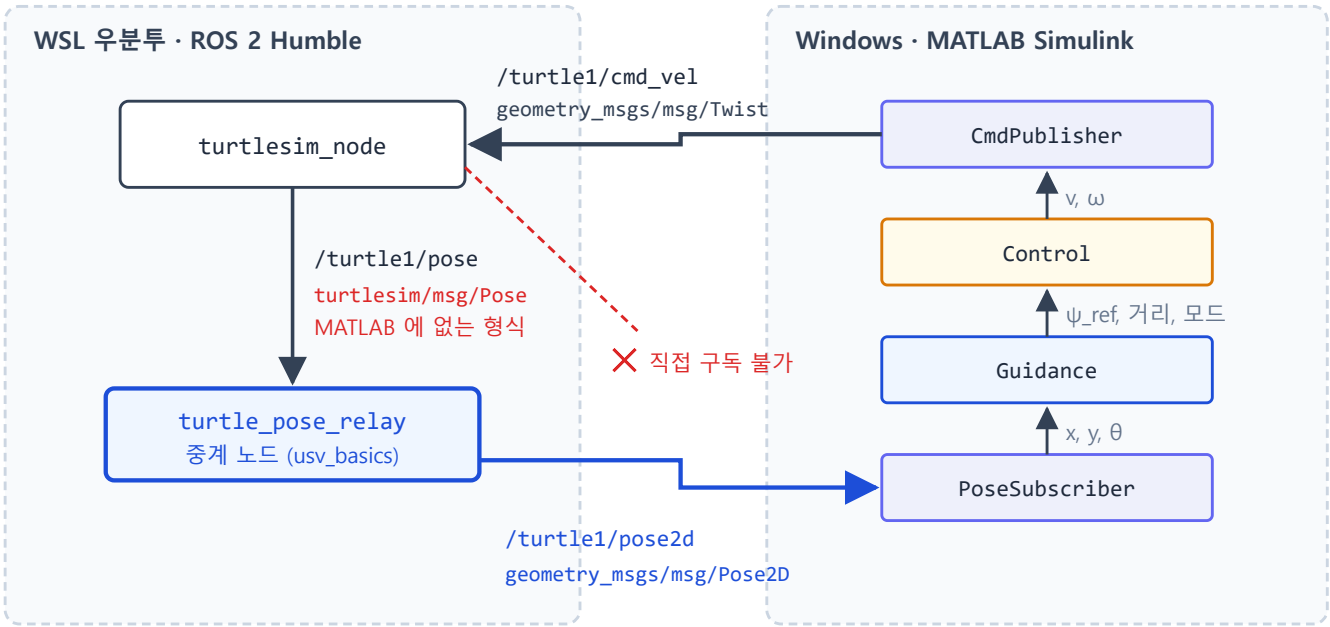
```
turtlesim/Pose은(는) 인식할 수 없는 메시지 유형입니다. 사용 가능한 유형을 보려면 ros2 msg list을(를) 사용하십시오.
```

메시지 형식	MATLAB R2024b 내장 여부 (MATLAB `ros2 msg list` 358종 중)
`turtlesim/msg/Pose`	**없음**
`geometry_msgs/msg/Pose2D`	있음
`geometry_msgs/msg/Twist`	있음 — `/turtle1/cmd_vel` 이 이 형식이라 **보내기는 바로 됨**

> **ℹ 없는 형식을 MATLAB 에 추가하는 방법도 있음**
>
> `ros2genmsg` 로 사용자 메시지를 빌드하면 됨. 다만 Python·CMake·C++ 컴파일러가 Windows 에 따로 필요해 설치가 무거움
> 본 과목은 **중계 노드**를 씀 — 우분투 안에서 파이썬 파일 하나로 끝남

해법 — 중계 노드가 가운데에 선다

turtlesim 과 Simulink 사이의 토픽 흐름



중계 노드는 값을 바꾸지 않음 — 같은 x, y, θ 를 MATLAB 에 내장된 형식으로 옮겨 담을 뿐임

그림의 위치	무엇인가
왼쪽 위 `turtlesim_node` → 아래 화살표	turtlesim 이 자기 형식(`turtlesim/msg/Pose`)으로 자세를 냄
빨간 점선 ✕ **직접 구독 불가**	MATLAB 은 이 형식을 모름
파란 상자 `turtle_pose_relay`	이 절에서 만드는 노드. **같은 값을 표준 형식에 옮겨 담음**
파란 화살표 `/turtle1/pose2d`	MATLAB 이 아는 `geometry_msgs/msg/Pose2D`
오른쪽 네 상자	§2-11 의 Simulink 모델. 받고 → 판단하고 → 제어하고 → 보냄
위쪽 `/turtle1/cmd_vel`	원래 표준 형식(`Twist`)이라 중계가 필요 없음

두 형식을 나란히 본다

```
ros2 interface show turtlesim/msg/Pose
```

- 정상 출력

```
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
```

```
ros2 interface show geometry_msgs/msg/Pose2D
```

- 정상 출력 (앞쪽 `#` 주석 줄 생략)

```
float64 x
float64 y
float64 theta
```

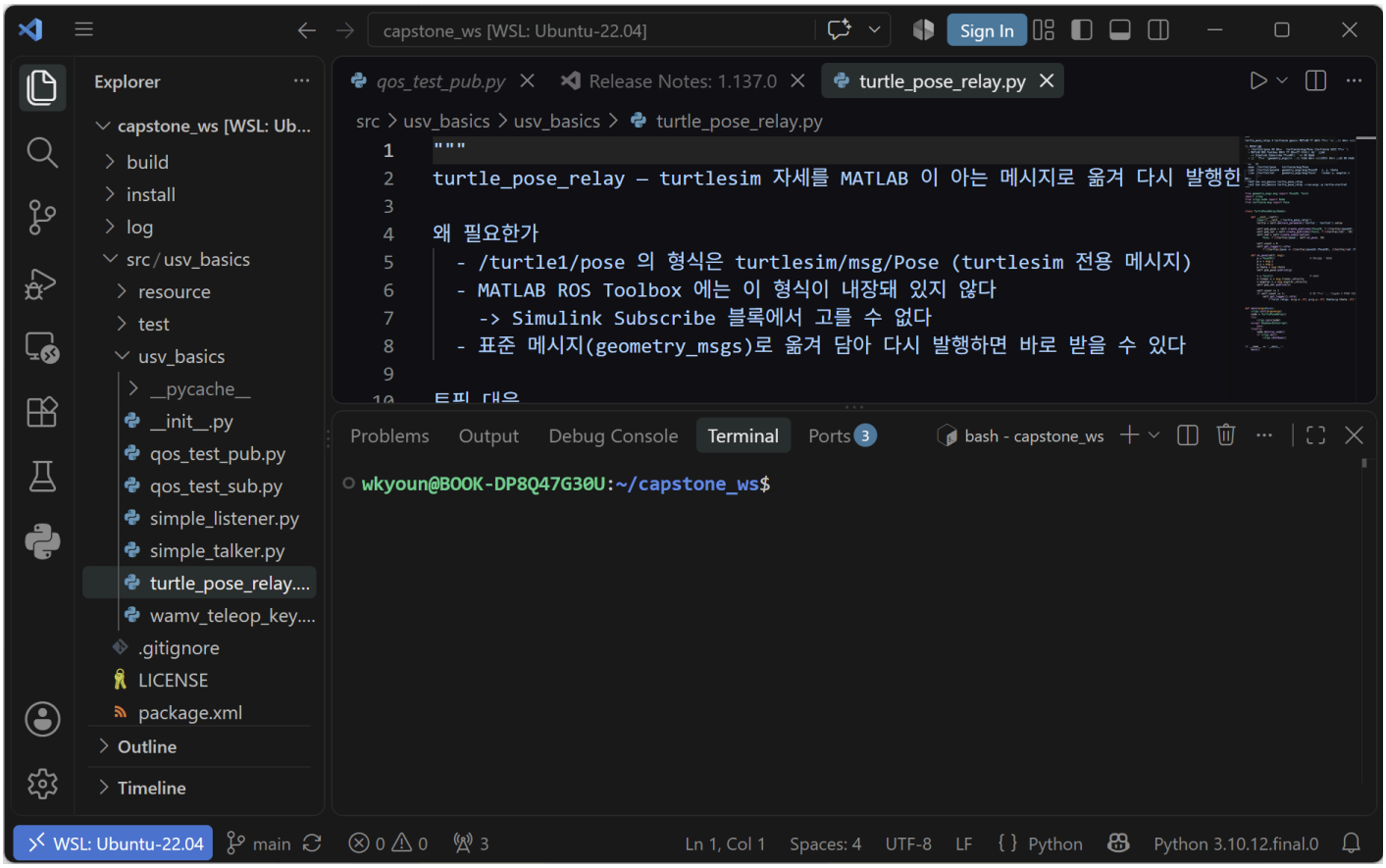
`turtlesim/msg/Pose`	→	옮겨 담는 곳
`x` · `y` · `theta`	→	`/turtle1/pose2d` (`Pose2D`) 의 `x` · `y` · `theta`
`linear_velocity`	→	`/turtle1/vel` (`Twist`) 의 `linear.x`
`angular_velocity`	→	`/turtle1/vel` (`Twist`) 의 `angular.z`

- `float32` → `float64` 는 파이썬이 알아서 넓혀 줌. 변환 코드가 필요 없음

> ℹ `Pose2D` 주석에 "Deprecated as of Foxy" 가 보임
> 3차원 `Pose` 를 권장한다는 안내일 뿐, Humble 에서 정상 동작함. MATLAB 에도 내장돼 있음
> 평면에서 움직이는 거북이에는 `x, y, theta` 세 값이 그대로 대응하는 `Pose2D` 가 가장 읽기 쉬움

1단계 — 파일을 만든다

- 위치: `~/capstone_ws/src/usv_basics/usv_basics/turtle_pose_relay.py`
- §2-7 과 같은 방법으로 VS Code 탐색기에서 새 파일을 만듦



화면에서 확인할 것	무엇
탐색기 `src / usv_basics` → `usv_basics` 아래 `turtle_pose_relay.py`	**안쪽** 모듈 폴더에 있어야 함 (§2-7 경고와 같다)
탭 제목 `turtle_pose_relay.py` · 위쪽 경로 `src > usv_basics > usv_basics`	올바른 위치
왼쪽 아래 `WSL: Ubuntu-22.04` · `✕ 0 ⚠ 0`	우분투 안의 파일이고 오류가 없음

- 아래 내용을 붙여넣고 저장 (`Ctrl + S`)

```
"""
turtle_pose_relay — turtlesim 자세를 MATLAB 이 아는 메시지로 옮겨 다시 발행한다.

왜 필요한가
  - /turtle1/pose 의 형식은 turtlesim/msg/Pose (turtlesim 전용 메시지)
  - MATLAB ROS Toolbox 에는 이 형식이 내장돼 있지 않다
    -> Simulink Subscribe 블록에서 고를 수 없다
  - 표준 메시지(geometry_msgs)로 옮겨 담아 다시 발행하면 바로 받을 수 있다

토픽 대응
  입력  /turtle1/pose    turtlesim/msg/Pose
  출력  /turtle1/pose2d  geometry_msgs/msg/Pose2D   x, y, theta
  출력  /turtle1/vel     geometry_msgs/msg/Twist    linear.x, angular.z

실행
  ros2 run usv_basics turtle_pose_relay
  ros2 run usv_basics turtle_pose_relay --ros-args -p turtle:=turtle2
"""

from geometry_msgs.msg import Pose2D, Twist
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose


class TurtlePoseRelay(Node):

    def __init__(self):
        super().__init__('turtle_pose_relay')
        turtle = self.declare_parameter('turtle', 'turtle1').value

        self.pub_pose = self.create_publisher(Pose2D, f'/{turtle}/pose2d', 10)
        self.pub_vel = self.create_publisher(Twist, f'/{turtle}/vel', 10)
        self.sub = self.create_subscription(
            Pose, f'/{turtle}/pose', self.on_pose, 10)

        self.count = 0
        self.get_logger().info(
            f'/{turtle}/pose -> /{turtle}/pose2d (Pose2D), /{turtle}/vel (Twist)')

    def on_pose(self, msg):
        p = Pose2D()                        # 위치와 선수각
        p.x = msg.x
        p.y = msg.y
        p.theta = msg.theta
        self.pub_pose.publish(p)

        v = Twist()                         # 속도
        v.linear.x = msg.linear_velocity
```

```
        v.angular.z = msg.angular_velocity
        self.pub_vel.publish(v)

        self.count += 1
        if self.count == 1:                 # 첫 메시지만 알린다 — 연결 확인용
            self.get_logger().info(
                f'first relay: x={p.x:.3f} y={p.y:.3f} theta={p.theta:.3f}')


def main(args=None):
    rclpy.init(args=args)
    node = TurtlePoseRelay()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

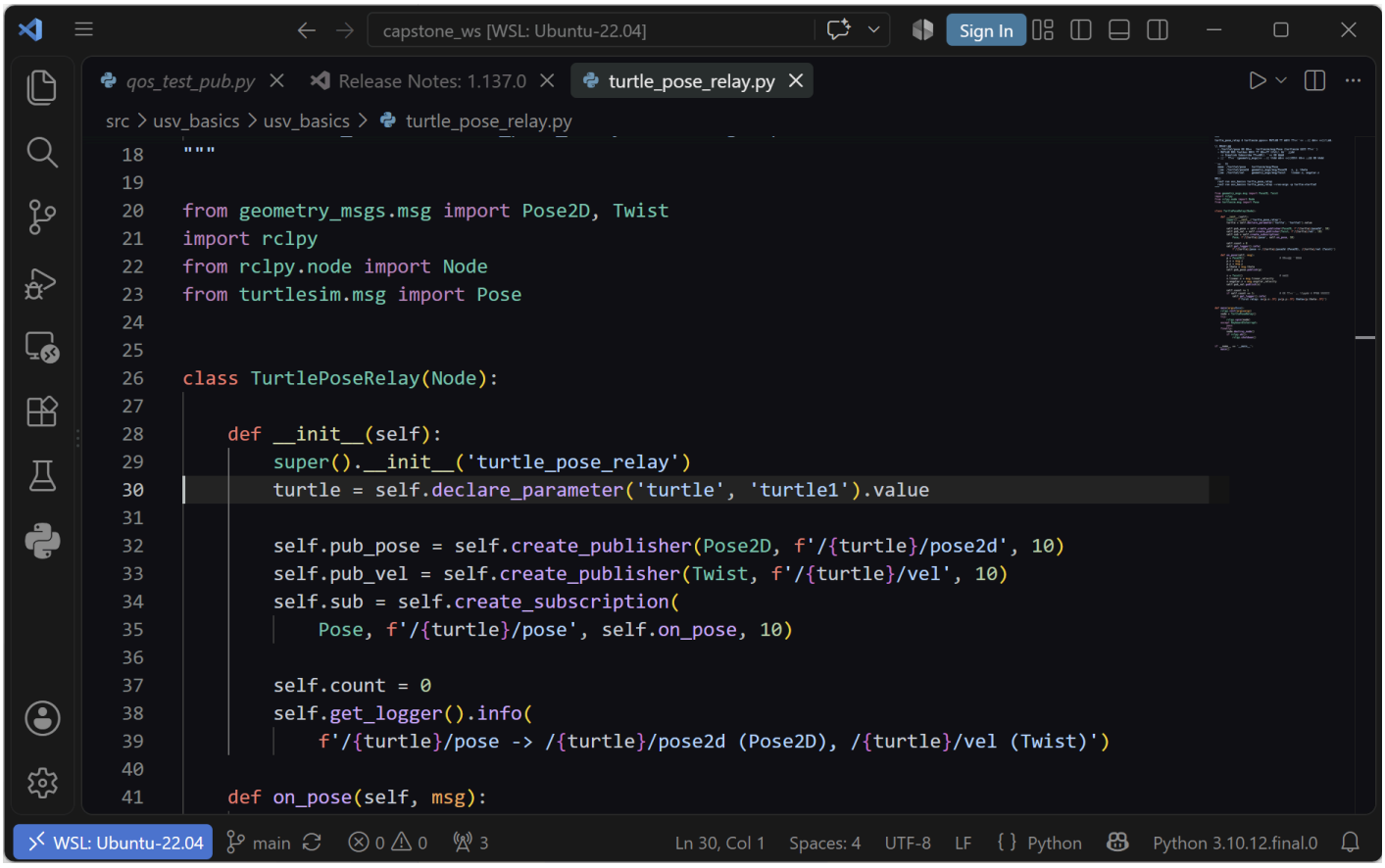
✎ 이미 `usv_basics` 저장소를 받아 둔 경우

저장소에 이 파일이 들어 있음. 아래 한 줄로 최신 상태를 받고 **3단계(빌드)** 로 건너뜀

```
cd ~/capstone_ws/src/usv_basics && git pull
```

- 출력에 `turtle_pose_relay.py` 가 들어 있으면 받아진 것

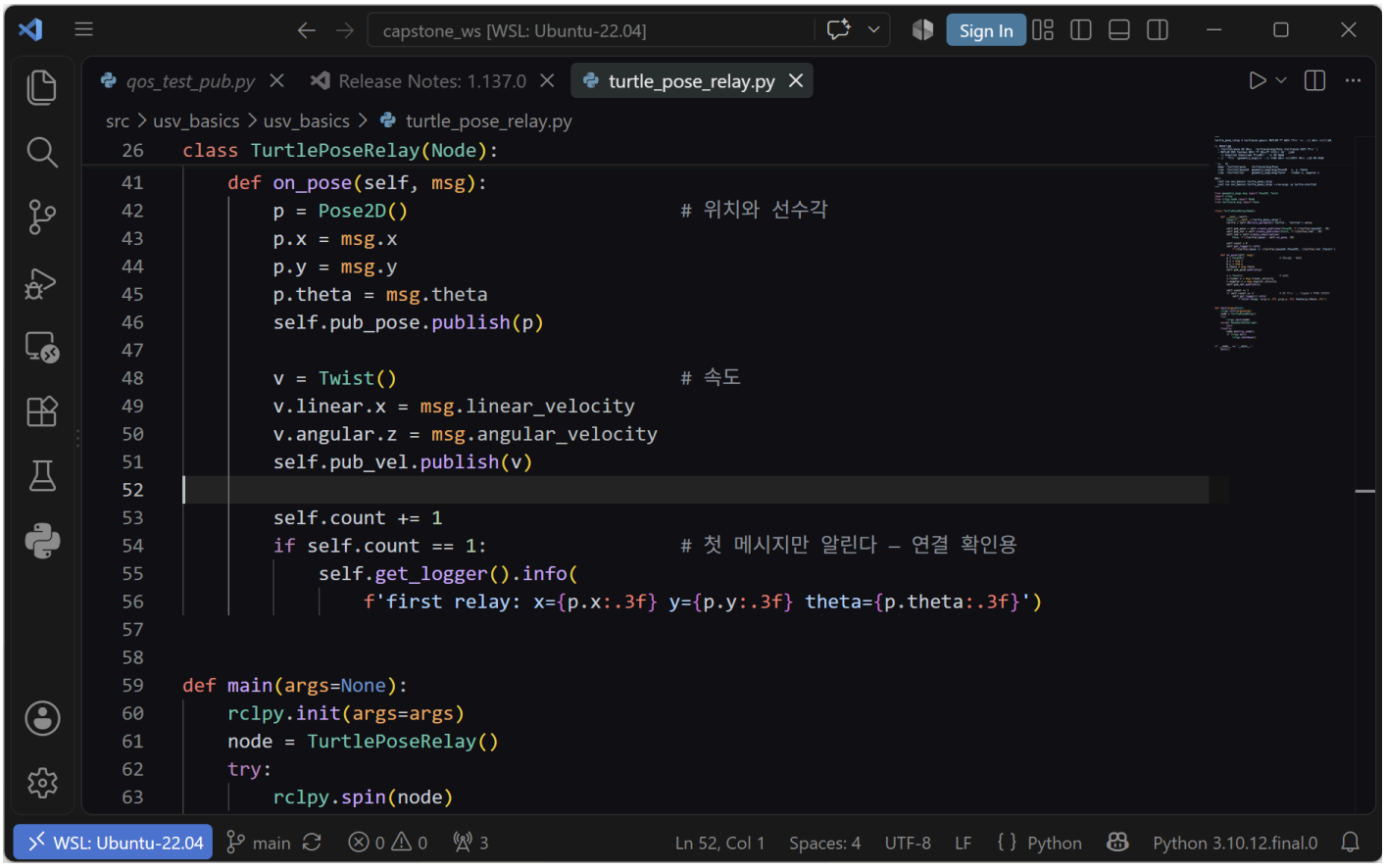
코드 읽기 ① — 발행자 둘, 구독자 하나



줄	코드	뜻
20	`from geometry_msgs.msg import Pose2D, Twist`	**내보낼** 두 형식. MATLAB 이 아는 형식
23	`from turtlesim.msg import Pose`	**받을** 형식. MATLAB 이 모르는 형식
29	`super().__init__('turtle_pose_relay')`	노드 이름. `ros2 node list` 에 이 이름이 뜸
30	`self.declare_parameter('turtle', 'turtle1')`	**파라미터**(§1-2). 기본값 `turtle1`, 실행할 때 바꿀 수 있음
32	`create_publisher(Pose2D, f'/{turtle}/pose2d', 10)`	발행자 ① — 위치와 선수각
33	`create_publisher(Twist, f'/{turtle}/vel', 10)`	발행자 ② — 속도
34~35	`create_subscription(Pose, f'/{turtle}/pose', self.on_pose, 10)`	구독자 — 메시지가 오면 `on_pose` 를 부름

- §2-7 의 `simple_talker`(발행자 1개)와 `simple_listener`(구독자 1개)를 **한 노드에 합친 모양**
- 차이는 **타이머가 없다**는 것 하나 — 발행은 시계가 아니라 **메시지 도착**이 일으킴

코드 읽기 ② — 받자마자 옮겨 담아 보낸다



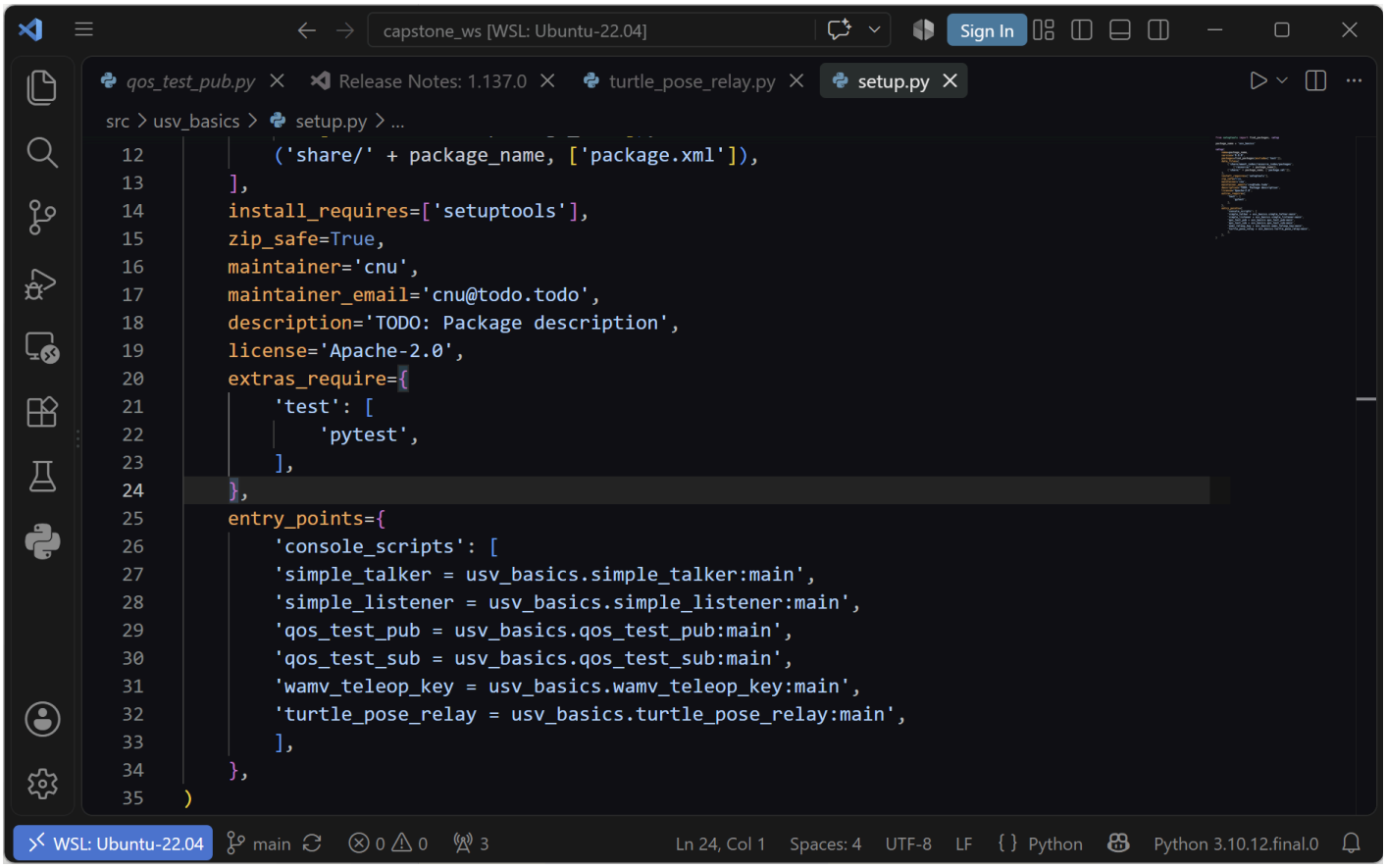
줄	코드	뜻
41	`def on_pose(self, msg)`	turtlesim 이 자세를 낼 때마다(약 62.5 Hz) 불림
42	`p = Pose2D()`	빈 `Pose2D` 양식을 만듦
43~45	`p.x = msg.x` …	**값을 그대로 옮겨 적음.** 계산이 없음
46	`self.pub_pose.publish(p)`	`/turtle1/pose2d` 로 내보냄
48~51	`v = Twist()` … `publish(v)`	속도도 같은 방식으로 `/turtle1/vel` 에
53~56	`if self.count == 1:`	**첫 메시지 한 번만** 로그를 찍음. 매번 찍으면 초당 62줄이 쏟아짐
59~69	`def main()`	§2-7 의 `main()` 과 같은 뼈대

> **★ 중계 노드는 값을 바꾸지 않음**
>
> 들어온 `x, y, theta` 와 나간 `x, y, theta` 는 같은 숫자. **형식(양식)만 바뀜**
> 그래서 중계 노드에서 계산 실수를 할 여지가 없음. 제어 계산은 전부 Simulink 쪽에서 함

2단계 — 실행파일 등록

- `setup.py` 의 `console_scripts` 에 한 줄 추가 (저장소 기준 32행)

```
'turtle_pose_relay = usv_basics.turtle_pose_relay:main',
```



화면에서 확인할 것	무엇
27~31행	저장소 기준 기존 실행파일 5개 — §2-7 · §2-9 의 4개 + 3주차 `wamv_teleop_key`. 직접 만든 경우는 4개
32행 `'turtle_pose_relay = usv_basics.turtle_pose_relay:main',`	이번에 추가한 줄. **끝의 쉼표** 확인

- `package.xml` 의 `<depend>` 에 두 줄 추가 — 이 노드가 쓰는 메시지 패키지

```
  <depend>geometry_msgs</depend>
  <depend>turtlesim</depend>
```

3단계 — 빌드

```
cd ~/capstone_ws
colcon build --symlink-install --packages-select usv_basics
source install/setup.bash
```

- 정상 출력 (기준 환경 실측)

```
Starting >>> usv_basics
Finished <<< usv_basics [0.56s]

Summary: 1 package finished [0.70s]
```

```
ros2 pkg executables usv_basics
```

- 정상 출력 — `turtle_pose_relay` 가 목록에 있어야 함

- 직접 만든 경우 5줄. 저장소에서 받았다면 3주차의 `wamv_teleop_key` 까지 6줄 (아래)

```
usv_basics qos_test_pub
usv_basics qos_test_sub
usv_basics simple_listener
usv_basics simple_talker
usv_basics turtle_pose_relay
usv_basics wamv_teleop_key
```

4단계 — 실행 (터미널 두 개)

- **왼쪽 터미널**

```
ros2 run turtlesim turtlesim_node
```

- 정상 출력

```
[INFO] [1789440411.425973831] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1789440411.430387320] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], th
eta=[0.000000]
```

- **오른쪽 터미널**

```
ros2 run usv_basics turtle_pose_relay
```

- 정상 출력 — 두 줄이 나오고 조용해지면 정상

```
[INFO] [1789440416.820377552] [turtle_pose_relay]: /turtle1/pose -> /turtle1/pose2d (Pose2D), /turtle
1/vel (Twist)
[INFO] [1789440416.829528022] [turtle_pose_relay]: first relay: x=5.544 y=5.544 theta=0.000
```

줄	뜻
첫 줄	노드가 떴고 발행자·구독자를 만들었음
둘째 줄 `first relay`	**turtlesim 의 자세가 실제로 들어와 옮겨졌음.** 좌표가 turtlesim 시작 위치와 같음
둘째 줄이 안 나옴	turtlesim 이 안 떠 있음. 왼쪽 터미널 확인

5단계 — 확인 (세 번째 터미널)

```
ros2 topic list -t
```

- 정상 출력 — 아래 두 줄이 **새로 생겼음**

```
/turtle1/pose2d [geometry_msgs/msg/Pose2D]
/turtle1/vel [geometry_msgs/msg/Twist]
```

```
ros2 topic echo --once /turtle1/pose2d
```

- 정상 출력

```
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
---
```

```
ros2 topic hz /turtle1/pose2d
```

- 정상 출력 (기준 환경 실측)

```
average rate: 62.504
    min: 0.015s max: 0.017s std dev: 0.00057s window: 64
```

확인 항목	기대값	실측
새 토픽 형식	`geometry_msgs/msg/Pose2D`	같음
값	turtlesim 시작 자세 `(5.544, 5.544, 0)`	`x: 5.5444…` , `theta: 0.0`
주기	turtlesim 원본과 같음 (약 62.5 Hz)	`average rate: 62.504`

> ℹ `x: 5.544444561004639` **처럼 자릿수가 긴 이유**
>
> 원본 `float32` 의 `5.544445` 를 `float64` 에 옮기면 float32 가 표현하던 **이진 근삿값이 그대로 드러남.** 오차가 생긴 것이 아님

2-11. Simulink 로 turtlesim 을 목표 자세까지 보낸다

> ★ **이 절의 성격 — 미리 보기 실습**
>
> - Simulink 기초는 **4주차**에 배움. 이번 주차에는 모델을 **만들지 않고 실행해서 구조를 읽음**
> - 목표: §2-10 의 중계 노드가 **실제 제어 루프의 한 부분**으로 쓰이는 것을 눈으로 확인
> - MATLAB 이 아직 설치되지 않았다면 이 절은 4주차 이후 다시 와도 됨

준비물

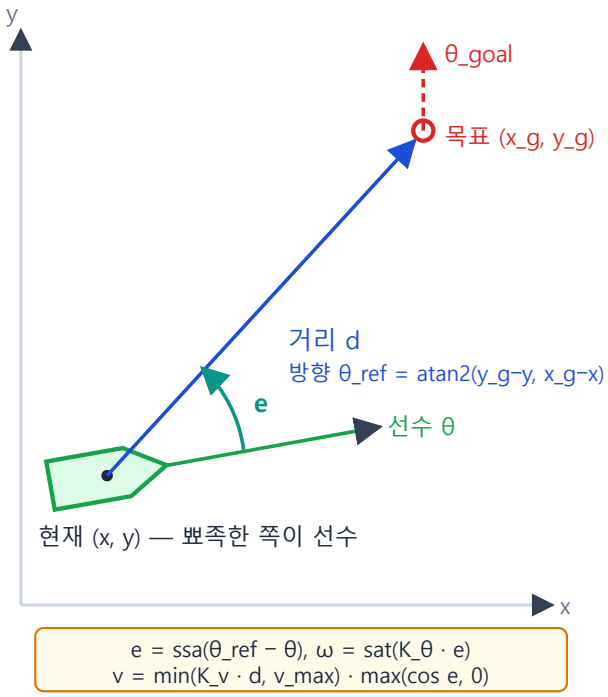
항목	내용
MATLAB	**R2024b** + Simulink + **ROS Toolbox** (학교 라이선스)
모델 폴더	강의자료 `10-주차별-강의자료/W02_simulink/` — 이 폴더를 통째로 복사해 씀
우분투 쪽	turtlesim + 중계 노드가 **떠 있어야** 함 (§2-10 4단계)
Domain ID	우분투의 `echo $ROS_DOMAIN_ID` 값 — `W02_setup.m` 과 Simulink ROS 네트워크 프로필에 같은 값을 넣음
네트워크	`.wslconfig` 에 `networkingMode=mirrored` → `wsl --shutdown` (WSL-VRX-환경구축 §8.2)

폴더에 들어 있는 것

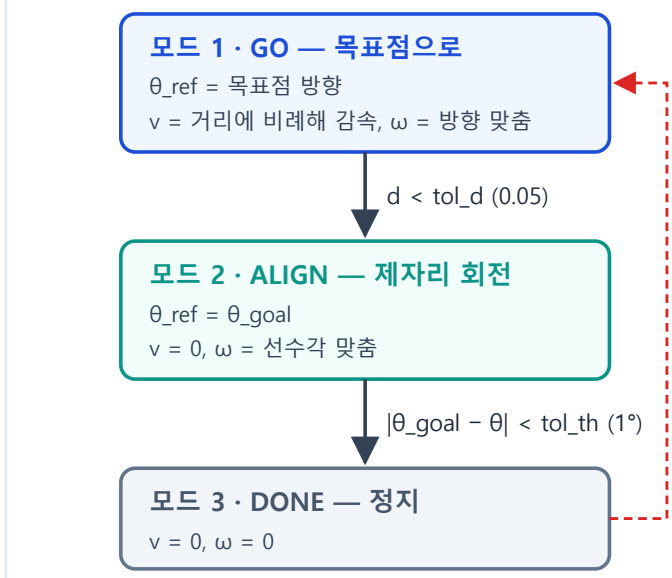
파일	하는 일	고칠 일
`W02_setup.m`	목표 자세·게인·Domain ID 설정. **모델보다 먼저** 실행	**이 파일만 고침**
`build_w02_models.m`	모델 3개를 코드로 생성. 모델이 망가지면 다시 실행	없음
`W02_1_pose_sub.slx`	1단계 — `/turtle1/pose2d` 를 받아 숫자로 보여줌	없음
`W02_2_goto_offline.slx`	2단계 — **turtlesim 없이** 같은 운동식으로 미리 돌려봄	없음
`W02_3_goto_turtlesim.slx`	3단계 — 실제 turtlesim 을 목표 자세로 보냄	없음
`W02_plot.m` · `W02_animate.m` · `W02_hull.m`	결과 그림 · 실시간 그림 · 선체 모양	없음
`ros2/turtle_pose_relay.py`	§2-10 의 중계 노드와 같은 파일 (저장소 없이 쓸 때)	없음

제어 원리 — 한 번에 하나씩 맞춘다

(a) 제어기가 계산하는 세 가지 값



(b) 한 번에 하나씩 — 모드 3개



목표점을 바꿔 d > 3·tol_d 가 되면 모드 1 로 되돌아감 (빨간 점선)
모드는 Unit Delay 로 한 스텝 늦춰 되먹임 — 대수 루프 방지

그림의 위치	무엇인가
(a) 초록 선체 · 초록 화살표	현재 위치와 선수 방향 θ. **뾰족한 쪽이 선수**
(a) 파란 화살표	목표점까지의 거리 d 와 방향 θ_{ref} (모델의 태그 이름은 `psi_ref`)
(a) 청록 호 e	선수가 목표 방향에서 벗어난 각도 — 이것을 0 으로 만듦
(b) 모드 1 → 2 → 3	먼저 **위치**, 다음에 **선수각**, 끝나면 정지
(b) 빨간 점선	목표를 바꾸면 처음(모드 1)부터 다시

- 목표 방향과 선수각 오차

$$\theta_{\text{ref}} = \operatorname{atan2}(y_{\text{goal}} - y,\ x_{\text{goal}} - x), \qquad e = \operatorname{ssa}(\theta_{\text{ref}} - \theta)$$

- 제어 법칙 — 선수각은 P 제어, 속도는 거리에 비례

$$\omega = \mathrm{sat}_{\omega_{\max}}(K_\theta\, e), \qquad v = \min(K_v\, d,\ v_{\max}) \cdot \max(\cos e,\ 0)$$

기호	뜻	값 (`W02_setup.m`)
$(x_{\text{goal}}, y_{\text{goal}}, \theta_{\text{goal}})$	목표 자세. 3주차 무게중심 x_g 와 구분하려고 `goal` 을 붙임	`(9.0, 2.0, 90°)`
d	목표점까지 거리	계산값
ssa	가장 짧은 쪽 각도 차이. $-\pi \sim \pi$ 로 접음	—
K_θ	선수각 게인	`Kpsi = 4.0`
K_v	속도 게인	`Kv = 1.0`
$v_{\max}$, $\omega_{\max}$	속도 · 회전 한계	`2.0` , `2.0 rad/s`
`tol_d` , `tol_th`	도착 판정	`0.05` , `1°`

- $\max(\cos e, 0)$ 의 역할 — 목표를 **등지고 있으면 전진하지 않고** 먼저 돎
- turtlesim 은 `cmd_vel` 의 속도를 **그대로** 따르는 운동학 모델임. 그래서 추력 모델이나 D 항 없이 P 제어로 충분함

> **⚠ 이 식은 turtlesim 좌표계의 식임 — 3주차부터는 규약이 바뀜**
>
> turtlesim 은 수학 교과서와 같은 평면을 씀. x 오른쪽, y 위쪽, 각 θ 는 x 축에서 **반시계**가 +
> 3주차부터 쓰는 NED 는 x 가 **북**(위쪽), y 가 **동**(오른쪽), 각 ψ 는 북에서 **시계**가 +
> 3주차부터 θ 는 **피치각**이고 선수각은 ψ 임
>
> | | 2주차 turtlesim | 3주차 이후 NED |
> |---|---|---|
> | 목표 방향 | $\mathrm{atan2}(y_{\text{goal}} - y,\ x_{\text{goal}} - x)$ | $\mathrm{atan2}(y_{\text{goal}} - y,\ x_{\text{goal}} - x)$ (x 북, y 동) |
> | 각의 기준 · 방향 | x 축(오른쪽)에서 반시계 | 북(위쪽)에서 시계 |
> | v | 거북이의 **전진** 속도 | 배의 **좌우**(sway) 속도 |
> | 회전 속도 | ω | r (요각속도) |
>
> 두 식은 **모양이 같음** — 둘 다 "두 번째 축의 차이, 첫 번째 축의 차이" 순서. NED 에서는 첫 번째 축이 북이기 때문에 같은 모양의 식이 시계방향 각을 줌. 식을 외우지 말고 **어느 축이 첫 번째인지**를 볼 것
> 그리고 v 는 3주차부터 **옆으로 미끄러지는 속도**를 뜻함. 앞으로 가는 속도는 u

1단계 — MATLAB 준비

1. MATLAB 을 엶
2. 위쪽 **현재 폴더** 주소줄에서 `W02_simulink` 폴더로 이동
3. `W02_setup.m` 을 열어 **Domain ID 를 우분투와 같게** 고침 (21행)

```
ros_domain_id = '7';        % 우분투의 echo $ROS_DOMAIN_ID 값
```

- 파일의 `'7'` 은 §1-5 · §2-6 예시와 같은 값. 자기 팀 번호로 바꿈. 아래 정상 출력의 `7` 도 같은 이유

4. 명령 창에서 실행

```
W02_setup
```

- 정상 출력

```
W02_setup 완료 — 목표 (9.00, 2.00, 90.0 deg), ROS_DOMAIN_ID=7
```

5. Simulink 쪽 도메인 확인 — 툴스트립 **시뮬레이션 → ROS 네트워크** 의 Domain ID 를 우분투 값과 같게 함 (4주차 2-14-5 와 같은 절차)
   - Simulink ROS 2 블록은 `ROS_DOMAIN_ID` 환경변수가 아니라 **이 프로필 값**을 읽음. `W02_setup` 의 `setenv` 만으로는 모델이 값을 받지 못함
6. 우분투의 토픽이 MATLAB 에서 보이는지 확인

```
ros2("topic","list")
```

- 정상 출력 — `/turtle1/pose2d` 가 있어야 함 (중계 노드가 떠 있을 때)

```
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/turtle1/pose2d
/turtle1/rotate_absolute/_action/feedback
/turtle1/rotate_absolute/_action/status
/turtle1/vel
```

> ⚠ **목록은 보이는데 다음 단계에서 값이 0 으로만 나오는 경우**
> 우분투 쪽 노드가 **멈춘 채 목록에만 남아 있는** 상태. 목록은 발견 정보라 한동안 지워지지 않음
> 우분투 터미널에서 `ros2 topic hz /turtle1/pose2d` 로 실제 주기가 나오는지 먼저 봄
> 안 나오면 두 노드를 `Ctrl + C` 로 끄고 다시 띄움

2단계 — 모델 생성

```
build_w02_models
```

- 정상 출력 (마지막 부분)

```
  W02_1_pose_sub 생성
  W02_2_goto_offline 생성
  W02_3_goto_turtlesim 생성

완료. 생성된 모델:
  W02_1_pose_sub.slx
  W02_2_goto_offline.slx
  W02_3_goto_turtlesim.slx
```

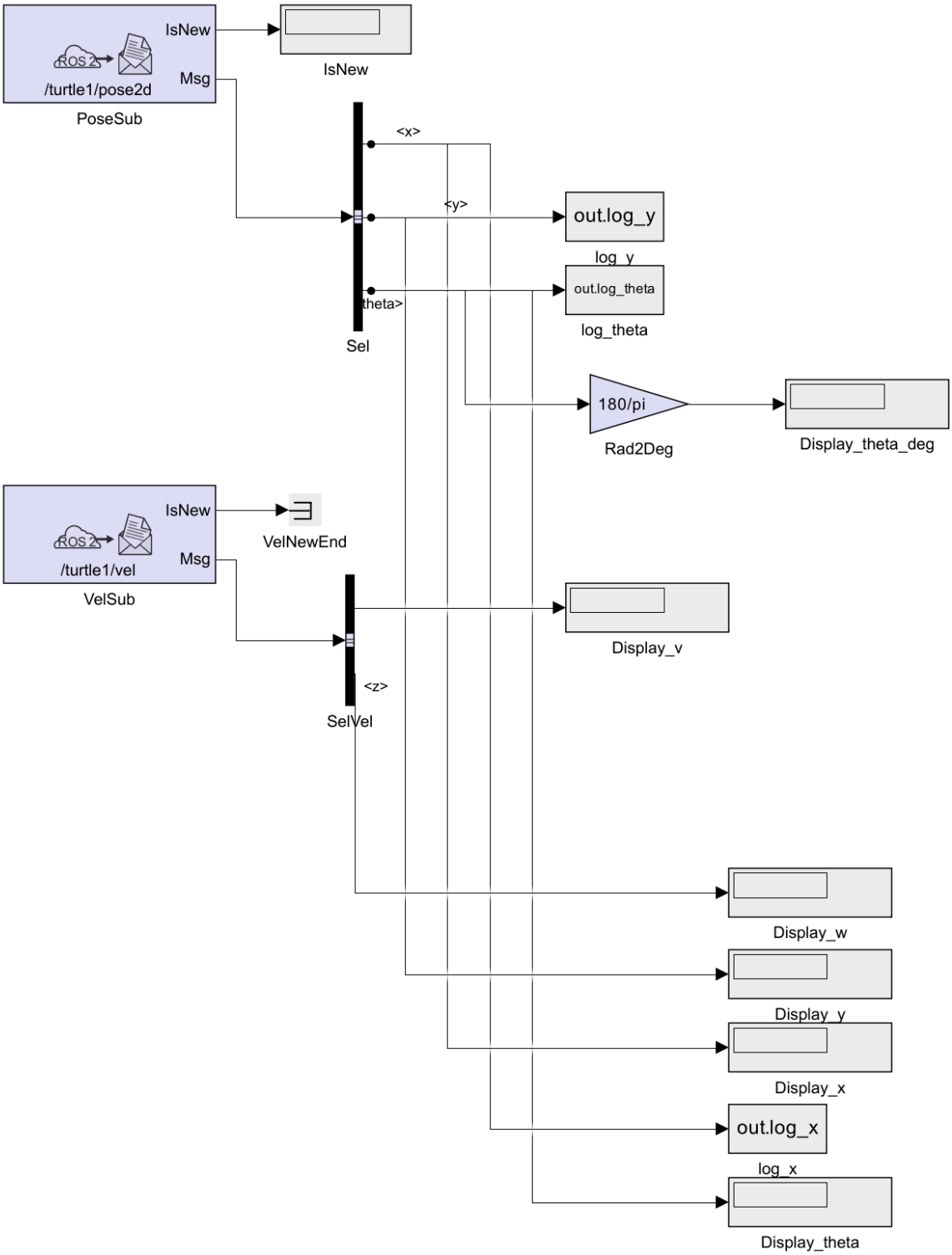
- `Automated layout might not improve upon original layout` 경고가 몇 줄 섞여 나옴. **무시해도 됨**
- 폴더를 볼트 밖으로 복사해 쓰면 `tidy_model` 등 배치 정리 함수가 없다는 경고가 모델마다 나옴. 모델 생성과는 무관하므로 무시해도 됨

> ✎ **모델을 만지다 망가뜨렸을 때**
> `W02_setup` → `build_w02_models` 두 줄이면 처음 상태로 돌아옴. 마음껏 고쳐 볼 것

3단계 — 자세만 받아 본다 (`W02_1_pose_sub`)

```
open_system('W02_1_pose_sub')
```

W02 1단계 — turtlesim 자세 구독
/turtle1/pose (turtlesim/Pose) 는 MATLAB 에 없는 형식이다.
중계 노드가 /turtle1/pose2d (geometry_msgs/Pose2D) 로 다시 발행한 것을 받는다.
실행 전: (1) turtlesim_node (2) ros2 run usv_basics turtle_pose_relay (3) W02_setup



블록	하는 일
`PoseSub` (ROS 2 Subscribe)	토픽 `/turtle1/pose2d`, 형식 `geometry_msgs/Pose2D` 를 받음
`Sel` (Bus Selector)	메시지에서 `x` · `y` · `theta` 세 칸을 꺼냄
`IsNew`	이번 스텝에 새 메시지가 왔으면 1
`Display_x/y/theta` · `Display_theta_deg`	숫자 표시. 마지막은 도 단위
`VelSub` → `SelVel` → `Display_v/w`	`/turtle1/vel` 의 속도

1. Simulink 창 위쪽 **실행**(초록 삼각형) 클릭 — 정지 시간이 `inf` 라 계속 돎
2. 우분투의 네 번째 터미널에서 거북이를 움직여 봄

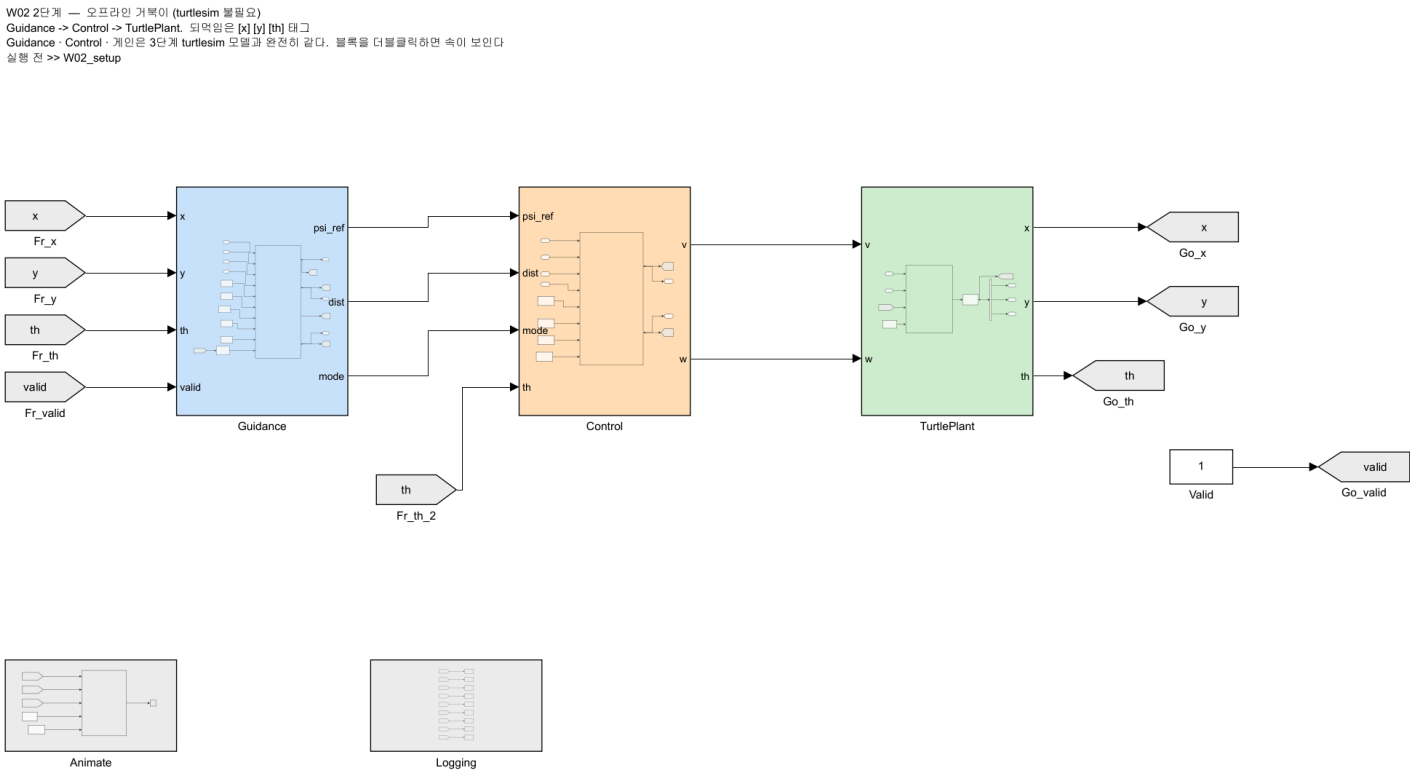
```
ros2 run turtlesim turtle_teleop_key
```

3. 방향키를 누를 때마다 `Display_x` · `Display_theta_deg` 의 숫자가 바뀌면 성공
4. Simulink 의 **정지**(빨간 네모) 클릭

4단계 — 오프라인으로 먼저 돌린다 (`W02_2_goto_offline`)

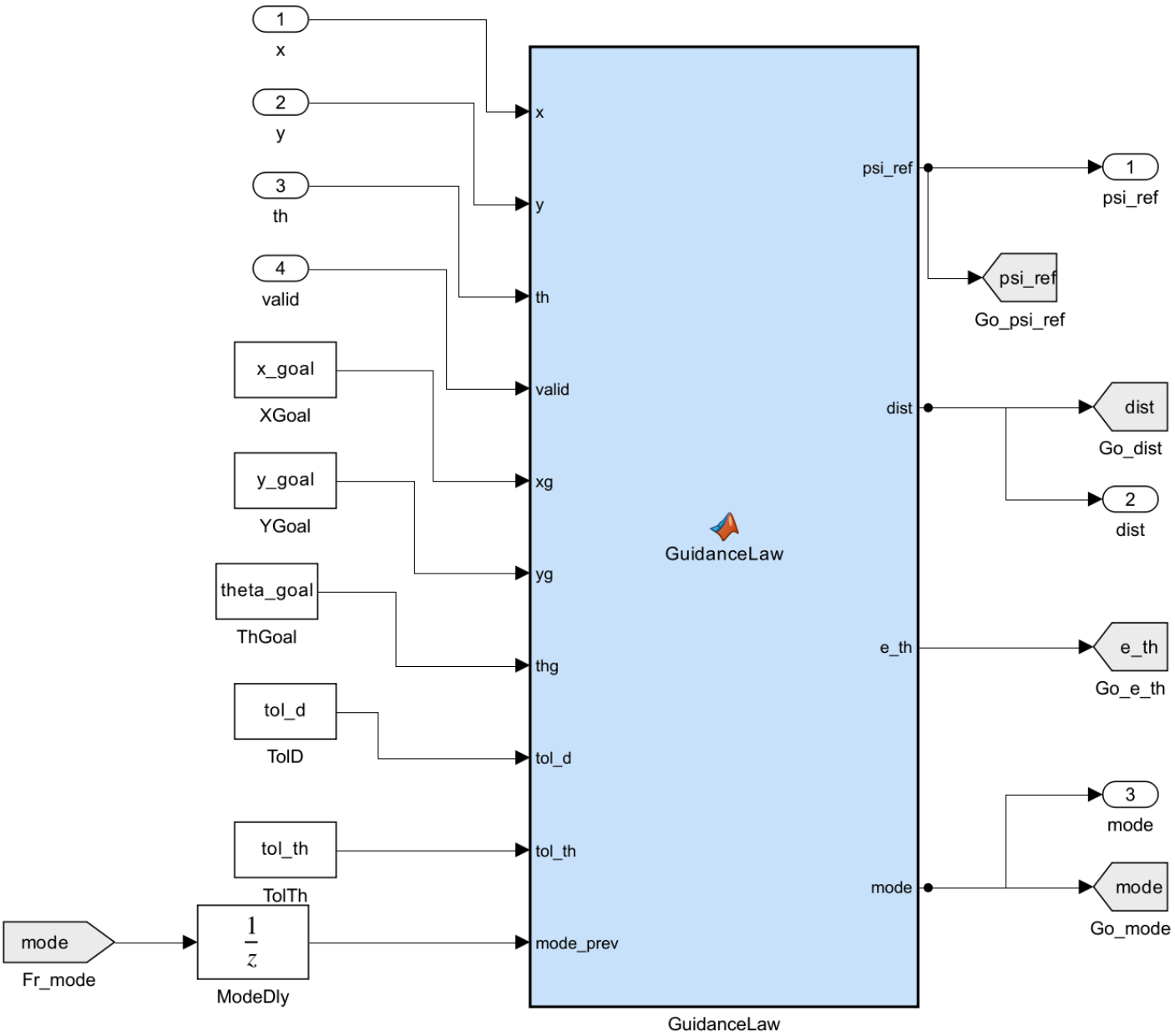
> **★ 실물 전에 오프라인 — 이 과목 내내 지키는 순서**
>
> 오프라인 모델은 turtlesim 과 **같은 운동식**(`turtle.cpp`)으로 만들었음
> 여기서 목표에 도착하지 못하면 실물에서도 못 함. 반대로 여기서 맞춘 게인은 실물에 그대로 통함

```
open_system('W02_2_goto_offline')
```



색	서브시스템	입력 → 출력	속을 보려면
파랑	`Guidance` (유도)	`x, y, th, valid` → `psi_ref, dist, mode`	더블클릭
주황	`Control` (제어)	`psi_ref, dist, mode, th` → `v, w`	더블클릭
초록	`TurtlePlant` (운동모델)	`v, w` → `x, y, th`	더블클릭
회색	`Animate` · `Logging`	선 없음. 태그 `[x]` `[y]` … 로 받음	더블클릭
흰 오각형	`[x]` `[y]` `[th]` `[valid]`	되먹임 태그. 오른쪽에서 보낸 값을 왼쪽에서 받음	—

- 신호는 **왼쪽 → 오른쪽** 한 방향으로만 흐름. 되돌아오는 선은 태그로 대신했음
- `Guidance` 를 더블클릭한 화면

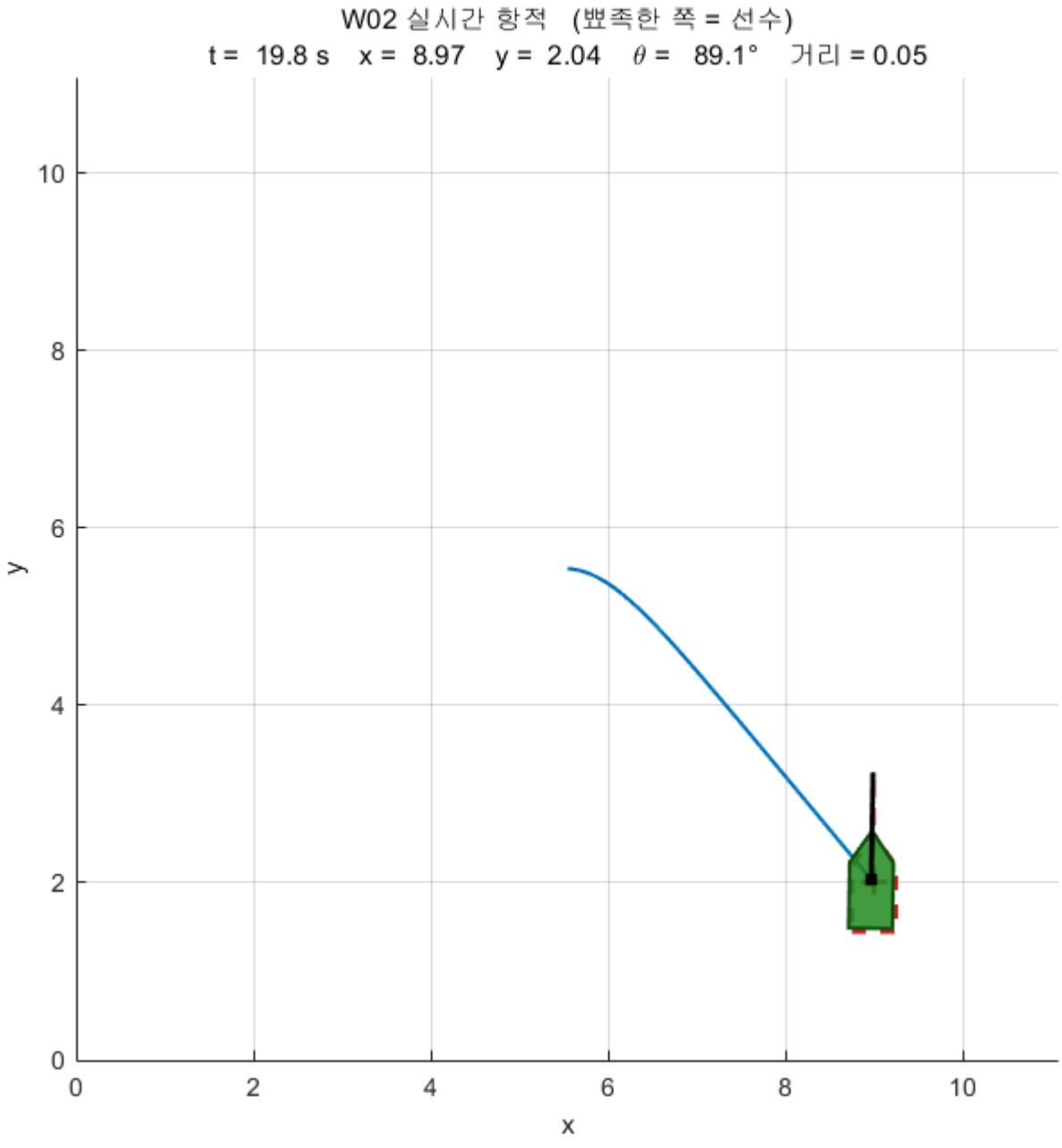


블록	뜻
가운데 `GuidanceLaw` (MATLAB Function)	위 수식과 모드 전환. 더블클릭하면 코드가 보임
왼쪽 흰 상자 5개	목표 `x_goal` `y_goal` `theta_goal`, 판정 `tol_d` `tol_th` — `W02_setup.m` 의 변수
왼쪽 아래 보라 `1/z` (Unit Delay)	한 스텝 전 모드. 자기 출력을 바로 입력으로 쓰면 **대수 루프** 오류가 남
오른쪽 오각형 `psi_ref` `dist` `e_th` `mode`	로깅용 태그

- 실행 — 명령 창에서

```
W02_setup
out = sim('W02_2_goto_offline');
S = W02_plot(out, '오프라인')
```

- 실행 중 뜨는 실시간 그림 (끝난 순간)



그림에서	뜻
초록 선체의 **뾰족한 쪽**	현재 선수 방향
빨간 점선 선체	목표 자세. 초록 선체가 **그 안에 들어가 같은 방향**을 보면 성공
제목 줄	시각 · 위치 · 선수각 · 목표까지 거리

- 정상 출력

```
[오프라인] 위치오차 0.0481 | 선수각오차 0.903 deg | 위치도착 5.20 s | 완료 6.95 s
```

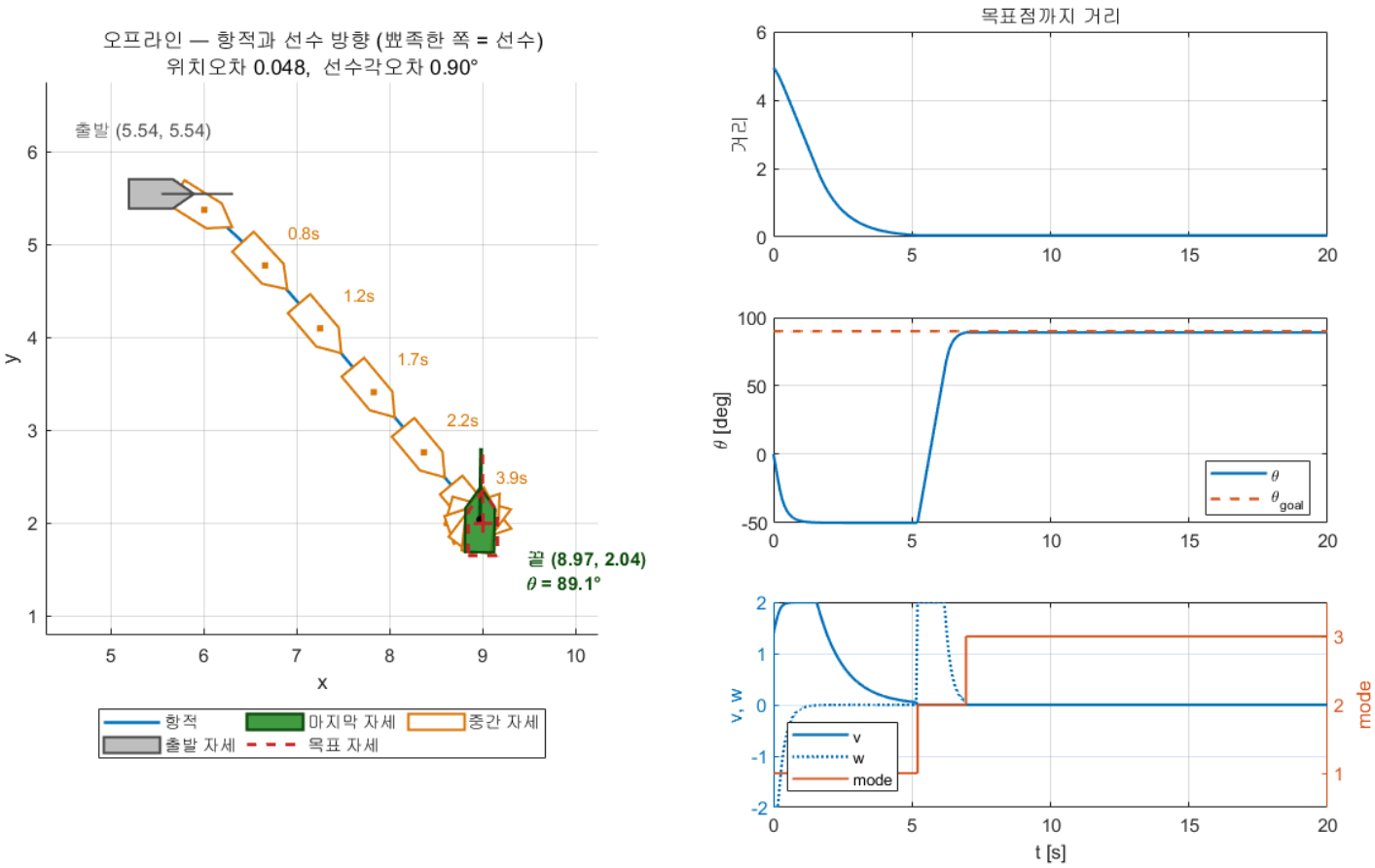
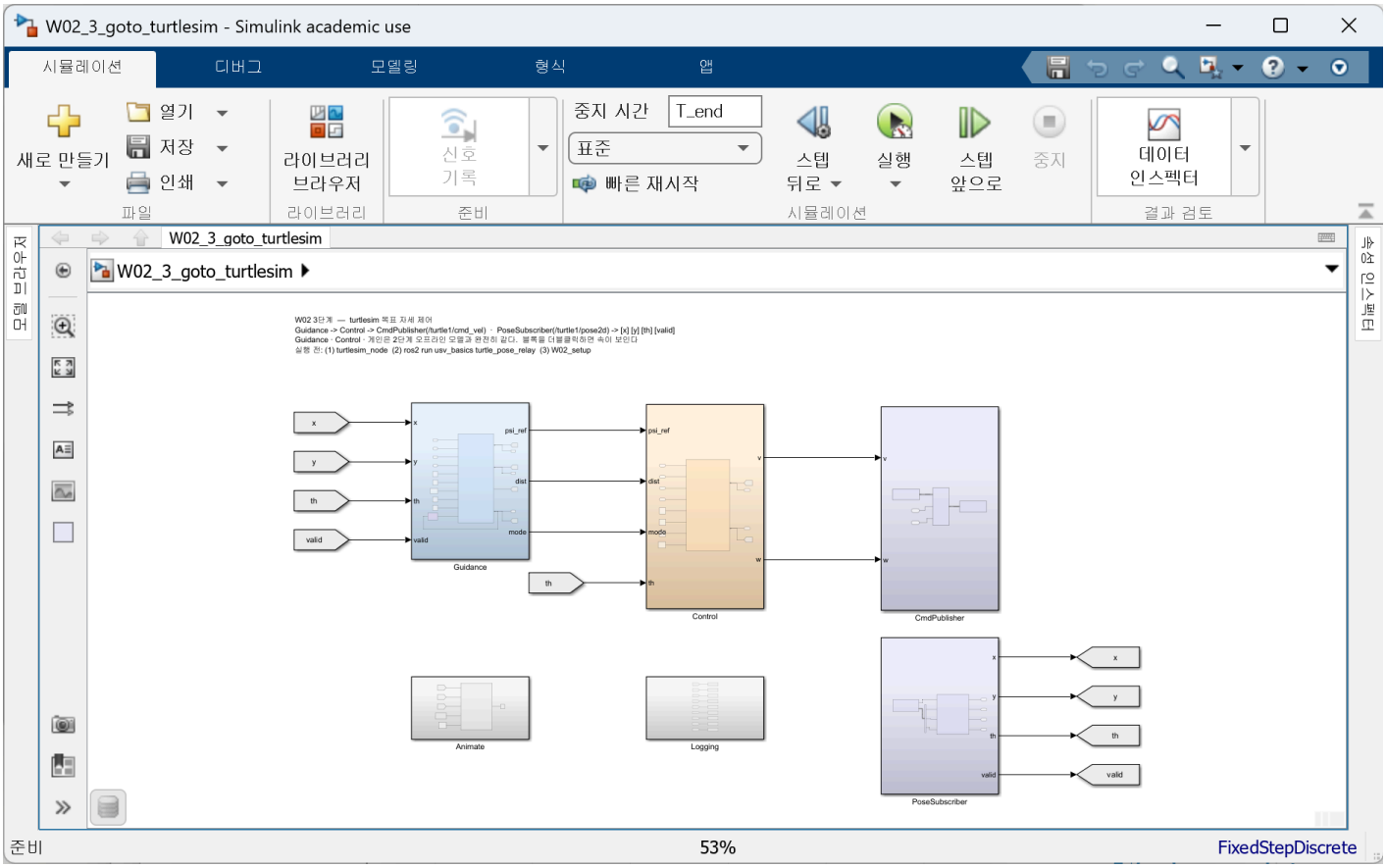


그림	읽는 법
왼쪽 회색 선체	출발 자세 `(5.54, 5.54)`, 선수 오른쪽(0°)
왼쪽 주황 윤곽 선체 + 시각	이동 0.8 마다 한 척. **간격이 점점 좁아짐** = 목표에 가까울수록 감속
왼쪽 초록 선체 + 좌표	마지막 자세. 빨간 점선(목표)과 겹침
오른쪽 위 거리	약 5 초에 0 근처
오른쪽 가운데 θ	먼저 약 −46° 로 돌아 목표를 향하고(모드 1), 도착 뒤 90° 로 돎(모드 2). $\operatorname{atan2}(2-5.544,\ 9-5.544) = -45.7°$
오른쪽 아래 mode	1 → 2 (5.20 s) → 3 (6.95 s)

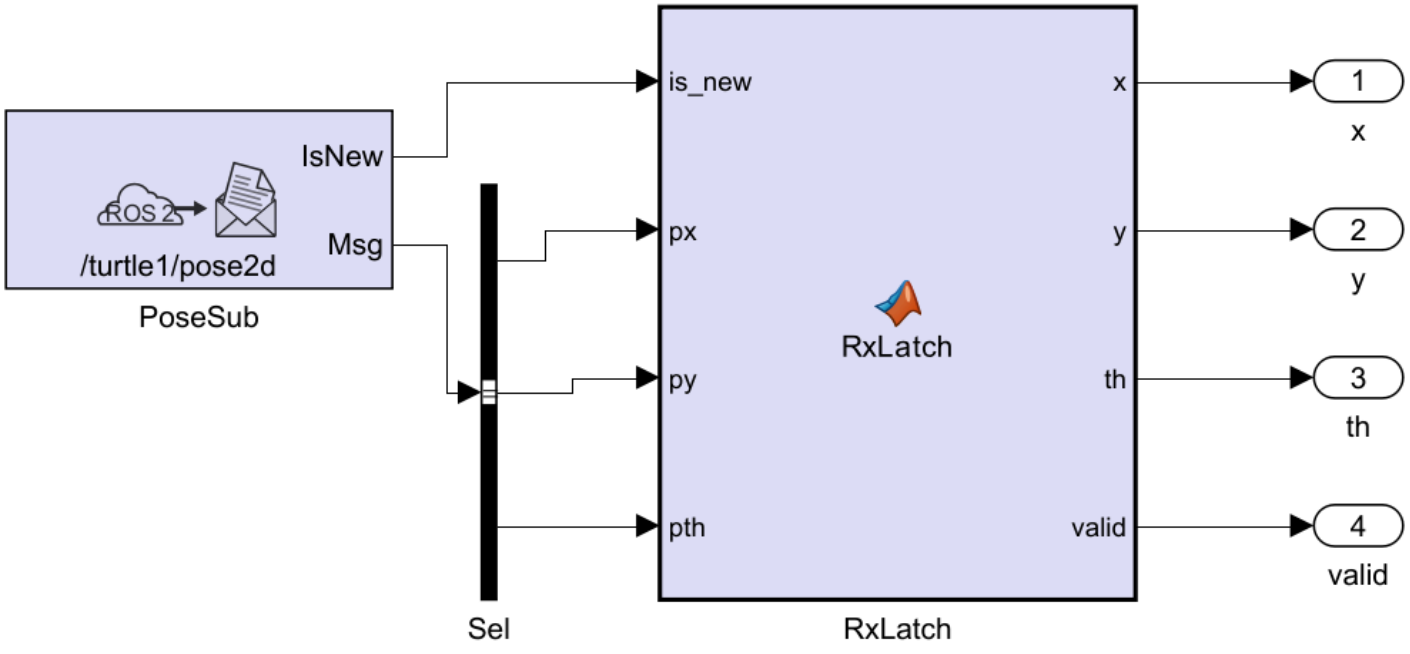
5단계 — 실제 turtlesim 을 움직인다 (`W02_3_goto_turtlesim`)

```
open_system('W02_3_goto_turtlesim')
```



화면의 위치	무엇인가
위쪽 리본 **실행**(초록 삼각형) · **정지 시간** `T_end`	누르면 20 초 동안 돎
파랑 `Guidance` · 주황 `Control`	**2단계와 똑같은 서브시스템**
연보라 `CmdPublisher`	`v, w` 를 `Twist` 에 담아 `/turtle1/cmd_vel` 로 보냄 — 2단계의 `TurtlePlant` 자리
연보라 `PoseSubscriber`	`/turtle1/pose2d` 를 받아 `[x] [y] [th] [valid]` 태그로 내보냄

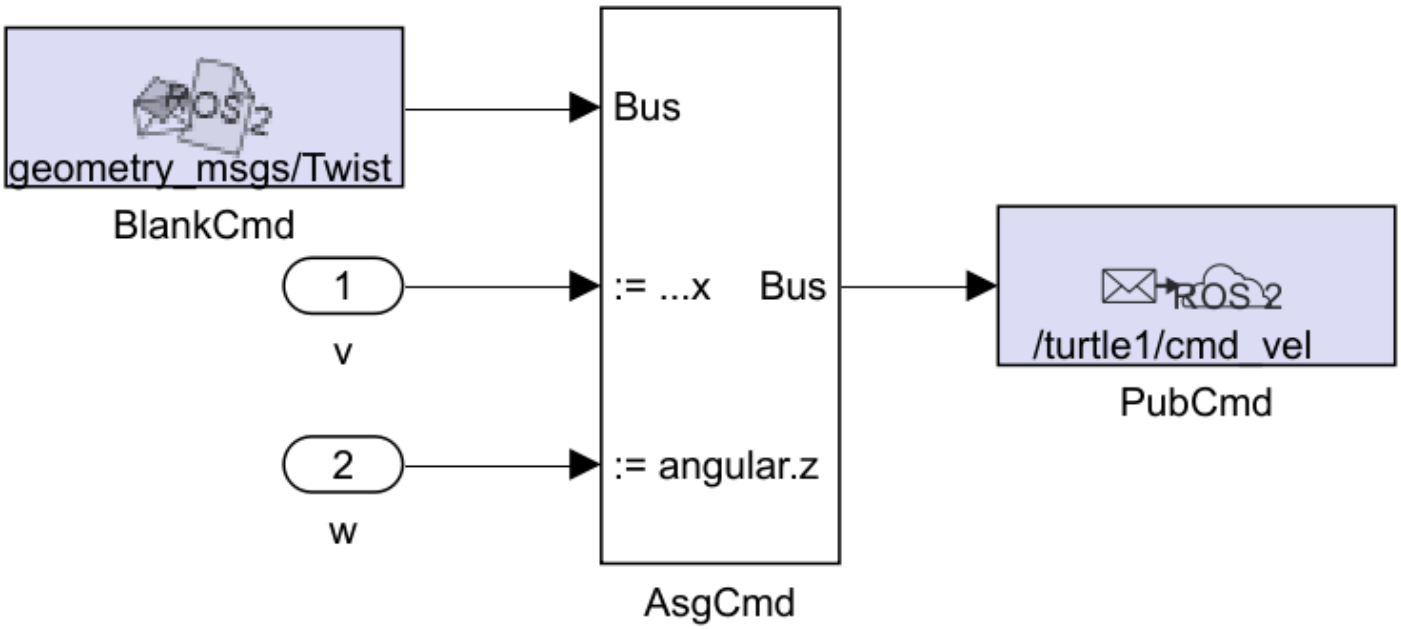
- `PoseSubscriber` 를 더블클릭한 화면



블록	뜻
`PoseSub`	§2-10 중계 노드가 내보낸 `/turtle1/pose2d` 를 받음
`Sel`	`x` · `y` · `theta` 를 꺼냄
`RxLatch`	**첫 메시지가 오기 전에는 `valid = 0`** — 그동안 거북이를 움직이지 않음

> **⚠ 첫 메시지 전에는 좌표가 (0, 0) 으로 들어옴**
>
> Subscribe 블록은 메시지가 오기 전에 **모든 칸을 0 으로 채운 값**을 냄
> turtlesim 에서 (0, 0) 은 실제로 갈 수 있는 **왼쪽 아래 모서리**라 값만 보고는 거를 수 없음
> 그래서 `IsNew` 가 한 번이라도 1 이 됐는지를 기억해(`RxLatch`) 판단함

- `CmdPublisher` 를 더블클릭한 화면



블록	뜻
`BlankCmd` (Blank Message)	빈 `geometry_msgs/Twist` 양식
`AsgCmd` (Bus Assignment)	`linear.x ← v` , `angular.z ← w` 두 칸만 채움
`PubCmd` (Publish)	`/turtle1/cmd_vel` 로 보냄

1. 우분투에서 turtlesim 과 중계 노드가 떠 있는지 다시 확인

```
ros2 topic hz /turtle1/pose2d
```

2. 거북이를 시작 위치로 되돌림 (이전 실습으로 옮겨졌다면)

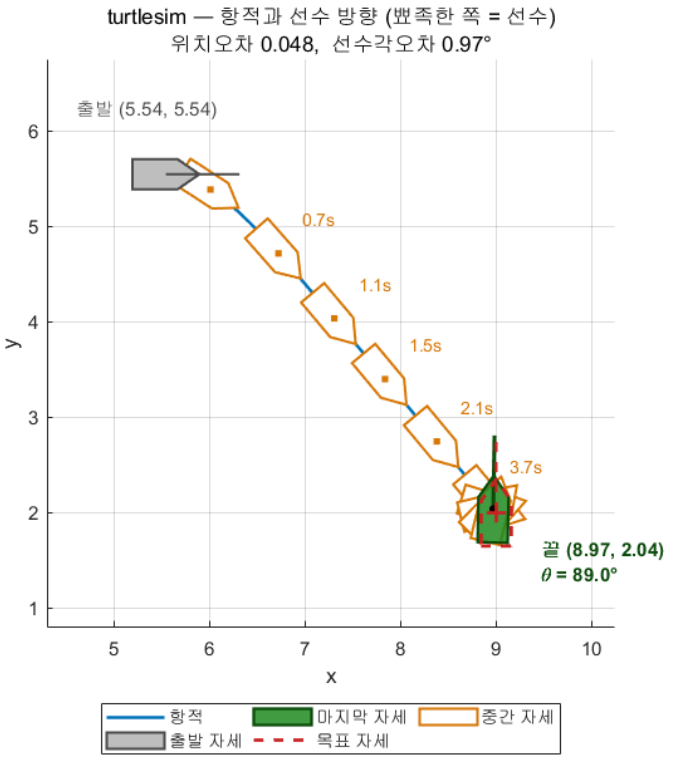
```
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5.544445, y: 5.54444
5, theta: 0.0}"
```

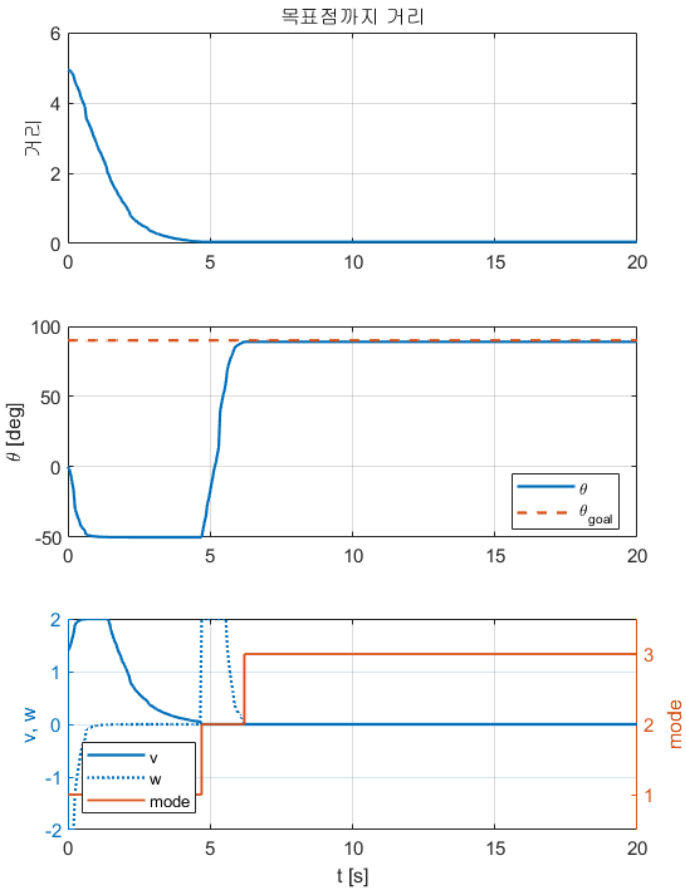
3. MATLAB 명령 창에서 실행

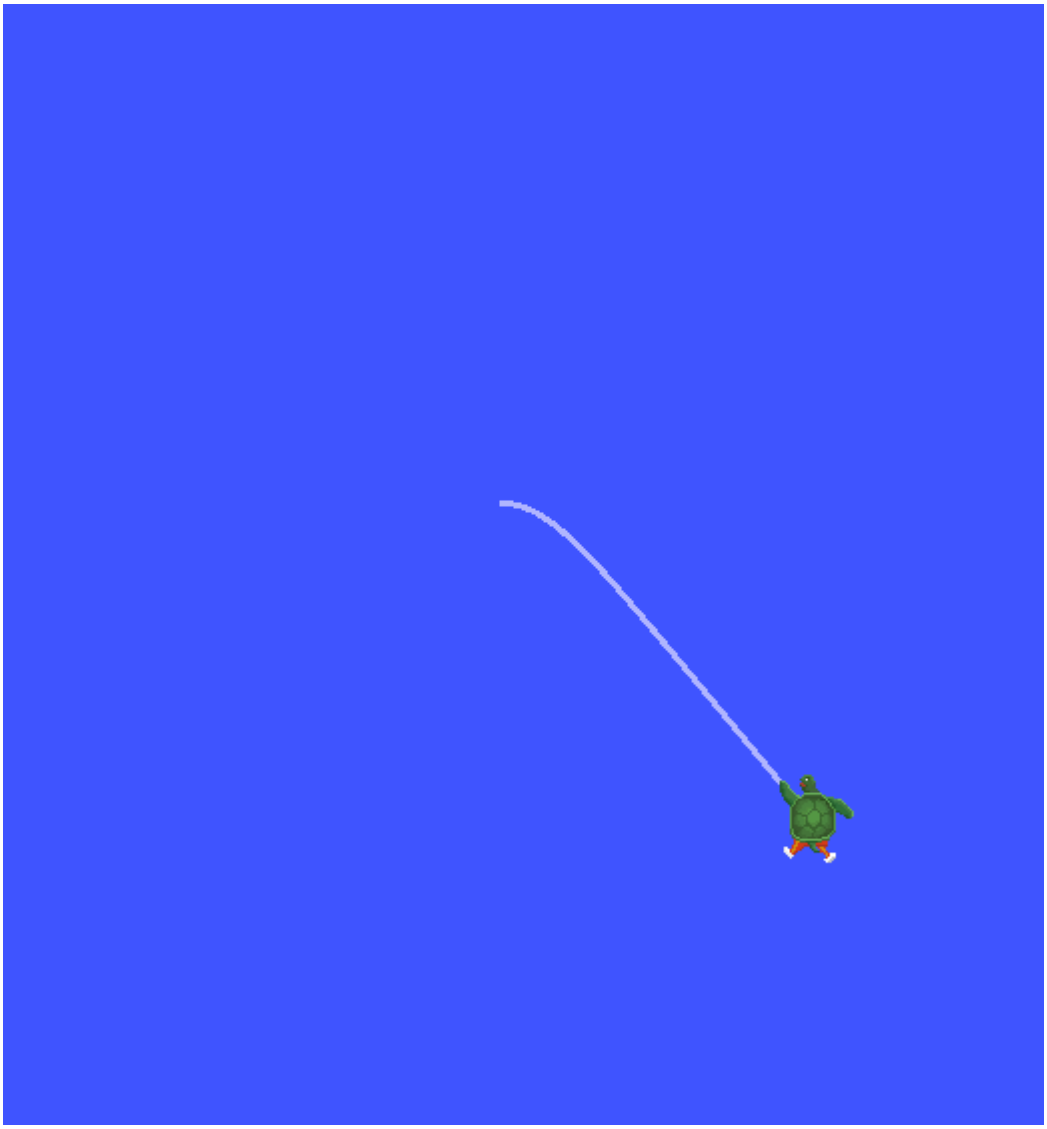
```
W02_setup
out = sim('W02_3_goto_turtlesim');
S = W02_plot(out, 'turtlesim')
```

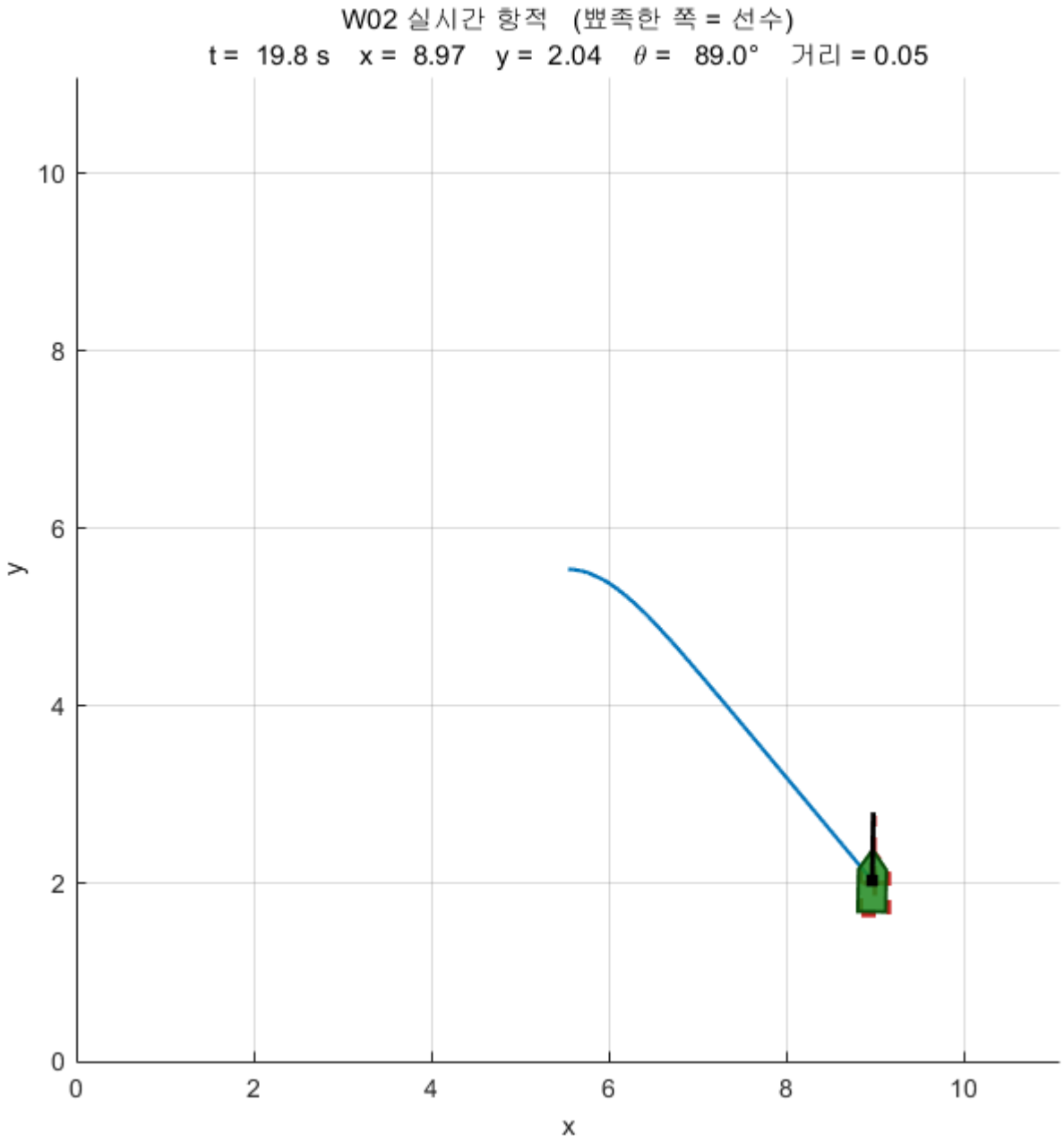
- 실행하는 20 초 동안 **turtlesim 창의 거북이가 오른쪽 아래로 가서 위를 보고 멈춤**
- 정상 출력 (기준 환경 실측)

```
[turtlesim] 위치오차 0.0478 | 선수각오차 0.973 deg | 위치도착 4.70 s | 완료 6.20 s
```









- MATLAB 쪽 실시간 그림은 **turtlesim 이 실제로 보낸 자세**를 그린 것. turtlesim 창의 흰 선과 같은 궤적이 나와야 함

화면에서 확인할 것	무엇
흰 선이 가운데에서 오른쪽 아래로	거북이가 지나간 길. 결과 그림 왼쪽의 파란 항적과 같은 모양
거북이 머리가 **화면 위쪽**	목표 선수각 90°
거북이 위치 `(9, 2)` 부근	목표점

오프라인과 실물 대조

항목	오프라인 (`W02_2`)	turtlesim (`W02_3`)	차이
최종 위치 오차	0.048	0.048	0.000
최종 선수각 오차	0.90°	0.97°	+0.07°
위치 도착 (모드 1 → 2)	5.20 s	4.70 s	−0.50 s
완료 (모드 3)	6.95 s	6.20 s	−0.75 s

- 측정 조건 — 목표 `(9.0, 2.0, 90°)`, 출발 `(5.544, 5.544, 0°)`, 제어 주기 0.05 s, 20 초 실행의 **마지막 샘플**
- **두 곳 모두 판정 기준**(`tol_d = 0.05`, `tol_th = 1°`) **안에서 멈췄음** — 선수각 오차 차이 0.07° 는 기준 1° 보다 작음
- **시각은 실행할 때마다 흔들림.** 같은 모델을 한 번 더 돌린 기록

turtlesim 실행	위치 오차	선수각 오차	위치 도착	완료
1회차	0.047	0.88°	5.30 s	7.05 s
2회차 (위 표)	0.048	0.97°	4.70 s	6.20 s

> ℹ **실물 대조는 초 단위 시각보다 최종 오차를 먼저 봄**
>
> turtlesim 은 **벽시계**로 움직이고, Simulink 의 기록 시각은 **시뮬레이션 시계**임. 서로 다른 두 시계 페이싱(`PacingRate = 1`)으로 맞추지만 회차 간 차이가 실측으로 위치 도착 0.60 s, 완료 0.85 s 났음 시뮬레이터와 Simulink 의 시계를 맞추는 페이싱은 4주차 VRX 연동에서 자세히 다룸

해 볼 것 — 목표를 바꾼다

1. `W02_setup.m` 의 목표를 고침

```
x_goal     = 2.0;
y_goal     = 9.0;
theta_goal = deg2rad(180);   % 화면 왼쪽
```

2. `W02_setup` → 오프라인으로 먼저 확인 → turtlesim 에서 실행
3. 오프라인과 turtlesim 의 도착 시각이 비슷한지 표로 비교

> ✕ **목표를 벽(0 또는 11.09) 가까이 두지 않음**
>
> turtlesim 은 벽에 닿으면 멈추고 경고를 냄. 목표는 `1 ~ 10` 사이에 둠

마무리

이번 주차 요약

순서	한 일	확인 방법
1	ROS 2 Humble 설치	`ros2 doctor` → `All 5 checks passed`
2	Domain ID 설정	`echo $ROS_DOMAIN_ID` → 팀 번호
3	**VS Code 설치와 WSL 연결**	왼쪽 아래 `WSL: Ubuntu-22.04`
4	talker / listener 통신	`I heard: [Hello World: N]`
5	**turtlesim — 토픽·서비스·액션·파라미터**	거북이 2마리 · 초록 배경 · `SUCCEEDED`
6	조사 명령어 사용	`ros2 topic list` / `hz` / `info --verbose`
7	워크스페이스와 패키지 생성	`colcon build` → `1 package finished`
8	노드 직접 작성 및 통신	`received: USV alive: 0`
9	**rqt_graph · Topic Monitor · rqt_console · RViz2**	창 4개가 뜸
10	QoS 불일치 재현 · 진단 · 해결	수신 0건 → 16건 (8 s)
11	**중계 노드** `turtle_pose_relay`	`/turtle1/pose2d [geometry_msgs/msg/Pose2D]` · 62.5 Hz
12	**Simulink 로 turtlesim 목표 자세 제어**	오프라인 위치오차 0.048 · turtlesim 0.048

수업 진도 체크

> ★ 다음 주 수업을 따라가기 위한 최소 조건

이론 이해

- [ ] ROS 2가 무엇을 해결하는지 설명할 수 있음
- [ ] `roscore` · `rospy` · `catkin_make` 가 나오면 **ROS 1 예제**임을 앎
- [ ] 노드 · 토픽 · 발행 · 구독 · 메시지를 각각 설명할 수 있음
- [ ] **토픽 · 서비스 · 액션 · 파라미터**를 언제 쓰는지 하나씩 예를 들 수 있음
- [ ] 서비스는 **답이 올 때까지 기다린다**는 것을 앎
- [ ] **워크스페이스 · 패키지 · 노드**의 차이를 설명할 수 있음
- [ ] `setup.py` 의 `console_scripts` 한 줄이 무엇을 정하는지 앎
- [ ] `header.stamp` 와 `frame_id` 가 왜 필요한지 앎
- [ ] `ROS_DOMAIN_ID` 를 왜 팀별로 나누는지 앎
- [ ] QoS 불일치가 **프로그램을 죽이지 않고** 실패한다는 것을 앎
- [ ] 에일리어싱이 무엇인지 그림으로 설명할 수 있음
- [ ] MATLAB 이 `/turtle1/pose` 를 **왜 못 받는지**, 중계 노드가 **무엇을 바꾸고 무엇을 안 바꾸는지** 설명할 수 있음
- [ ] 목표 자세 제어의 모드 1 · 2 · 3 이 각각 무엇을 맞추는지 설명할 수 있음

환경 구축

- [ ] `ros2 doctor` 가 `All 5 checks passed` 로 끝남
- [ ] PowerShell 에서 `code --version` 이 버전을 출력함
- [ ] `code --list-extensions` 에 `ms-vscode-remote.remote-wsl` 이 있음
- [ ] VS Code 왼쪽 아래에 **`WSL: Ubuntu-22.04`** 가 보임
- [ ] `code --remote wsl+Ubuntu-22.04 --list-extensions` 에 `ms-python.python` 이 있음
- [ ] VS Code 통합 터미널의 프롬프트가 `사용자명@컴퓨터:~/capstone_ws$` 형태임
- [ ] 통합 터미널을 **좌우로 나눠** 두 노드를 동시에 실행해 봤음
- [ ] `tail -3 ~/.bashrc` 에 `source /opt/ros/humble/setup.bash` 와 `ROS_DOMAIN_ID` 가 있음
- [ ] **새 터미널을 열자마자** `ros2 topic list` 가 바로 동작함
- [ ] Windows 탐색기에서 `\\wsl.localhost\Ubuntu-22.04\home\사용자명` 이 열림

실습 완료

- [ ] `ros2 run demo_nodes_cpp talker` / `demo_nodes_py listener` 통신 성공
- [ ] **turtlesim** 을 띄우고 `cmd_vel` 토픽으로 움직였음
- [ ] **서비스** `/spawn` 으로 거북이를 하나 더 만들었음
- [ ] **파라미터**로 배경색을 바꾸고 `/clear` 로 반영시켰음
- [ ] **액션** `rotate_absolute` 를 보내 `SUCCEEDED` 를 확인했음
- [ ] `ros2 topic list` / `echo` / `hz` / `info --verbose` 를 모두 써 봤음
- [ ] `~/capstone_ws` 워크스페이스 생성 및 빌드 성공
- [ ] 직접 만든 `simple_talker` ↔ `simple_listener` 통신 성공
- [ ] `rqt_graph` 로 노드 그래프를 확인했음
- [ ] `rqt_topic` 에서 `/usv_chatter` 의 `Hz` 가 **2.00** 인 것을 확인했음
- [ ] `rqt_console` 에서 로그를 확인했음
- [ ] `rviz2` 를 띄워 `RViz is ready.` 를 확인했음
- [ ] **QoS 불일치를 재현하고 진단한 뒤 해결했음** (수신 0건 → 정상 수신)
- [ ] `~/.bashrc` 에 `ROS_DOMAIN_ID` 를 팀 번호로 설정했음
- [ ] `ros2 run usv_basics turtle_pose_relay` 실행 후 `ros2 topic hz /turtle1/pose2d` 가 약 62 Hz 를 출력했음
- [ ] (MATLAB 이 있으면) `W02_2_goto_offline` 결과 위치오차가 `tol_d = 0.05` 보다 작았음
- [ ] (MATLAB 이 있으면) `W02_3_goto_turtlesim` 으로 turtlesim 거북이가 `(9, 2)` 에서 위를 보고 멈췄음

다음 주 준비

- [ ] 저장공간 20 GB 이상 확보

과제 2 — 가상 IMU 표본화 실험

- **제출 기한**: 3주차 수업 전
- **제출**: 코드 + 그래프 + 짧은 분석

① 가상 IMU 발행 노드 (`imu_sim`)

- 토픽 `/sim/imu` , 타입 `sensor_msgs/Imu` , **50 Hz** 발행
- `angular_velocity.z` 에 아래 신호를 실을 것

$$\omega_z(t) = 0.5\sin(2\pi \cdot 0.5\,t) + 0.2\sin(2\pi \cdot 9\,t) + n(t) + 0.01$$

항	뜻
$0.5\sin(2\pi \cdot 0.5\,t)$	저주파 0.5 Hz — 실제 선회 운동
$0.2\sin(2\pi \cdot 9\,t)$	고주파 9 Hz — 파랑 진동
$n(t)$	가우시안 잡음, 표준편차 0.02
0.01	상수 바이어스

- `header.stamp` 를 현재 시각으로 정확히 채울 것
- `header.frame_id = "imu_link"`

② 재표본화 구독 노드 (`imu_resampler`)

- `/sim/imu` 를 구독 → **10 Hz** 로 `/sim/imu_10hz` 에 재발행
- **두 방식을 모두 구현**하고 비교
  - (a) **단순 데시메이션** — 5개 중 1개만 그대로 통과
  - (b) **이동평균 후 데시메이션** — 최근 5개 평균을 낸 뒤 통과

③ 결과 그래프

- 한 그래프에 세 신호를 겹쳐 그릴 것
  - 원본 50 Hz
  - (a) 단순 데시메이션 10 Hz
  - (b) 이동평균 후 10 Hz
- 축 라벨 · 단위 · 범례 필수

④ 분석 (5~10줄)

1. 9 Hz 성분을 10 Hz로 표본화하면 **몇 Hz로 둔갑**하는가? 그래프에서 확인되는가?
2. (a)와 (b) 중 어느 쪽이 나은가? **왜** 그런가?
3. 이 문제가 실제 배의 헤딩 제어기에서 어떤 증상으로 나타나겠는가?

> ✎ **힌트**
> 에일리어싱된 주파수 $f_a = |f - k f_s| \le f_s/2$ 를 만족하는 값 (f 신호 주파수, f_s 표본화 주파수, k 정수)

평가 기준

항목	배점
두 노드 정상 동작 (`ros2 topic hz` 로 주기 확인 가능)	40%
그래프 품질 (축 라벨, 범례, 단위)	20%
분석의 정확성 — 특히 1번	30%
코드 가독성 (함수 분리, 주석)	10%

막혔을 때

> ℹ **아래 메시지는 전부 기준 환경에서 재현해 받은 실제 출력임**
> 검색할 때는 대괄호 안의 시각을 빼고 메시지 본문만 넣음

ROS 2 실행

화면에 나오는 것	원인	해결
`bash: ros2: command not found`	환경 미적용	`source /opt/ros/humble/setup.bash`
`Package 'usv_basics' not found`	**워크스페이스 미적용**	`source ~/capstone_ws/install/setup.bash`
`Package 'usv_basic' not found`	패키지 **이름 오타**	`ros2 pkg list | grep usv` 로 확인
`No executable found`	`setup.py` 의 `console_scripts` 미등록 또는 실행파일 이름 오타	`ros2 pkg executables usv_basics` 로 확인
코드를 고쳤는데 반영 안 됨	`--symlink-install` 없이 빌드	옵션 붙여 재빌드
`colcon build` 에서 `setup.py` 오류	`entry_points` 오타	`패키지명.파일명:main` 형식 확인
`ros2 node list` 실행 시 `WARNING: Be aware that are nodes in the graph that share an exact name, this can have unintended side effects.` (Humble 원문 그대로 — `there` 가 빠져 있음, 2026-09-19 실측)	**같은 노드를 두 번 실행**	하나를 `Ctrl + C` 로 끔. `ros2 topic hz` 값이 배로 뛰는 것이 신호
`sudo rosdep init` 이 실패	이미 초기화됨	무시하고 `rosdep update` 진행

토픽이 안 보이거나 데이터가 안 온다

증상	원인	해결
직접 띄운 토픽만 목록에 없음	**`ROS_DOMAIN_ID` 불일치**	두 터미널에서 `echo $ROS_DOMAIN_ID` 비교
옆자리 학생의 토픽이 보임	Domain ID 가 같음	팀 번호로 변경 후 `source ~/.bashrc`
토픽은 보이는데 데이터를 못 받음	**QoS 불일치**	`ros2 topic info <토픽> --verbose` 로 `Reliability` 비교
`incompatible QoS ... Last incompatible policy: RELIABILITY`	위와 같음	§2-9 참조

중계 노드 · Simulink

화면에 나오는 것	원인	해결
`turtlesim/Pose은(는) 인식할 수 없는 메시지 유형입니다`	MATLAB 에 turtlesim 메시지가 없음	중계 노드를 켜고 `/turtle1/pose2d` 를 구독 (§2-10)
`ModuleNotFoundError: No module named 'turtlesim'`	우분투에 turtlesim 이 없음	`sudo apt install -y ros-humble-turtlesim`
중계 노드가 첫 줄만 찍고 `first relay` 가 안 나옴	turtlesim 이 안 떠 있음	`ros2 run turtlesim turtlesim_node`
MATLAB 에서 토픽은 보이는데 Simulink 값이 계속 0	우분투 노드가 멈춘 채 **목록에만 남음** · Domain ID 불일치	우분투에서 `ros2 topic hz /turtle1/pose2d` 확인 → 노드 재실행 · `W02_setup.m` 의 `ros_domain_id` 확인
`ros2("topic","list")` 에는 보이는데 모델 값이 0	Simulink ROS 네트워크 프로필의 Domain ID 가 다름	**시뮬레이션 → ROS 네트워크** 에서 우분투 값과 맞춤 (§2-11 1단계 5)
MATLAB `ros2("topic","list")` 에 `/parameter_events` · `/rosout` 만 보임	WSL 미러 네트워크 미설정	`.wslconfig` 에 `networkingMode=mirrored` → `wsl --shutdown` (WSL-VRX-환경구축 §8.2)
거북이가 안 움직이고 결과가 `위치오차 9.2195`	위와 같음 — 한 번도 자세를 못 받아 `valid = 0` 으로 정지	위와 같음. 9.2195 는 (0,0) 에서 (9,2) 까지 거리
`먼저 W02_setup 을 실행하십시오.`	설정 변수가 없음	`W02_setup` 실행 후 다시
모델을 고치다 망가뜨림	—	`W02_setup` → `build_w02_models`

- Domain ID 차이는 이렇게 눈으로 확인할 수 있음 (실측)

```
ROS_DOMAIN_ID=7  ros2 topic list      # /usv_chatter 가 보인다
ROS_DOMAIN_ID=42 ros2 topic list      # /parameter_events 와 /rosout 만 보인다
```

VS Code

증상	원인	해결
왼쪽 아래에 `WSL: Ubuntu-22.04` 가 없음	Windows 쪽 폴더를 연 것	`><` → **Connect to WSL** 후 폴더를 다시 엶
파이썬 자동완성이 안 됨	Python 확장이 **Windows 쪽에만** 설치됨	WSL 에 연결한 상태에서 **Install in WSL** 클릭
위쪽에 노란 `Restricted Mode` 띠	폴더를 아직 신뢰하지 않음	**Manage → Trust**
저장했는데 빌드에 반영 안 됨	저장이 안 됐거나 Windows 쪽 파일	탭 이름의 **흰 점(●)** 이 사라졌는지 확인
`/bin/bash^M: bad interpreter`	파일이 **CRLF** 로 저장됨	상태 표시줄의 `CRLF` 를 눌러 `LF` 로 바꾼 뒤 저장
우분투에서 `code .` · `explorer.exe .` 가 `Exec format error`	WSL 의 **Windows 프로그램 연동 (interop)** 이 꺼져 있음	아래 "interop 켜기" 참조

interop 켜기 — 우분투에서 Windows 프로그램을 못 부를 때

- 증상 확인

```
cat /proc/sys/fs/binfmt_misc/WSLInterop
```

- 정상이면 첫 줄이 `enabled`. **파일이 없다고 나오면 꺼진 것**임
- 고치는 법 — `/etc/wsl.conf` 에 아래 세 줄을 넣음

```
sudo tee -a /etc/wsl.conf > /dev/null <<'EOF'

[interop]
enabled=true
appendWindowsPath=true
EOF
```

- **PowerShell** 에서 WSL 을 완전히 껐다 켬

```
wsl --shutdown
```

- 다시 우분투를 열고 확인

```
cat /proc/sys/fs/binfmt_misc/WSLInterop
```

```
enabled
interpreter /init
```

```
which code
```

```
/mnt/c/Users/<사용자>/AppData/Local/Programs/Microsoft VS Code/bin/code
```

> **그래도 안 되면 Windows 쪽에서 직접 엶**
>
> ```
> code --remote wsl+Ubuntu-22.04 /home/사용자명/capstone_ws
> ```

GUI 도구

증상	원인	해결
`rqt_graph` · `rviz2` 창이 안 뜸	WSLg 문제	1주차 2-4절 `xeyes` 확인으로 복귀
`ros2 node list` 에 **죽은 노드**가 계속 보임	ROS 2 데몬이 옛 정보를 들고 있음	`ros2 daemon stop` 후 다시 명령 실행
`turtlesim` 창이 안 뜸	WSLg 문제	위와 같음
`turtle_teleop_key` 로 방향키를 눌러도 안 움직임	**그 터미널에 포커스가 없음**	터미널 창을 클릭한 뒤 방향키
`ros2 param set` 했는데 화면이 그대로	turtlesim 은 다시 그릴 때 반영	`ros2 service call /clear std_srvs/srv/Empty {}`
`rqt_graph` 가 비어 있음	자동 갱신 안 됨	왼쪽 위 **파란 회전 화살표**
`QStandardPaths: wrong permissions on runtime directory`	WSLg 의 알려진 경고	**무시해도 됨.** 창은 정상적으로 뜸
Topic Monitor 가 `not monitored`	체크박스를 안 켬	토픽 왼쪽 **체크박스**를 켬
RViz2 에 `Global Status: Warn` / `No tf data`	TF 발행 노드가 없음	**이번 주차에는 정상**. 3주차에 해결됨

apt · 권한

화면에 나오는 것	원인	해결
`E: Could not open lock file /var/lib/dpkg/lock-frontend - open (13: Permission denied)`	`sudo` 없이 `apt` 실행	명령 앞에 `sudo` 를 붙임
`E: Unable to acquire the dpkg frontend lock ... are you root?`	위와 같음	위와 같음

참고 자료

이번 주차 실습 코드

- `usv_basics` **패키지** — https://github.com/wkyouncnu/usv_basics
  - 이번 주차에 만드는 노드 4개와 중계 노드 `turtle_pose_relay` 가 그대로 들어 있음
  - 받는 법은 §2-6 "저장소에서 받기"
  - 라이선스 Apache-2.0. 자유롭게 고쳐 써도 됨
- **Simulink 모델** — 강의자료 `10-주차별-강의자료/W02_simulink/` (§2-11)
  - MATLAB ROS 2 사용자 메시지 — https://www.mathworks.com/help/ros/ref/ros2genmsg.html

공식 문서 (북마크 권장)

- ROS 2 Humble 문서 — https://docs.ros.org/en/humble/
- 초급 튜토리얼 (CLI) — https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html
- **turtlesim 으로 시작하기** — https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html
- **서비스 이해하기** — https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html

- **파라미터 이해하기** — https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Parameters/Understanding-ROS2-Parameters.html
- **액션 이해하기** — https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html
- 초급 튜토리얼 (노드 작성) — https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html
- **QoS 설정 상세** — https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html
- 표준 메시지 정의 — https://github.com/ros2/common_interfaces

볼트 내 문서

- QoS가-어긋나면-에러없이-끊긴다 — 이번 주차 실습의 배경
- WSL-VRX-환경구축 §3 — ROS 2 설치 절차

다음 주 예고

- **3주차 — Gazebo Garden + VRX 설치, 좌표계와 TF2**
- 할 일
  - **WAM-V 를 시뮬레이션 환경에 배치** — 시드니 레가타 해역에 WAM-V 스폰
  - 직접 조종해서 움직여 봄
  - ENU / NED 좌표계와 쿼터니언 정리
- 준비물
  - 이번 주차에 작성한 ROS 2 환경
  - **저장공간 20 GB 이상** (VRX 빌드에 필요)
  - VRX 빌드는 기준 환경에서 약 1~2분 (3주차 실측). 패키지 내려받기가 더 오래 걸리므로 충전기 지참